1/11/2026 - 4:08:08 p.m.

Actúa como asesor de finanzas corporativas y programador en R. Soy estudiante de la Maestría en Finanzas de EGADE (curso Productos Derivados y Administración de Riesgos Corporativos, profesor Dr. Raymundo Díaz Robles). Mi equipo (Equipo 6) entrega el 2do avance del proyecto final: un dashboard, código en R Markdown (Rmd) y su HTML, con información de LSEG Workspace. Empresa: Walmart (WMT.O). Situación problema: caso Molson Coors Brewing Co., "Valuing a Convertible Bond" (nota de enseñanza adjunta). La empresa debe refinanciar deuda barata que vence en un entorno de tasas más alto y cubrir el riesgo que eso genera. El caso valúa un convertible como bono recto más una opción call (Black-Scholes), calcula el cupón para emitir a la par, y diseña un call spread (compra de calls a 25% de prima, venta de warrants a 60%) y un T-lock. Archivos que adjunto: precios diarios de WMT (CSV), estructura de deuda y bonos (Excel de LSEG), calificaciones crediticias, CDS, 10-K y 10-Q de Walmart, y la nota de enseñanza del caso. Quiero que me ayudes en pasos. En este mensaje no escribas código todavía: resume en 8 líneas lo que entendiste del caso y de mis datos, y dime qué faltaría para un análisis completo. Usa español, sin guiones largos.



ChatGPT (GPT 5.2) •1/11/2026 - 4:08:08 p.m.

Entendí que tu proyecto usa a Walmart (WMT.O) como empresa objetivo para replicar la lógica del caso Molson Coors sobre refinanciar deuda barata que vence en un entorno de tasas más altas y cubrir el riesgo de tasa que eso implica.
El caso base valúa un bono convertible descomponiéndolo en bono recto más una opción call sobre la acción, usando Black-Scholes para la parte de equity opcional.
Con esa valuación, se determina el cupón que permitiría emitir el convertible a la par dadas las condiciones de mercado.
Además, el caso diseña una estrategia de call spread para ajustar la dilución y el costo, comprando calls hasta 25% de prima y vendiendo warrants hasta 60%.
En paralelo, se plantea un T-lock para fijar o asegurar el nivel de tasas antes de la emisión o el refinanciamiento.
Tus datos incluyen precios diarios de WMT (CSV), estructura de deuda y bonos desde LSEG (Excel), calificaciones crediticias y CDS, además de 10-K y 10-Q para contexto y supuestos.
El entregable es un dashboard con Rmd y HTML que muestre inputs de mercado, valuación del convertible, sensibilidad y el diseño de coberturas (call spread y T-lock).
Faltaría para un análisis completo: términos específicos del convertible hipotético o a emitir (monto, vencimiento, call/put, conversión, dividend protection), curva libre de riesgo y/o OIS, supuestos de volatilidad (histórica vs implícita) y dividend yield, estimación de spread de crédito y tasa de recuperación, convención day count y frecuencia de cupones, además de parámetros exactos del T-lock y de las opciones (maturities, strikes) y una fecha de valuación consistente.

[DONE]

1/11/2026 - 4:09:13 p.m.



1/11/2026 - 4:09:14 p.m.

Te anexo todos los archivos

ChatGPT (GPT 5.2) •1/11/2026 - 4:09:14 p.m.

Recibido: adjuntaste 3 Excel, 1 archivo .xls, 1 CSV (precios diarios), y 2 PDFs (probablemente 10-K/10-Q y la nota del caso).

Siguiente paso propuesto, para avanzar ordenados sin escribir código todavía:

1. Definir la fecha de valuación del dashboard (por ejemplo, el último día del CSV o una fecha del caso).
2. Identificar qué archivo contiene qué: estructura de deuda y bonos, ratings/CDS, y términos relevantes para armar el “bono recto” y el spread de crédito.
3. Acordar el instrumento objetivo: un convertible hipotético (términos a definir) o alguno comparable que aparezca en LSEG.

Dime cuál de estos dos caminos quieren seguir: A) Replicar el caso tal cual con un convertible “nuevo” para Walmart (definimos términos y calculamos cupón a la par).
B) Tomar un bono específico de Walmart de la base LSEG y construir la narrativa de refinanciamiento y coberturas alrededor de ese vencimiento.

En tu próximo mensaje, confirma A o B y la fecha de valuación que usarán. Con eso ya puedo revisar la estructura de los archivos y mapear las variables que necesita el dashboard.

code execution file 'documento__684b792a-c69c-4937-8ee2-a8f780c6669e__14d15b9b-2c31-41ec-80c6-01c40c07fb56.xls' could not be uploaded: 422, message='Unprocessable Entity', url='http://52.171.224.55/files'
Thinking

[DONE]

1/11/2026 - 4:09:40 p.m.

Propón la estructura de un dashboard para el segundo avance, en este orden: portada con indicadores clave (precio, capitalización, deuda, deuda neta/EBITDA, calificación, CDS), resumen de riesgos (tasa, cambiario, crédito) con los instrumentos de cobertura propuestos, perfil financiero, estructura y vencimientos de la deuda, composición por divisa, curvas de rendimiento, pared de refinanciamiento, calificaciones y CDS, volatilidad con GARCH y convertible con call spread. Para cada sección dime qué gráfica o tabla usar y qué pregunta responde. Que sea original y no copie otros dashboards.



ChatGPT (GPT 5.2) •1/11/2026 - 4:09:40 p.m.

1) Portada: indicadores clave

Qué mostrar

- Tarjetas KPI: Precio WMT, retorno YTD, volatilidad 30d, capitalización, deuda total, caja, deuda neta, deuda neta/EBITDA, rating (S&P/Moody’s/Fitch), CDS 5Y (bp) y cambio semanal. **Gráfica/tabla**
- Mini sparkline por KPI (precio, CDS, yield proxy, FX si aplica) + tabla compacta “Market snapshot” con fecha de corte y fuentes. **Pregunta que responde**
- Dónde está hoy Walmart en mercado y balance, y qué tan “caro” está el riesgo de crédito.

---

2) Resumen de riesgos: tasa, cambiario, crédito y coberturas propuestas

Qué mostrar

- Matriz de riesgos con exposición, drivers, métrica, severidad y hedge recomendado.
- Instrumentos: T-lock para tasa, cross-currency swap o forwards para FX (si hay deuda no USD o cashflows relevantes), y CDS o ajuste de spread/estructura para crédito. **Gráfica/tabla**
- Heatmap 3x3 (riesgo vs impacto/probabilidad) + tabla "Hedge playbook" con: objetivo, instrumento, nocional sugerido, tenor, trigger, pros/contras. **Pregunta que responde**
- Cuáles son los riesgos financieros relevantes para el refinanciamiento y cómo se cubren de forma accionable.

---

3) Perfil financiero

Qué mostrar

- Evolución trimestral/anual (10-K/10-Q): ingresos, EBITDA, margen EBITDA, FCF, cobertura de intereses, leverage. **Gráfica/tabla**
- Área apilada para fuentes/uso de efectivo (CFO, Capex, CFF) + líneas para Debt/EBITDA y EBITDA/Intereses. **Pregunta que responde**
- Qué tan robusta es la capacidad de pago y qué espacio tiene para absorber mayor costo financiero.

---

4) Estructura y vencimientos de la deuda

Qué mostrar

- Inventario de bonos/préstamos: monto, cupón, vencimiento, tipo (fijo/variable), moneda, seniority. **Gráfica/tabla**
- "Timeline ladder" tipo Gantt por instrumento (maturity ladder) + tabla detallada filtrable por tipo/moneda/tasa. **Pregunta que responde**
- Qué vence, cuándo vence, y qué parte del pasivo está expuesta a reinversión/refinanciamiento.

---

5) Composición por divisa

Qué mostrar

- Deuda por moneda y, si se puede, por moneda neta tras coberturas existentes. **Gráfica/tabla**
- Donut o barras horizontales por moneda + waterfall "Bruta → Cobertura → Neta" (si hay datos de swaps/forwards). **Pregunta que responde**
- Cuánto riesgo cambiario existe por descalce entre deuda y flujos/caja en moneda.

---

6) Curvas de rendimiento (tasas base)

Qué mostrar

- Curva UST (o SOFR/OIS si aplica) por tenor; comparación vs una fecha histórica (inicio del año) para ilustrar el "shock de tasas". **Gráfica/tabla**
- Curva interactiva con dos líneas (hoy vs referencia) + tabla de puntos clave (2Y, 5Y, 10Y, 30Y) y cambios en bp. **Pregunta que responde**
- Cómo ha cambiado el costo base del dinero y por qué el refinanciamiento ahora es más caro.

---

7) Pared de refinanciamiento (refinancing wall)

Qué mostrar

- Vencimientos agregados por año y costo estimado al refinanciar: cupón actual vs cupón estimado "hoy". **Gráfica/tabla**
- Barras por año (monto que vence) con una segunda dimensión: color por tipo fijo/variable o por moneda; línea encima con "costo estimado". **Pregunta que responde**
- En qué años se concentra el riesgo de refinanciamiento y cuánto podría subir el gasto por intereses.

---

8) Calificaciones y CDS

Qué mostrar

- Ratings actuales y outlook, historial si está disponible; CDS 5Y y su relación con spreads. **Gráfica/tabla**
- Termómetro de rating (escalas) + serie de tiempo CDS con anotaciones (resultados, guidance, shocks de mercado).
- Scatter "CDS vs precio/volatilidad" o "CDS vs yield UST 5Y" para ver sensibilidad. **Pregunta que responde**

- Qué tan percibido está el riesgo de crédito y si el mercado lo está revaluando recientemente.

---

9) Volatilidad con GARCH (insumo para el convertible)

Qué mostrar

- Volatilidad histórica vs volatilidad condicional estimada (GARCH), y distribución de retornos. **Gráfica/tabla**
- Serie de tiempo: retorno diario y banda de volatilidad GARCH (annualizada) + gráfico de QQ/histograma para colas.
- Tabla de parámetros del modelo y métricas (AIC/BIC, persistencia). **Pregunta que responde**
- Qué volatilidad "usable" para valuación es consistente con los datos y qué tan inestable es el riesgo de equity.

---

10) Convertible + call spread (valuación y diseño)

Qué mostrar

- Descomposición: valor bono recto + valor opción (BS) = valor convertible; cupón necesario para emitir a la par.
- Diseño del call spread: strike de conversión, call comprada (25% premium), warrant vendido (60%); payoff y efectos en dilución/costo. **Gráfica/tabla**
- Waterfall de valuación (straight bond, option, total, par gap) + tabla de supuestos (S0, r, q, sigma GARCH/otra, spread crédito, T).
- Gráfica de payoff al vencimiento: acciones vs valor para inversionista, empresa y neto con call spread; mapa de sensibilidad (sigma vs spread vs cupón par). **Pregunta que responde**
- Qué cupón y estructura hacen viable el refinanciamiento con un convertible, y cómo el call spread modifica costo/dilución y el perfil de riesgo.

Si quieres, en el siguiente paso puedo convertir esto en un índice de pestañas para R Markdown con nombres exactos, filtros y las variables que cada sección consume de tus archivos (LSEG, CSV, PDFs), para asegurar trazabilidad y consistencia de fecha de valuación.

[DONE]

1/11/2026 - 4:10:02 p.m.

Con mi Excel de bonos escribe código en R (dplyr, plotly) que calcule: deuda vigente, valor de mercado, cupón y YTM ponderados, vida promedio, escalera de vencimientos, composición por divisa y la deuda con cupón menor a 3% que vence antes de 2032. Comenta cada bloque y explica qué significa cada resultado en una frase.

ChatGPT (GPT 5.2) •1/11/2026 - 4:10:02 p.m.

A continuación te dejo un script en R que funciona como plantilla robusta para tu Excel de bonos de LSEG. Como no veo los nombres exactos de columnas de tu archivo aquí, incluí un bloque de "mapeo" donde solo ajustarás los nombres para que coincidan con tu Excel (una vez mapeado, lo demás corre igual). Usa `dplyr` y `plotly` como pediste.

> Supuesto general: cada fila es un instrumento de deuda vigente (bond/loan) con monto en circulación, moneda, cupón, YTM, fecha de vencimiento y, si existe, valor de mercado o precio.

```
# ==========================
# 0) Paquetes y lectura
# ==========================
library(readxl)
library(dplyr)
library(lubridate)
library(stringr)
library(plotly)
library(scales)

# Ajusta ruta y hoja
path_xlsx <- "bonos_walmart.xlsx"
sheet_name <- 1

raw <- read_excel(path_xlsx, sheet = sheet_name)

# Comentario: Leemos el Excel tal cual para inspeccionar columnas.
# Significado del resultado (1 frase): Este objeto contiene la tabla base con todos los instrumentos de deuda reportados por LSEG.
# ==========================
# 1) Estandarizar nombres y mapear columnas clave
# ==========================
names(raw) <- names(raw) |>
  str_trim() |>
  str_replace_all("\\s+", " ") |>
  str_replace_all("[^A-Za-z0-9_]", "") |>
  tolower()

# ---- MAPEO: ajusta estos nombres a los de tu archivo ----
# Reemplaza el lado derecho por el nombre real de tu columna estandarizada.
col_map <- list(
  instrument_id   = "isin",              # o "ric", "ticker", "id"
  instrument_name = "issue_name",        # o "security_name"
  currency        = "currency",          # ej. "usd", "eur"
```

```
  outstanding     = "amount_outstanding",# monto en circulación (idealmente
en moneda del bono)
  maturity_date   = "maturity_date",     # fecha vencimiento
  coupon          = "coupon",            # cupón anual (%)
  ytm             = "ytm",               # yield to maturity (%)
  market_value    = "market_value",      # valor de mercado (si existe) en
moneda del bono o en USD
  price           = "price"              # opcional, si no hay market value
)

# Función auxiliar para tomar una columna si existe; si no, crea NA.
get_col <- function(df, nm) if (nm %in% names(df)) df[[nm]] else NA

bonds <- raw %>%
  transmute(
    instrument_id   = get_col(., col_map$instrument_id),
    instrument_name = get_col(., col_map$instrument_name),
    currency        = toupper(as.character(get_col(., col_map$currency))),
    outstanding_raw = get_col(., col_map$outstanding),
    maturity_date   = as.Date(get_col(., col_map$maturity_date)),
    coupon_raw      = get_col(., col_map$coupon),
    ytm_raw         = get_col(., col_map$ytm),
    market_value_raw= get_col(., col_map$market_value),
    price_raw       = get_col(., col_map$price)
  )

# Comentario: Creamos un dataset "limpio" con solo las variables necesarias
para métricas y gráficas.
# Significado del resultado (1 frase): Esta tabla estandarizada es la base
para calcular ponderaciones, plazos, vencimientos y cortes por cupón/fecha.
# ========================
# 2) Limpieza numérica (%, montos) y fecha de valuación
# ========================
# Fecha de corte: usa la que defina tu equipo; por default, hoy.
val_date <- Sys.Date()

to_num <- function(x) {
  # Convierte "3.25%", "3,25", "$1,000" a número
  x <- as.character(x)
  x <- str_replace_all(x, ",", "")
  x <- str_replace_all(x, "\\$", "")
  x <- str_replace_all(x, "%", "")
  suppressWarnings(as.numeric(x))
}

bonds2 <- bonds %>%
  mutate(
    outstanding = to_num(outstanding_raw),
    coupon      = to_num(coupon_raw) / 100, # a tasa decimal
    ytm         = to_num(ytm_raw) / 100,    # a tasa decimal
    market_value= to_num(market_value_raw),
    price       = to_num(price_raw),
    years_to_maturity = as.numeric(difftime(maturity_date, val_date, units
= "days"))/365.25
  ) %>%
  # nos quedamos con deuda vigente (vencimientos futuros) y montos
positivos
  filter(!is.na(maturity_date), years_to_maturity > 0, !is.na(outstanding),
outstanding > 0)
```

```r
# Si no hay market_value pero hay price, aproximamos MV = outstanding *
price/100 (bonds típicamente cotizan sobre 100)
bonds2 <- bonds2 %>%
  mutate(
    market_value = ifelse(is.na(market_value) & !is.na(price),
                          outstanding * (price/100),
                          market_value)
  )

# Comentario: Convertimos variables a numéricas comparables y calculamos
años al vencimiento.
# Significado del resultado (1 frase): Ahora puedes comparar instrumentos
en la misma escala (tasas decimales, montos numéricos) y medir su "plazo"
en años.
# ==========================
# 3) Deuda vigente y valor de mercado (totales)
# ==========================
totals <- bonds2 %>%
  summarise(
    deuda_vigente = sum(outstanding, na.rm = TRUE),
    valor_mercado = sum(market_value, na.rm = TRUE),
    n_instrumentos= n()
  )

totals

# Comentario: Sumamos el monto en circulación y el valor de mercado
aproximado/observado.
# Significado del resultado (1 frase): Esto cuantifica el tamaño del pasivo
financiero vigente y su valoración agregada en mercado (si hay precios/MV
disponibles).
# ==========================
# 4) Cupón y YTM ponderados (por outstanding y por market value)
# ==========================
weighted_metrics <- bonds2 %>%
  summarise(
    coupon w outstanding = weighted.mean(coupon, w = outstanding, na.rm =
TRUE),
    ytm_w_outstanding    = weighted.mean(ytm,    w = outstanding, na.rm =
TRUE),
    coupon w mktvalue    = weighted.mean(coupon, w = market value, na.rm =
TRUE),
    ytm_w_mktvalue       = weighted.mean(ytm,    w = market_value, na.rm =
TRUE)
  )

weighted_metrics

# Comentario: Calculamos promedios ponderados para reflejar el "costo"
relevante a nivel portafolio.
# Significado del resultado (1 frase): Estos promedios representan el cupón
contractual promedio y el costo de financiamiento implícito (YTM) del total
de la deuda, ponderados por tamaño.
# ==========================
# 5) Vida promedio (Weighted Average Life, WAL)
# ==========================
wal <- bonds2 %>%
  summarise(
    wal_outstanding = sum(outstanding * years_to_maturity, na.rm = TRUE) /
sum(outstanding, na.rm = TRUE),
```

```
    wal_mktvalue    = sum(market_value * years_to_maturity, na.rm = TRUE) /
sum(market_value, na.rm = TRUE)
  )

wal

# Comentario: WAL mide el "plazo promedio" de la deuda, ponderando por
tamaño.
# Significado del resultado (1 frase): La vida promedio indica cuántos
años, en promedio ponderado, faltan para que la deuda actual venza y deba
refinanciarse o pagarse.
# =========================
# 6) Escalera de vencimientos (por año)
# =========================
maturity_ladder <- bonds2 %>%
  mutate(maturity_year = year(maturity_date)) %>%
  group_by(maturity_year) %>%
  summarise(
    amount_outstanding = sum(outstanding, na.rm = TRUE),
    market_value       = sum(market_value, na.rm = TRUE),
    n_issues           = n(),
    .groups = "drop"
  ) %>%
  arrange(maturity_year)

maturity_ladder

# Gráfica interactiva: monto que vence por año
p_ladder <- plot_ly(
  maturity_ladder,
  x = ~maturity_year,
  y = ~amount_outstanding,
  type = "bar",
  hovertemplate = paste(
    "Año: %{x}<br>",
    "Vence (outstanding): %{y:,.0f}<br>",
    "N instrumentos: %{customdata}<extra></extra>"
  ),
  customdata = ~n_issues
) %>%
  layout(
    title = "Escalera de vencimientos (monto en circulación)",
    xaxis = list(title = "Año de vencimiento"),
    yaxis = list(title = "Monto", tickformat = ",.0f")
  )

p_ladder

# Comentario: Agregamos vencimientos por año para identificar
concentraciones (refinancing wall).
# Significado del resultado (1 frase): La escalera muestra en qué años se
concentra el riesgo de refinanciamiento por volumen de vencimientos.
# =========================
# 7) Composición por divisa
# =========================
fx comp <- bonds2 %>%
  group_by(currency) %>%
  summarise(
    amount_outstanding = sum(outstanding, na.rm = TRUE),
    market_value       = sum(market_value, na.rm = TRUE),
```

```
    .groups = "drop"
  ) %>%
  mutate(
    pct_outstanding = amount_outstanding / sum(amount_outstanding, na.rm =
TRUE)
  ) %>%
  arrange(desc(amount_outstanding))

fx_comp

p_fx <- plot_ly(
  fx_comp,
  labels = ~currency,
  values = ~amount_outstanding,
  type = "pie",
  textinfo = "label+percent",
  hovertemplate = paste(
    "Moneda: %{label}<br>",
    "Outstanding: %{value:,.0f}<extra></extra>"
  )
) %>%
  layout(title = "Composición de deuda por divisa (outstanding)")

p_fx

# Comentario: Resumimos la deuda por moneda para dimensionar riesgo
cambiario por pasivo.
# Significado del resultado (1 frase): Esta composición indica qué
porcentaje de la deuda está expuesta a moneda distinta a la funcional (si
aplica) y, por tanto, al riesgo FX.
# ========================
# 8) Deuda con cupón < 3% que vence antes de 2032
# ========================
cheap_debt <- bonds2 %>%
  filter(
    !is.na(coupon),
    coupon < 0.03,
    maturity_date < as.Date("2032-01-01")
  ) %>%
  arrange(maturity_date) %>%
  mutate(
    maturity_year = year(maturity_date)
  )

cheap debt summary <- cheap debt %>%
  summarise(
    n_instrumentos = n(),
    outstanding_total = sum(outstanding, na.rm = TRUE),
    market_value_total= sum(market_value, na.rm = TRUE),
    coupon w = weighted.mean(coupon, w = outstanding, na.rm = TRUE),
    ytm w    = weighted.mean(ytm,    w = outstanding, na.rm = TRUE)
  )

cheap_debt_summary
cheap debt

# Gráfica: pared de refinanciamiento para “deuda barata”
cheap_ladder <- cheap_debt %>%
  group_by(maturity_year) %>%
  summarise(outstanding = sum(outstanding, na.rm = TRUE), .groups = "drop")
```

```
p_cheap <- plot_ly(
  cheap ladder, x = ~maturity year, y = ~outstanding, type = "bar",
  hovertemplate = "Año: %{x}<br>Outstanding: %{y:,.0f}<extra></extra>"
) %>%
  layout(
    title = "Vencimientos de deuda con cupón < 3% (antes de 2032)",
    xaxis = list(title = "Año"),
    yaxis = list(title = "Monto", tickformat = ",.0f")
  )

p_cheap

# Comentario: Filtramos instrumentos "baratos" que vencerán pronto para
cuantificar el problema de refinanciar en tasas altas.
# Significado del resultado (1 frase): Este bloque identifica cuánta deuda
con cupones muy bajos se tendrá que reemplazar antes de 2032, que es justo
donde el aumento de tasas golpea el costo financiero.
```

Para que esto corra a la primera en tu archivo

1. Pega aquí los **nombres reales de columnas** (tal como quedan tras el bloque de estandarización) o comparte un screenshot de `names(raw)`.
2. Confirma si `outstanding` está en **USD** o en **moneda del bono** y si `market_value` viene ya en USD. Eso define si la "composición por divisa" y el "valor de mercado total" se deben convertir (si tienes FX).

Si me mandas `names(raw)` y 3 filas de ejemplo (sin datos sensibles), te lo devuelvo ya 100% alineado a tu Excel específico.

Thinking
[DONE]

1/11/2026 - 4:11:11 p.m.

Con los precios diarios de WMT calcula rendimientos logarítmicos y estima un GARCH(1,1) por máxima verosimilitud en R (con optim, sin paquetes externos). Muestra omega, alfa, beta, persistencia, varianza de largo plazo, volatilidad de hoy y volatilidad promedio esperada a 1, 3 y 6 años. Haz una gráfica de la volatilidad condicional y una del pronóstico por horizonte. Explica cómo se interpreta cada parámetro.

ChatGPT (GPT 5.2) •1/11/2026 - 4:11:11 p.m.

```
# ============================================================
# GARCH(1,1) para WMT con MLE usando optim (sin paquetes externos)
# - Lee precios diarios desde CSV
```

```r
# - Calcula rendimientos logarítmicos
# - Estima omega, alpha, beta por máxima verosimilitud (Normal)
# - Reporta persistencia, varianza de largo plazo, vol hoy
# - Pronostica volatilidad promedio esperada a 1, 3 y 6 años
# - Grafica volatilidad condicional y pronóstico por horizonte
# ============================================================

# ------------------------------
# 1) Cargar CSV y preparar returns
# ------------------------------
path_csv <- "WMT_prices.csv"   # <-- cambia el nombre/ruta a tu CSV

dat <- read.csv(path_csv, stringsAsFactors = FALSE)

# Detectar columna de fecha y precio de forma flexible
name_lc <- tolower(names(dat))

date_col <- names(dat)[match(TRUE, name_lc %in% c("date","data","fecha"))]
if (is.na(date_col)) date_col <- names(dat)[1]

price_candidates <- c("adj
close","adj_close","adjusted_close","adjustedclose",
                      "close","px_last","last","price","prc")
price_col <- names(dat)[match(TRUE, name_lc %in% gsub(" ", "_",
price_candidates) |
                                        name_lc %in% price_candidates)]
if (is.na(price_col)) {
  stop("No pude identificar la columna de precio. Renómbrala como Close o
Adj Close.")
}

# Parseo de fecha
dat[[date_col]] <- as.Date(dat[[date_col]])
dat <- dat[order(dat[[date_col]]), ]

P <- as.numeric(dat[[price_col]])
D <- dat[[date_col]]

# Rendimientos logarítmicos
r <- diff(log(P))
r <- r[is.finite(r)]
dates_r <- D[-1][is.finite(diff(log(P)))]

# (Opcional) centrar returns en media cero para GARCH con media ~ 0
r <- r - mean(r, na.rm = TRUE)

# Significado: r_t mide el cambio porcentual continuo diario, y es la serie
típica para modelar volatilidad con GARCH.


# ------------------------------
# 2) Funciones GARCH(1,1) y log-verosimilitud
#    Modelo: h_t = omega + alpha * r_{t-1}^2 + beta * h_{t-1}
#    r_t | F_{t-1} ~ N(0, h_t)
# ------------------------------

# Transformación para garantizar restricciones:
# omega > 0, alpha > 0, beta > 0 y alpha+beta < 1
par_to_garch <- function(theta) {
  omega <- exp(theta[1])
```

```r
  a <- exp(theta[2])
  b <- exp(theta[3])
  alpha <- a / (1 + a + b)
  beta  <- b / (1 + a + b)
  list(omega = omega, alpha = alpha, beta = beta)
}

garch filter <- function(r, omega, alpha, beta) {
  n <- length(r)
  h <- numeric(n)
  # Inicialización (varianza muestral como arranque estable)
  h[1] <- var(r, na.rm = TRUE)
  for (t in 2:n) {
    h[t] <- omega + alpha * r[t-1]^2 + beta * h[t-1]
  }
  h
}

negloglik_garch <- function(theta, r) {
  p <- par_to_garch(theta)
  omega <- p$omega; alpha <- p$alpha; beta <- p$beta

  # Filtrar varianzas condicionales
  h <- garch_filter(r, omega, alpha, beta)

  # Penalizar si por problemas numéricos aparece h<=0
  if (any(!is.finite(h)) || any(h <= 0)) return(1e12)

  # Loglik Normal
  ll <- -0.5 * sum(log(2*pi) + log(h) + (r^2)/h)
  return(-ll)  # optim minimiza
}

# Significado: la verosimilitud elige omega, alpha, beta para que los
retornos observados sean "más probables"
# bajo una normal con varianza que cambia en el tiempo según GARCH.


# ------------------------------
# 3) Estimación por optim
# ------------------------------
# Valores iniciales razonables (en espacio theta)
theta0 <- c(log(var(r)), log(0.05), log(0.90))

fit <- optim(
  par = theta0,
  fn  = negloglik_garch,
  r   = r,
  method = "BFGS",
  control = list(maxit = 2000, reltol = 1e-10)
)

p_hat <- par_to_garch(fit$par)
omega_hat <- p_hat$omega
alpha hat <- p hat$alpha
beta hat  <- p hat$beta
persist   <- alpha_hat + beta_hat

# Filtrado con parámetros estimados
h_hat <- garch_filter(r, omega_hat, alpha_hat, beta_hat)
```

```r
# Varianza de largo plazo (si persistencia < 1)
h long <- omega hat / (1 - persist)
vol_long_daily <- sqrt(h_long)
vol_today_daily <- sqrt(tail(h_hat, 1))

# Annualización (aprox 252 días hábiles)
ann <- sqrt(252)
vol_long_ann <- vol_long_daily * ann
vol_today_ann <- vol_today_daily * ann

# Significado: h_t es la varianza condicional diaria; su raíz es la
volatilidad diaria;
# al multiplicar por sqrt(252) obtienes volatilidad anualizada aproximada.


# ------------------------------
# 4) Pronóstico de volatilidad por horizonte (1, 3 y 6 años)
#    Fórmula: E[h_{t+k}] = h_bar + persist^k * (h_t - h_bar)
#    Volatilidad promedio esperada en el horizonte H:
#       sqrt( mean_{k=1..H} E[h_{t+k}] * 252 )
# ------------------------------
forecast_h <- function(h_t, h_bar, persist, H) {
  k <- 1:H
  h_bar + (persist^k) * (h_t - h_bar)
}

h_t <- tail(h_hat, 1)
h_bar <- h_long

years_vec <- c(1, 3, 6)
H_vec <- as.integer(round(years_vec * 252))

fc_list <- lapply(H_vec, function(H) forecast_h(h_t, h_bar, persist, H))

vol avg ann <- sapply(fc list, function(hk) sqrt(mean(hk) * 252))
vol end ann <- sapply(fc list, function(hk) sqrt(tail(hk, 1) * 252))

# Significado: vol_avg_ann resume la volatilidad anualizada "promedio" que
esperas experimentar
# a lo largo de 1, 3 o 6 años, mientras vol end ann es la volatilidad
esperada al final del horizonte.


# ------------------------------
# 5) Imprimir resultados solicitados
# ------------------------------
cat("\n===== Estimación GARCH(1,1) (Normal) =====\n")
cat(sprintf("omega (ω): %0.6e\n", omega_hat))
cat(sprintf("alpha (α): %0.6f\n", alpha hat))
cat(sprintf("beta  (β): %0.6f\n", beta hat))
cat(sprintf("persistencia (α+β): %0.6f\n", persist))
cat(sprintf("varianza largo plazo h¯ = ω/(1-α-β): %0.6e\n", h_long))
cat(sprintf("volatilidad hoy (anualizada): %0.2f%%\n", 100 *
vol today ann))
cat(sprintf("volatilidad largo plazo (anualizada): %0.2f%%\n", 100 *
vol_long_ann))

cat("\n===== Pronóstico volatilidad (anualizada) =====\n")
out_fc <- data.frame(
```

```r
  horizonte_anios = years_vec,
  vol_promedio_esperada = vol_avg_ann,
  vol al final horizonte = vol end ann
)
print(transform(out_fc,
                vol_promedio_esperada = sprintf("%0.2f%%",
100*vol_promedio_esperada),
                vol al final horizonte = sprintf("%0.2f%%",
100*vol_al_final_horizonte)),
      row.names = FALSE)

# Cada resultado en una frase:
# omega: nivel base de varianza "nueva" que entra cada día.
# alpha: sensibilidad de la varianza a shocks recientes (retornos grandes
ayer elevan h hoy).
# beta: persistencia/inercia de la volatilidad (si fue alta ayer, tiende a
seguir alta).
# alpha+beta: qué tan rápido se disipa un shock; cercano a 1 implica alta
persistencia.
# h¯: varianza a la que converge el proceso si no hay shocks; define la vol
de largo plazo.
# vol hoy: estimación anualizada del riesgo de corto plazo con la info más
reciente.
# vol promedio por horizonte: riesgo anualizado esperado "en promedio"
durante el periodo.


# ------------------------------
# 6) Gráfica 1: Volatilidad condicional histórica (anualizada)
# ------------------------------
vol_hat_ann <- sqrt(h_hat) * ann

plot(dates_r, vol_hat_ann,
     type = "l", lwd = 2, col = "steelblue",
     xlab = "Fecha", ylab = "Volatilidad condicional anualizada",
     main = "WMT: Volatilidad condicional GARCH(1,1) (anualizada)")
grid()

# Significado: la línea muestra cómo el modelo evalúa el riesgo
(volatilidad) día a día,
# subiendo en episodios de shocks y decayendo gradualmente por la
persistencia.


# ------------------------------
# 7) Gráfica 2: Pronóstico por horizonte (promedio y final)
# ------------------------------
plot(years_vec, 100*vol_avg_ann,
     type = "b", pch = 19, lwd = 2, col = "darkgreen",
     ylim = range(c(100*vol avg ann, 100*vol end ann)),
     xlab = "Horizonte (años)", ylab = "Volatilidad anualizada (%)",
     main = "Pronóstico de volatilidad: promedio en el horizonte y al
final")
lines(years_vec, 100*vol_end_ann, type = "b", pch = 17, lwd = 2, col =
"firebrick")
legend("topright",
       legend = c("Vol promedio esperada en el horizonte", "Vol esperada al
final del horizonte"),
       col = c("darkgreen", "firebrick"), lwd = 2, pch = c(19, 17), bty =
"n")
```

```
grid()

# Significado: compara cómo el riesgo esperado se "mean-revierte" hacia la
volatilidad de largo plazo
# conforme aumenta el horizonte.
```

Cómo interpretar cada parámetro (en lenguaje financiero)

- **ω (omega)**: es la "inyección" base de varianza cada día; determina el piso del riesgo cuando no hay shocks.
- **α (alpha)**: captura el efecto ARCH, cuánto reaccionan las varianzas a noticias recientes; retornos extremos ayer elevan la volatilidad hoy.
- **β (beta)**: captura la inercia; si la volatilidad fue alta, tiende a permanecer alta por varios días o semanas.
- **α + β (persistencia)**: qué tan lento decae un shock; si está cerca de 1, la volatilidad tarda más en volver a su nivel normal.
- **Varianza de largo plazo $\bar{h} = \omega/(1-\alpha-\beta)$**: el punto de reversión a la media del proceso; su raíz (annualizada) es la **volatilidad "normal"** de largo plazo.

Si me dices el nombre exacto de la columna de precio en tu CSV (Close vs Adj Close) y la ventana temporal del archivo, te ajusto el bloque de lectura para que quede idéntico al formato de LSEG/Workspace que estés usando en el dashboard.

[DONE]

1/11/2026 - 4:11:48 p.m.

Replica la nota de enseñanza del caso Molson Coors con datos de Walmart: precio de la acción, plazo de 6 años, nominal de 1,000, tasa libre de riesgo y tasa de descuento del bono, y la volatilidad del GARCH. Calcula con primas de 15%, 25%, 40% y 60%: precio de conversión, acciones por bono, valor de la opción con Black-Scholes, con árbol binomial CRR de 500 pasos (europeo y americano) y con Monte Carlo, valor del bono recto y cupón para emitir a la par. Después calcula el costo del call spread (comprar calls a 25%, vender warrants a 60%) y compáralo con el ahorro de cupón.

ChatGPT (GPT 5.2) •1/11/2026 - 4:11:48 p.m.

Abajo tienes un guion completo en R para replicar la lógica del caso (bono recto + call) con Walmart. Está diseñado para que tú solo pegues los **inputs** (S0, r libre de riesgo, tasa de descuento del bono, volatilidad GARCH anualizada) y corra todas las primas: 15%, 25%, 40%, 60%, y luego el **call spread 25%-60%** y la comparación contra el **ahorro de cupón**.

Supuestos explícitos (ajústalos si tu nota usa otros):

1. Convertible sin cupones intermedios especiales, sin call/put, sin dividend protection.
2. Cupón anual, pagos anuales, nominal 1,000, plazo 6 años.
3. La parte "opción" se valúa sobre **acción** con strike = precio de conversión.
4. La opción se multiplica por "acciones por bono" = nominal / precio de conversión.
5. Para el bono recto se descuenta a una **tasa de descuento del bono** (yield requerido).
6. Para la opción en BS/CRR/MC se usa r (libre de riesgo), volatilidad = sigma GARCH anualizada, y dividend yield q (si lo quieres, si no q = 0).

Código R (base R) para cálculos y comparación

```
# ============================================================
# Inputs (PEGA tus números aquí)
# ============================================================
S0    <- 0      # precio spot de WMT hoy (USD), por ejemplo 82.35
Tmat  <- 6      # años
F     <- 1000   # nominal por bono

r     <- 0      # tasa libre de riesgo anual continua (o efectiva; ver nota
abajo)
y_b   <- 0      # tasa de descuento del bono (yield requerido del straight
bond)

sigma <- 0      # volatilidad anualizada (GARCH) en decimal, ej. 0.22
q     <- 0      # dividend yield anual continuo (si no lo modelas, deja 0)

premia <- c(0.15, 0.25, 0.40, 0.60)

# Convenciones:
# - Usaremos capitalización continua para la opción (BS) y descuento exp(-
rT)
# - Para el bono recto, usaremos descuento exp(-y_b * t) (continua)
# Si tu caso usa tasas efectivas discretas, te lo adapto fácil.
# ============================================================


# ============================================================
# 1) Black-Scholes call (dividend yield q)
# ============================================================
bs_call <- function(S, K, r, q, sigma, T) {
  if (T <= 0) return(max(S - K, 0))
  d1 <- (log(S/K) + (r - q + 0.5*sigma^2)*T) / (sigma*sqrt(T))
  d2 <- d1 - sigma*sqrt(T)
  S*exp(-q*T)*pnorm(d1) - K*exp(-r*T)*pnorm(d2)
}


# ============================================================
# 2) Binomial CRR (N pasos): europeo y americano
```

```r
# ============================================================
crr_call <- function(S, K, r, q, sigma, T, N = 500, american = FALSE) {
  dt <- T / N
  u  <- exp(sigma * sqrt(dt))
  d  <- 1 / u
  disc <- exp(-r * dt)
  p <- (exp((r - q)*dt) - d) / (u - d)

  # precios al vencimiento
  j <- 0:N
  ST <- S * (u^j) * (d^(N - j))
  V  <- pmax(ST - K, 0)

  # backward induction
  for (i in (N-1):0) {
    j <- 0:i
    Sij <- S * (u^j) * (d^(i - j))
    V <- disc * (p * V[2:(i+2)] + (1 - p) * V[1:(i+1)])
    if (american) {
      V <- pmax(V, Sij - K)  # ejercicio temprano
    }
  }
  as.numeric(V[1])
}


# ============================================================
# 3) Monte Carlo (GBM) europeo
#    (Para americano, MC requiere Longstaff-Schwartz; aquí no lo implemento
#     porque el usuario pidió "Monte Carlo" sin especificar americano.)
# ============================================================
mc_call <- function(S, K, r, q, sigma, T, n_paths = 200000, seed = 123) {
  set.seed(seed)
  Z <- rnorm(n_paths)
  ST <- S * exp((r - q - 0.5*sigma^2)*T + sigma*sqrt(T)*Z)
  payoff <- pmax(ST - K, 0)
  exp(-r*T) * mean(payoff)
}


# ============================================================
# 4) Valor del bono recto y cupón para emitir a la par
#    Bono con cupón anual c (tasa anual sobre nominal), maturity Tmat.
#    PV_bond(c) = F*c*sum_{t=1..T} exp(-y_b t) + F*exp(-y_b T)
#    Precio convertible = PV bond(c) + Nshares * OptionValuePerShare
#    Resolver c tal que precio = F (emitir a la par)
# ============================================================
pv_straight_bond <- function(coupon_rate, F, y_b, Tmat) {
  t <- 1:Tmat
  df <- exp(-y_b * t)
  pv_coupons <- F * coupon_rate * sum(df)
  pv_principal <- F * exp(-y_b * Tmat)
  pv_coupons + pv_principal
}

coupon_to_par <- function(F, y_b, Tmat, option_per_bond) {
  t <- 1:Tmat
  A <- sum(exp(-y_b * t))        # PV factor para cupones
  B <- exp(-y_b * Tmat)          # PV factor principal
  # F*c*A + F*B + option = F  =>  c = (F - option - F*B)/(F*A)
```

```r
  (F - option_per_bond - F*B) / (F*A)
}


# ============================================================
# 5) Correr las 4 primas y consolidar resultados
# ============================================================
calc_row <- function(prem) {
  Kconv <- S0 * (1 + prem)      # precio de conversión
  nShares <- F / Kconv          # acciones por bono

  # Opción por acción (3 métodos)
  c_bs  <- bs_call(S0, Kconv, r, q, sigma, Tmat)
  c_eur <- crr_call(S0, Kconv, r, q, sigma, Tmat, N = 500, american =
FALSE)
  c_am  <- crr_call(S0, Kconv, r, q, sigma, Tmat, N = 500, american = TRUE)
  c_mc  <- mc_call(S0, Kconv, r, q, sigma, Tmat, n_paths = 200000, seed =
123)

  opt_bond_bs  <- nShares * c_bs
  opt_bond_eur <- nShares * c_eur
  opt_bond_am  <- nShares * c_am
  opt_bond_mc  <- nShares * c_mc

  # Cupón par usando la opción BS (puedes cambiar a CRR/MC si quieres)
  cpn_par_bs <- coupon_to_par(F, y_b, Tmat, opt_bond_bs)
  pv_bond_at_cpn <- pv_straight_bond(cpn_par_bs, F, y_b, Tmat)

  data.frame(
    premium = prem,
    conversion_price = Kconv,
    shares_per_bond = nShares,

    call_BS_per_share  = c_bs,
    call_CRR_EUR_per_share = c_eur,
    call_CRR_AM_per_share  = c_am,
    call_MC_per_share  = c_mc,

    option_value_BS_per_bond  = opt_bond_bs,
    option_value_CRR_EUR_per_bond = opt_bond_eur,
    option_value_CRR_AM_per_bond  = opt_bond_am,
    option_value_MC_per_bond  = opt_bond_mc,

    straight_bond_value_at_par_coupon = pv_bond_at_cpn,
    coupon_rate_par_(BS_option) = cpn_par_bs
  )
}

res <- do.call(rbind, lapply(premia, calc_row))

# Valor de bono recto "si fuera straight a la par" (sin opción)
coupon_par_straight <- coupon_to_par(F, y_b, Tmat, option_per_bond = 0)

res
coupon_par_straight
```

Call spread 25%-60% y comparación con “ahorro de cupón”

En el caso, el call spread se monta típicamente para “recomprar” parte de la opción que vendiste vía conversión y limitar dilución. Aquí lo hacemos coherente con el **convertible con prima 25%** (strike K1 = S0·1.25) y vendemos warrants a **60%** (strike K2 = S0·1.60) con el mismo vencimiento.

```
# ============================================================
# 6) Call spread: comprar calls K1=25%, vender warrants K2=60%
# ============================================================
K1 <- S0 * 1.25
K2 <- S0 * 1.60
nShares_25 <- F / K1   # acciones por bono usando el strike de conversión
del 25%

# Valuar ambas patas con Black-Scholes (puedes cambiar a CRR/MC igual)
call_K1 <- bs_call(S0, K1, r, q, sigma, Tmat)
call_K2 <- bs_call(S0, K2, r, q, sigma, Tmat)

cost_call_spread_per_bond <- nShares_25 * (call_K1 - call_K2)

cost call spread per bond


# ============================================================
# 7) Comparar costo del call spread vs ahorro de cupón del convertible
#    "Ahorro de cupón" = cupón straight par - cupón convertible par (con
prima 25%)
# ============================================================
coupon_conv_25 <- res$coupon_rate_par.BS_option.[res$premium == 0.25]
coupon_saving_rate <- coupon_par_straight - coupon_conv_25

annual_interest_saving <- F * coupon_saving_rate  # USD por año por bono

# Comparación simple:
comparison <- data.frame(
  coupon par straight = coupon par straight,
  coupon_par_convertible_25 = coupon_conv_25,
  coupon_saving_rate = coupon_saving_rate,
  annual_interest_saving_usd = annual_interest_saving,
  call_spread_cost_usd = cost_call_spread_per_bond,
  years_to_breakeven_simple = cost_call_spread_per_bond /
annual_interest_saving
)

comparison
```

Qué significa cada salida, en una frase (para tu Rmd)

- **Precio de conversión**: el precio al cual el inversionista “paga” implícitamente por acción al convertir el bono.

- **Acciones por bono**: cuántas acciones recibe el tenedor si convierte, y por ende la escala de la opción embebida.
- **Valor de la opción (BS/CRR/MC)**: valor teórico de la convexidad de equity incluida en el convertible bajo distintos métodos numéricos.
- **Valor del bono recto**: PV del flujo de cupones y principal descontados al yield requerido del crédito de Walmart.
- **Cupón para emitir a la par**: el cupón que hace que bono recto + opción sumen 1,000 hoy.
- **Costo del call spread**: costo neto de comprar convexidad entre 25% y 60% (reduce dilución arriba de K1, cede upside arriba de K2).
- **Ahorro de cupón**: cuánto baja el cupón del convertible vs un straight bond par, reflejando que parte del "pago" al inversionista se hace con la opción.
- **Breakeven simple**: cuántos años de ahorro de intereses necesitarías para "recuperar" el costo del call spread (medida rápida, no PV).

Dos datos que necesito para que quede 100% "como la nota"

1. ¿La nota usa **cupón anual o semestral** y descuento **discreto o continuo**? (ahora lo puse anual y continuo).
2. Para "tasa de descuento del bono", ¿quieres usar un **yield observado de un bono de Walmart** (de tu Excel) o construirlo como **r + spread** (por CDS/rating)?

Si me confirmas esas dos convenciones y me pasas tus valores de `S0`, `r`, `y_b` y `sigma` (del GARCH), te devuelvo el mismo script ajustado para que los números coincidan exactamente con su formato de caso.

Thinking
[DONE]

1/11/2026 - 4:15:18 p.m.

Calcula cuánto cuesta refinanciar la deuda de bajo cupón que vence antes de 2032 a la tasa actual de emisión, la sensibilidad si la tasa cambia -100 a +150 pb, y el DV01 de un T-lock a 10 años sobre ese monto. Explica cuándo conviene un T-lock, un swap con inicio futuro o una swaption.



ChatGPT (GPT 5.2) •1/11/2026 - 4:15:18 p.m.

1) Costo de refinanciar la deuda “barata” que vence antes de 2032

Definiciones (para que el cálculo sea consistente):

- NN = monto total a refinanciar (outstanding) de instrumentos con cupón < 3% y vencimiento < 2032.
- cviejoc_{\text{viejo}} = cupón promedio ponderado (por outstanding) de esa deuda.
- ynuevoy_{\text{nuevo}} = “tasa actual de emisión” para Walmart al plazo relevante (puede ser el yield de un bono Walmart cercano al plazo, o r+spreadr + \text{spread}).
- El costo incremental anual aproximado (en intereses) por refinanciar es: ΔIntere´s anual≈N·(ynuevo−cviejo) \Delta \text{Interés anual} \approx N \cdot (y_{\text{nuevo}} - c_{\text{viejo}}) Si quieres “costo total” en PV, entonces descuentas los diferenciales de intereses por el plazo esperado de la nueva emisión.

Sensibilidad a cambios de tasa (-100 a +150 pb):
ΔIntere´s anual(Δy)≈N·((ynuevo+Δy)−cviejo) \Delta \text{Interés anual}(\Delta y) \approx N \cdot \big((y_{\text{nuevo}}+\Delta y) - c_{\text{viejo}}\big) donde Δy∈{−1.00%,−0.50%,0,+0.50%,+1.00%,+1.50%}\Delta y \in \{-1.00\%, -0.50\%, 0, +0.50\%, +1.00\%, +1.50\%\}.

---

2) Código R (usando tu objeto de “deuda barata” del paso anterior)

Supongo que ya tienes `cheap_debt` con columnas `outstanding` y `coupon` (en decimal), como en el bloque que te pasé antes. También supongo que vas a introducir una sola tasa “de emisión actual” ynuevoy_{\text{nuevo}} (en decimal).

```
# ========================
# Inputs de mercado (ajusta)
# ========================
y_new <- 0.055  # tasa actual de emisión (ej. 5.50% en decimal)

# ========================
# 1) Agregados de la deuda barata
# ========================
N_refi <- sum(cheap_debt$outstanding, na.rm = TRUE)
c_old  <- weighted.mean(cheap_debt$coupon, w = cheap_debt$outstanding,
na.rm = TRUE)

# Costo incremental anual (base)
delta_int_base <- N_refi * (y_new - c_old)

data.frame(
  monto_refinanciar = N_refi,
  cupon_prom_viejo  = c_old,
  tasa_nueva        = y_new,
  costo incremental anual = delta int base
)

# Significado (1 frase):
```

```r
# Esto estima cuánto sube el gasto anual por intereses si reemplazas esa
deuda barata por deuda nueva al nivel actual de tasas.
# ========================
# 2) Sensibilidad por movimientos de tasa (-100 a +150 pb)
# ========================
shifts_bp <- c(-100, -50, 0, 50, 100, 150)
shifts    <- shifts_bp / 10000

sens <- data.frame(shift_bp = shifts_bp) |>
  transform(
    tasa_nueva_shift = y_new + shifts,
    costo_incremental_anual = N_refi * ((y_new + shifts) - c_old),
    cambio vs base = costo incremental anual - delta int base
  )

sens

# Significado (1 frase):
# Esta tabla te dice cuánto cambia el costo anual de refinanciar si la tasa
de emisión sube o baja respecto al nivel actual.
```

---

3) DV01 de un T-lock a 10 años sobre ese monto

Un **T-lock** (o "Treasury lock") busca cubrir el componente de tasa base (Treasury) antes de emitir. Su **DV01** te dice cuánto gana o pierde el hedge por un movimiento de 1 pb.

A falta de una curva completa, una aproximación muy usada es: DV01≈N·PVBP10y DV01 \approx N \cdot PVBP_{10y} donde PVBP10yPVBP_{10y} (también llamado PV01 por unidad de nocional) es el valor presente de 1 pb en un instrumento 10 años. En swaps USD 10y suele andar cerca de 7.5 a 8.5 USD por 1 millón por pb (depende de la curva), pero lo correcto es **calcularlo con la tasa/par swap** que uses.

Aproximación simple (si no tienes curva):

- Si tomas PVBP10y≈8.0PVBP_{10y} \approx 8.0 USD por **1MM** por **1bp**, entonces: DV01≈8.0×N1,000,000 DV01 \approx 8.0 \times \frac{N}{1{,}000{,}000}

```r
# ========================
# 3) DV01 T-lock 10y (aprox)
# ========================
pvbp_10y_per_1mm <- 8.0  # USD por 1MM por 1bp (aprox; ideal: calcularlo
con curva)

dv01 tlock <- pvbp 10y per 1mm * (N refi / 1e6)  # USD por 1bp

dv01_tlock

# Significado (1 frase):
# DV01 te indica cuántos dólares por cada 1 pb se movería el valor del
hedge; te sirve para dimensionar la cobertura del monto a refinanciar.
```

Si quieres que el DV01 sea “de libro” (construido desde curva), necesito que me confirmes si usarán UST par, SOFR OIS o par swap USD (y las tasas por tenor).

---

4) ¿Cuándo conviene T-lock vs forward-starting swap vs swaption?

T-lock (Treasury lock)

Conviene cuando:

- Ya decidiste que **sí emitirás** (alta certeza), pero te preocupa que suba la **tasa base** (Treasury) antes del pricing.
- Tu spread de crédito podría moverse, pero quieres al menos fijar la parte “risk-free”.
  Pros: simple, efectivo para bloquear el nivel de UST.
  Contras: no cubre el spread de crédito; si las tasas bajan, el hedge puede generar pérdida (aunque tu emisión sería más barata).

Swap con inicio futuro (forward-starting swap)

Conviene cuando:

- Tienes fecha probable de refinanciamiento y quieres fijar la **tasa swap** (más cercana al costo de financiación floating/hedge) desde hoy para un periodo futuro.
- Vas a emitir deuda variable o planeas “sintetizar” fijo/variable. **Pros:** fija tasa con precisión para el periodo; muy alineado a gestión ALM.
  Contras: te obliga más (si no emites o cambias monto/fecha, hay basis/ineficiencia); puede requerir CSA/márgenes.

Swaption (opción sobre swap)

Conviene cuando:

- La emisión/refinanciamiento es **incierta** (timing o monto) o quieres protegerte de subidas sin renunciar al beneficio de bajadas.
- Quieres un “seguro” contra tasas al alza, no un lock rígido. **Pros:** asimétrica, limita el peor caso y deja participar si tasas bajan.
  Contras: prima upfront (impacto en PnL/costo), y valoración/contabilidad más exigente.

Para cerrarlo con números “de Walmart” en tu dashboard

Dime cuál vas a usar como **tasa actual de emisión** ynuevoy_{\text{nuevo}}:

1. el **YTM ponderado** de bonos Walmart nuevos (por ejemplo 8–12 años) desde tu Excel, o
2. ynuevo=r10y+spready_{\text{nuevo}} = r_{10y} + \text{spread} (de CDS/rating/bonos comparables).

Y confirma si el monto NN lo quieres en **USD equivalente** (si hay deuda en otras divisas) para que el costo de refinanciar sea comparable y el DV01 tenga sentido.

[DONE]

1/11/2026 - 4:22:10 p.m.

Adjunto el Rmd y el dashboard HTML finales de mi equipo. Quiero que el código que me des produzca un dashboard igual a ese HTML. Usa mis archivos como especificación exacta y no inventes otro diseño. 1. Primero lista las secciones del HTML en orden, con el tipo de gráfica o tabla de cada una y qué datos usa. Confírmame que entendiste antes de escribir código. 2. Después escribe el código en R Markdown, una sección por mensaje, empezando por la portada con los 8 indicadores, el resumen de riesgos y la navegación. 3. Usa los mismos nombres de variables, fórmulas y supuestos que mi Rmd (GARCH(1,1) por máxima verosimilitud, Black-Scholes, árbol CRR de 500 pasos, Monte Carlo de 400,000 trayectorias, tasa libre de riesgo 4.86%, descuento del bono 5.06%). 4. Al final de cada sección, dame los números que debe mostrar para que yo los compare (por ejemplo, cupón del convertible con 25% de prima: 1.13%; volatilidad GARCH a 6 años: 23.2%). 5. Si algo de mi diseño no se puede replicar en R Markdown con plotly, dime qué cambia en lugar de sustituirlo en silencio. Escribe en español y sin guiones largos.



ChatGPT (GPT 5.2) •1/11/2026 - 4:22:11 p.m.

Para poder listar las secciones del HTML en el orden exacto y con el tipo de visualización y datos que usa cada una, necesito ver el **HTML** (o al menos su estructura) y el **Rmd** que lo genera. En este chat no me aparecen disponibles esos adjuntos todavía, así que no puedo extraer la especificación ni garantizar que el dashboard quede idéntico.

Lo que necesito que me compartas, en cualquiera de estas opciones (la que te sea más fácil):

1. Vuelve a adjuntar aquí el **.Rmd** y el **.html** (y si el HTML depende de carpetas como `www/` o `data/`, adjunta también esas).
2. Si el HTML es muy pesado, pega:
    - El contenido del Rmd (o al menos el YAML + el “toc” de secciones y los chunks donde se crean las gráficas).

    - El índice de secciones del HTML (capturas o el encabezado con los títulos y tabs).
3. Confirma también los **nombres exactos** de tus archivos de datos y en qué carpeta relativa están respecto al Rmd (por ejemplo `data/bonds.xlsx`, `data/wmt_prices.csv`, etc.).

Cuando los tenga, haré exactamente lo que pides:

1. Listaré las secciones del HTML en orden, indicando para cada una:
    - Tipo de gráfica/tabla (plotly, DT, valueBox, etc.)
    - Qué datos usa (archivo y columnas/objetos del Rmd)
    - Qué cálculo/objeto alimenta el output

      Y cerraré con una confirmación explícita de que entendí antes de escribir código.
2. Luego escribiré el R Markdown, una sección por mensaje, empezando por:
    - Portada con 8 indicadores
    - Resumen de riesgos
    - Navegación idéntica
3. Respetaré tus mismos supuestos y nombres: GARCH(1,1) por MLE con optim, Black-Scholes, CRR 500 pasos, Monte Carlo 400,000 trayectorias, rf 4.86%, descuento del bono 5.06%.
4. Al final de cada sección te pondré los números esperados para validación contra tu HTML.
5. Si algo no se puede replicar tal cual con R Markdown y plotly, te diré exactamente qué cambia y por qué, sin sustituirlo en silencio.

En tu siguiente mensaje, adjunta de nuevo el Rmd y el HTML o pega el contenido mínimo descrito arriba, y dime si el dashboard está hecho con `flexdashboard`, `shinydashboard`, `quarto`, o Rmd "base" (esto afecta cómo se replica la navegación).

[DONE]

1/11/2026 - 4:26:12 p.m.



1/11/2026 - 4:26:13 p.m.

Te dejo la vista del dashboard

ChatGPT (GPT 5.2) •1/11/2026 - 4:26:13 p.m.

Plan:

1. Abrir el PDF adjunto en el sandbox y extraer texto/estructura (títulos, encabezados, tablas) página por página.
2. Identificar el orden de secciones del dashboard y, para cada una, inferir el tipo de visual (KPI, tabla, serie, barras, etc.) a partir de lo que aparece en la vista.
3. Armar una lista ordenada: sección, visual, y qué dataset/inputs debería estar usando (precios WMT, bonos LSEG, CDS/ratings, supuestos de tasas/vol).
4. Confirmarte el entendimiento y señalar cualquier ambigüedad que requiera tu Rmd original o assets adicionales antes de escribir código.

Con la vista en PDF puedo reconstruir con bastante precisión la estructura y contenidos del dashboard, pero no puedo garantizar que sea idéntico al HTML a nivel de temas, tipografías, espaciados y widgets específicos sin ver el HTML o el Rmd original. Aun así, sí puedo listar las secciones y sus visualizaciones tal como aparecen en la vista.

1) Secciones del dashboard en orden, con tipo de gráfica o tabla y qué datos usa

0. Portada, header institucional y corte de datos

- Tipo: tarjetas KPI (indicadores grandes) + texto de metadatos (fecha de corte, fuentes).
- Datos: precio WMT y market cap (LSEG), deuda vigente y número de emisiones (Excel bonos LSEG), deuda neta/EBITDA (fundamentals LSEG o 10-K/10-Q), calificaciones (archivo ratings), CDS 5Y (archivo CDS), volatilidad GARCH (CSV precios WMT), cupón del convertible (módulo de valuación).

1. Resumen: “Tres riesgos, cuatro instrumentos”

- Tipo: tres bloques narrativos (tasa, cambiario, crédito) con métricas puntuales + lista de instrumentos propuestos (T-lock, swap inicio futuro, swaption, convertible+call spread, cross-currency).
- Datos: deuda barata que vence antes de 2032 (Excel bonos), porcentaje deuda en otras divisas y cross-currency swaps (Excel bonos + 10-Q/10-K), calificación y CDS (ratings + CDS), DV01 estimado T-lock (cálculo), cupón del convertible y costo neto del call spread (módulo convertible).

2. Tesis: “El caso Molson Coors, pero doce veces más grande”

- Tipo: bloque de texto + tabla/resumen con 6 métricas agregadas: deuda vigente, valor de mercado, cupón ponderado, YTM ponderado, brecha, vida promedio.
- Datos: Excel de bonos LSEG (outstanding, market value/precio, cupón, YTM, maturity).

3. Perfil financiero de Walmart

- Tipo: KPIs grandes (Ingresos TTM, EBITDA TTM, utilidad TTM, FCF TTM, deuda total, deuda neta/EBITDA) + 2 gráficas de series.
    - Gráfica A: ingresos trimestrales (barras o línea).
    - Gráfica B: deuda total, efectivo y deuda neta (líneas múltiples).
    - Tabla: ingresos por segmento y % del total.
- Datos: Company Fundamentals LSEG (trimestral) y segmentación (LSEG/10-Q).

4. Gasto financiero: "¿cuadran los números?"

- Tipo: tabla de reconciliación (conceptos y MUSD por trimestre) + gráfica de línea comparando "esperado según estructura" vs "gasto por intereses reportado" (último punto marcado).
- Datos: Excel bonos (cupones implícitos), arrendamientos/papel comercial (LSEG/10-Q), serie de gasto por intereses (fundamentals LSEG).

5. Escalera de vencimientos

- Tipo: barras apiladas por año y divisa (USD, GBP, CAD, otras), montos en USD equivalentes.
- Datos: Excel bonos (maturity, outstanding, currency) + tipo de cambio si convierten a USD equivalentes.

6. Composición por divisa

- Tipo: gráfico de composición (parece donut o barras) + cifra principal "$8.67B, 18.9%".
- Datos: Excel bonos (outstanding por moneda), conversión a USD equivalentes si aplica.

7. Curvas soberanas implícitas: Estados Unidos y Canadá

- Tipo: gráfica de curvas por tenor (dos curvas: UST y Gov Canada) y nota metodológica sobre Z-spread.
- Datos: curvas (LSEG rates) y/o spreads implícitos desde bonos; al menos tasas soberanas por tenor.

8. Calificaciones crediticias y riesgo de crédito de mercado

- Tipo: tabla/placas de rating (S&P, Moody's, Fitch) + serie o punto de CDS 5Y (y quizá comparativos).
- Datos: archivo de calificaciones y archivo de CDS (LSEG).

9. Estructura de deuda y derivados según el 10-K

- Tipo: tabla resumen de derivados (cross-currency swaps, swaps fijo-variable, etc.) y cifras clave (por ejemplo diferencial USD/CAD, valor razonable).
- Datos: 10-K/10-Q (extracción manual o tabla ya armada) + quizá Excel LSEG para contexto.

10. Volatilidad de WMT con GARCH(1,1)

- Tipo: panel con KPIs de volatilidad (hoy, largo plazo, persistencia α+β, histórica 1 año, promedio a 6 años) + gráfica de volatilidad condicional en el tiempo.
- Datos: CSV precios diarios WMT (log-returns) y estimación GARCH(1,1) por MLE.

11. Pared de refinanciamiento y bloque de convertible (en la vista aparecen juntos en la misma página final)

- Tipo:
    - "Pared de refinanciamiento": KPIs (monto total, incremento de interés anual, etc.) + gráfica por año de vencimientos relevantes.
    - "Convertible": tabla/mini-panel con: volatilidad, opción por bono (para una prima, típicamente 25%), cupón a la par.
- Datos: Excel bonos (deuda barata <3% antes de 2032), tasas supuestas (rf 4.86%, descuento 5.06%), sigma GARCH, y el módulo de valuación (BS, CRR 500, MC 400,000).

Confirmación de entendimiento antes de escribir código

Entiendo que quieres que replique este dashboard con la misma navegación y secciones, y que los cálculos sigan exactamente tus supuestos: GARCH(1,1) por máxima verosimilitud con optim, Black-Scholes, árbol CRR 500 pasos, Monte Carlo 400,000, rf 4.86% y descuento del bono 5.06%. También entiendo que al final de cada sección te debo dar los números resultantes para compararlos con los que salen en tu dashboard.

Limitación importante: el PDF me deja ver la "vista", pero no me revela si en el HTML original usaron `flexdashboard`, `shinydashboard`, CSS custom, `valueBox`, `DT`, etc. Si me confirmas si el dashboard está hecho con `flexdashboard` (lo más probable por el estilo de secciones numeradas) o me compartes el Rmd, puedo clonar la navegación y el layout con mucha más fidelidad.

Si me confirmas: "sí, es flexdashboard" (o me dices cuál), en el siguiente mensaje empiezo con el R Markdown de la portada con los 8 indicadores, el resumen de riesgos y la navegación, como pediste.

Thinking
code_execution_read_file

Thinking

code_execution_read_file

Thinking

[DONE]

1/11/2026 - 4:28:00 p.m.

Gracias, buen resumen. Correcciones antes de escribir código: 1. NO es flexdashboard. Usa un R Markdown con output html_document (toc, toc_float, self_contained) y un bloque de CSS con results='asis' para las tarjetas (tiles), las notas y la portada. Los gráficos con plotly. 2. Las secciones, en este orden, son 17: (1) Tesis, (2) Perfil financiero, (3) Gasto financiero, (4) Escalera de vencimientos, (5) Composición por divisa, (6) Emisión del 23-oct-2026, (7) Curva de rendimiento, (8) Curvas soberanas implícitas USD y CAD, (9) Pared de refinanciamiento, (10) Recompras ya ejecutadas, (11) Oportunidad de recompra, (12) Sensibilidad del costo de refinanciamiento y T-lock, (13) Calificaciones y CDS, (14) Estructura de deuda y derivados 10-K/10-Q, (15) GARCH de WMT, (16) Convertible estilo Molson Coors con call spread, (17) Qué sigue. Antes de ellas va la portada y el resumen "Tres riesgos, cuatro instrumentos". 3. El convertible es su propia sección: tabla con primas de 15, 25, 40 y 60%, comparación BS contra árbol contra Monte Carlo, cupón a la par, call spread 25%/60% y

---



ChatGPT (GPT 5.2) •1/11/2026 - 4:28:00 p.m.

Perfecto, entendido y gracias por las correcciones. Confirmo lo siguiente antes de escribir código:

1. El dashboard se genera con R Markdown normal, output `html_document` con `toc`, `toc_float` y `self_contained`, y un bloque de CSS inyectado desde un chunk con `results='asis'` para construir portada, tarjetas (tiles) y notas. No es flexdashboard.
2. La estructura del documento es: primero Portada y luego Resumen “Tres riesgos, cuatro instrumentos”, y después exactamente estas 17 secciones en este orden:

1. Tesis
2. Perfil financiero
3. Gasto financiero
4. Escalera de vencimientos
5. Composición por divisa

6. Emisión del 23-oct-2026
7. Curva de rendimiento
8. Curvas soberanas implícitas USD y CAD
9. Pared de refinanciamiento
10. Recompras ya ejecutadas
11. Oportunidad de recompra
12. Sensibilidad del costo de refinanciamiento y T-lock
13. Calificaciones y CDS
14. Estructura de deuda y derivados 10-K/10-Q
15. GARCH de WMT
16. Convertible estilo Molson Coors con call spread
17. Qué sigue

3. El módulo de convertible es una sección independiente y debe incluir:

- Tabla con primas 15, 25, 40 y 60%
- Comparación de valuación de la opción por Black-Scholes vs árbol CRR (500 pasos) vs Monte Carlo (400,000 trayectorias)
- Cupón a la par con tasa libre 4.86% y descuento del bono 5.06%
- Call spread 25%/60% y comparación contra ahorro de cupón

Dos puntos que necesito confirmar para replicar "igual" sin inventar: A) ¿Quieres que en la tabla del convertible la valuación del árbol sea europeo, americano o ambos? En la vista se suele mostrar uno; tú decidirás si es el europeo como en BS, o también americano.
B) Para los 8 indicadores de portada, los valores que aparecen en tu PDF son: precio, capitalización, deuda vigente (notas), deuda neta/EBITDA, calificación, CDS 5 años, volatilidad GARCH, cupón del convertible. ¿Confirmas que esos son los 8 y que los quieres con el mismo formato y textos de pie?

Si confirmas A y B, en el siguiente mensaje empiezo a escribir el Rmd por secciones como pediste, comenzando por:

- YAML (html_document con toc y self_contained)
- Bloque CSS con `results='asis'`
- Portada con las 8 tarjetas, más Resumen "Tres riesgos, cuatro instrumentos" y navegación por toc

Y al final de esa sección te pondré los números objetivo que debe mostrar (por ejemplo, cupón convertible 25%: 1.13%, volatilidad promedio a 6 años: 23.2%, rf 4.86%, descuento bono 5.06%).

[DONE]

1/11/2026 - 4:28:37 p.m.

A) Ambos. En la tabla del convertible muestra tres columnas de valuación de la opción: Black-Scholes, árbol CRR europeo y árbol CRR americano, más una columna de Monte Carlo con el error estándar (por ejemplo "$200.9 ± 0.6"). Con una call sin dividendos el europeo y el americano deben dar el mismo valor, y quiero que eso se vea en la tabla. B) Confirmado, son esos 8, en este orden y con estos textos de pie: 1. Precio WMT: $119.89, "23-oct-2026" 2. Capitalización: $956 B, "7.97 mil M de acciones" 3. Deuda vigente (notas): $45.9 B, "53 emisiones" 4. Deuda neta / EBITDA: 1.01x, "jul-2026" 5. Calificación: AA · Aa2 · AA, "S&P · Moody's · Fitch" 6. CDS 5 años: 26 pb, "29-oct-2026" 7. Volatilidad GARCH: 25.0%, "largo plazo 23.2%" 8. Cupón del convertible: 1.13%, "vs 5.06% bono recto" Se muestran en una cuadrícula de 4 columnas por 2 filas, sobre fondo azul oscuro. Dos reglas más: escribe todo en español, sin guiones largos, y sin lenguaje que suene a texto generado por IA. Empieza con YAML, CSS y portada, y al final dime los números que debe mostrar cada bloque.



ChatGPT (GPT 5.2) •1/11/2026 - 4:28:37 p.m.

```
---
title: "WMT Walmart Inc. | Estructura de deuda y estrategia de cobertura"
subtitle: "Segundo avance de proyecto final | EGADE Business School |
Productos Derivados y Administración de Riesgos Corporativos"
author: "Equipo 6"
date: "Corte de precios: 23-oct-2026"
output:
  html_document:
    toc: true
    toc_float: true
    self_contained: true
    number_sections: true
    theme: null
---
knitr::opts_chunk$set(
  echo = TRUE,
  warning = FALSE,
  message = FALSE
)

# Paquetes base para el documento (plotly se usará en secciones
posteriores)
library(dplyr)
library(plotly)

# Supuestos globales del proyecto (deben ser iguales a tu Rmd)
rf <- 0.0486      # tasa libre de riesgo
y_bono <- 0.0506  # descuento del bono recto

# Números de portada (como en el HTML/PDF; aquí se fijan para clonar el
diseño)
kpi_precio_wmt <- 119.89
kpi_mcap_b     <- 956
kpi_deuda_b    <- 45.9
kpi_netdebt_ebitda <- 1.01
```

```
kpi_rating <- "AA · Aa2 · AA"
kpi_cds_pb <- 26
kpi_vol_garch_hoy <- 0.250
kpi_vol_garch_lp  <- 0.232
kpi_cupon_conv <- 0.0113
cat("
<style>
  :root{
    --wmt-blue:#0b2a4a;
    --wmt-blue-2:#0f355c;
    --tile:#113c66;
    --tile-border:rgba(255,255,255,0.18);
    --text:#0f172a;
    --muted:rgba(255,255,255,0.78);
    --white:#ffffff;
  }

  /* Tipografía y márgenes del documento */
  body{
    font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto,
Arial, sans-serif;
    color: var(--text);
  }
  .main-container{
    max-width: 1100px;
  }

  /* Ocultar el título automático si tu HTML original no lo muestra arriba
del cover */
  h1.title, p.author, p.date{
    display: none;
  }

  /* Portada */
  .cover{
    background: var(--wmt-blue);
    color: var(--white);
    border-radius: 14px;
    padding: 18px 18px 16px 18px;
    margin: 10px 0 18px 0;
  }
  .cover-topline{
    font-size: 12px;
    letter-spacing: 0.6px;
    opacity: 0.95;
    text-transform: uppercase;
    margin-bottom: 10px;
  }
  .cover-title{
    display:flex;
    align-items:flex-end;
    justify-content:space-between;
    gap:16px;
    margin-bottom: 12px;
  }
  .cover-title .left h2{
    margin:0;
    font-size: 22px;
    font-weight: 700;
  }
```

```
.cover-title .left .sub{
  margin-top: 4px;
  font-size: 13px;
  opacity: 0.9;
}
.cover-title .right{
  text-align:right;
  font-size: 12px;
  opacity: 0.9;
  white-space: nowrap;
}

/* Grid KPI 4x2 */
.kpi-grid{
  display:grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 10px;
  margin-top: 10px;
}
.tile{
  background: var(--tile);
  border: 1px solid var(--tile-border);
  border-radius: 12px;
  padding: 12px 12px 10px 12px;
  min-height: 92px;
}
.tile .kpi-label{
  font-size: 12px;
  opacity: 0.95;
  text-transform: uppercase;
  letter-spacing: 0.4px;
  margin-bottom: 6px;
}
.tile .kpi-value{
  font-size: 24px;
  font-weight: 750;
  margin: 0 0 6px 0;
  line-height: 1.05;
}
.tile .kpi-foot{
  font-size: 12px;
  color: var(--muted);
  margin: 0;
}

/* Resumen: tres riesgos */
.summary{
  background: #ffffff;
  border: 1px solid rgba(15,23,42,0.08);
  border-radius: 14px;
  padding: 14px 16px 8px 16px;
  margin: 0 0 18px 0;
}
.summary h3{
  margin: 0 0 10px 0;
  font-size: 16px;
}
.summary p{
  margin: 0 0 10px 0;
  font-size: 13px;
```

```
    color: rgba(15,23,42,0.86);
  }
  .risk-grid{
    display:grid;
    grid-template-columns: repeat(3, 1fr);
    gap: 10px;
    margin-top: 8px;
  }
  .risk-card{
    border: 1px solid rgba(15,23,42,0.08);
    border-radius: 12px;
    padding: 12px;
    background: #fbfdff;
  }
  .risk-card .risk-title{
    font-weight: 800;
    font-size: 12px;
    letter-spacing: 0.4px;
    text-transform: uppercase;
    margin-bottom: 6px;
  }
  .risk-card .risk-level{
    font-size: 12px;
    margin-bottom: 8px;
    color: rgba(15,23,42,0.72);
  }
  .risk-card .risk-text{
    font-size: 13px;
    margin-bottom: 10px;
  }
  .risk-card .risk-tools{
    font-size: 12px;
    color: rgba(15,23,42,0.78);
  }

  /* Responsive */
  @media (max-width: 1100px){
    .kpi-grid{ grid-template-columns: repeat(2, 1fr); }
    .risk-grid{ grid-template-columns: 1fr; }
    .cover-title{ flex-direction: column; align-items:flex-start; }
    .cover-title .right{ text-align:left; }
  }
</style>
")
fmt_money <- function(x, digits = 2) paste0("$", formatC(x, format = "f",
digits = digits, big.mark = ","))
fmt_money_b <- function(x, digits = 1) paste0("$", formatC(x, format = "f",
digits = digits), " B")
fmt_pct <- function(x, digits = 1) paste0(formatC(100*x, format = "f",
digits = digits), "%")
fmt_x <- function(x, digits = 2) paste0(formatC(x, format = "f", digits =
digits), "x")
fmt_pb <- function(x) paste0(formatC(x, format = "f", digits = 0), " pb")

tile <- function(label, value, foot){
  paste0(
    "<div class='tile'>",
      "<div class='kpi-label'>", label, "</div>",
      "<div class='kpi-value'>", value, "</div>",
      "<p class='kpi-foot'>", foot, "</p>",
```

```
    "</div>"
  )
}

cat("<div class='cover'>")
cat("<div class='cover-topline'>EGADE BUSINESS SCHOOL · MAESTRÍA EN
FINANZAS · PRODUCTOS DERIVADOS Y ADMINISTRACIÓN DE RIESGOS
CORPORATIVOS</div>")

cat("<div class='cover-title'>",
      "<div class='left'>",
        "<h2>WMT Walmart Inc.</h2>",
        "<div class='sub'>Estructura de deuda y estrategia de
cobertura</div>",
      "</div>",
      "<div class='right'>",
        "Segundo avance de proyecto final<br/>",
        "Grupo 40 · Equipo 6",
      "</div>",
    "</div>", sep = "")

cat("<div class='kpi-grid'>")
cat(tile("Precio WMT", fmt_money(kpi_precio_wmt, 2), "23-oct-2026"))
cat(tile("Capitalización", fmt_money_b(kpi_mcap_b, 0), "7.97 mil M de
acciones"))
cat(tile("Deuda vigente (notas)", fmt_money_b(kpi_deuda_b, 1), "53
emisiones"))
cat(tile("Deuda neta / EBITDA", fmt_x(kpi_netdebt_ebitda, 2), "jul-2026"))

cat(tile("Calificación", kpi_rating, "S&P · Moody's · Fitch"))
cat(tile("CDS 5 años", fmt_pb(kpi_cds_pb), "29-oct-2026"))
cat(tile("Volatilidad GARCH", fmt_pct(kpi_vol_garch_hoy, 1), paste0("largo
plazo ", fmt_pct(kpi_vol_garch_lp, 1))))
cat(tile("Cupón del convertible", fmt_pct(kpi_cupon_conv, 2), "vs 5.06%
bono recto"))
cat("</div>")

cat("<div style='margin-top:10px; font-size:12px;
color:rgba(255,255,255,0.85);'>",
    "Profesor: Dr. Raymundo Díaz Robles · Bonos y precios al 23 oct 2026 ·
calificaciones, CDS y 10-K/10-Q a 1 nov 2026 · Fuente: LSEG Workspace",
    "</div>")

cat("</div>")
cat("<div class='summary'>")
cat("<h3>Resumen Tres riesgos, cuatro instrumentos</h3>")

cat("<p>Walmart no necesita un convertible para financiarse barato, pero
enfrenta la misma disyuntiva que Molson Coors en 2007: refinanciar deuda de
cupón bajo en un entorno de tasas más alto y cubrir lo que eso genera. Así
queda el diagnóstico.</p>")

cat("<div class='risk-grid'>")

cat(
  "<div class='risk-card'>",
    "<div class='risk-title'>Riesgo de tasa</div>",
    "<div class='risk-level'>Medio-alto</div>",
```

```
    "<div class='risk-text'>$3.95 B con cupón de 1.63% vencen antes de 2032
y hoy se refinanciarían cerca de 5.29%, unos $145 M más al año. El 27% de
la deuda ya es variable.</div>",
    "<div class='risk-tools'><b>T-lock · swap con inicio futuro ·
swaption</b><br/>T-lock a 10 años. DV01 de unos $3.1 M por pb sobre $3.95 B
del nominal.</div>",
  "</div>", sep = ""
)

cat(
  "<div class='risk-card'>",
    "<div class='risk-title'>Riesgo cambiario</div>",
    "<div class='risk-level'>Medio</div>",
    "<div class='risk-text'>18.9% de la deuda está en otras divisas y la
emisión del 23-oct trajo C$5,000 M. Ya hay $5.1 B en cross-currency swaps
(10-Q).</div>",
    "<div class='risk-tools'><b>Cross-currency swap CAD ·
forwards</b><br/>Diferencial USD/CAD de 129 pb en las $4.8 B ya contratadas
(10-K), valor razonable ≈ -$467 M.</div>",
  "</div>", sep = ""
)

cat(
  "<div class='risk-card'>",
    "<div class='risk-title'>Riesgo de crédito</div>",
    "<div class='risk-level'>Muy bajo</div>",
    "<div class='risk-text'>Calificación AA / Aa2 / AA estable y CDS de 26
pb. Acceso fluido a todos los plazos y monedas. Sin necesidad de cobertura
de crédito.</div>",
    "<div class='risk-tools'><b>Convertible + call spread</b><br/>Cupón
1.13% a la par, costo neto del call spread 7.3% (según el módulo de
valuación).</div>",
  "</div>", sep = ""
)

cat("</div>")
cat("</div>")
```

Números que debe mostrar esta sección (para que lo compares con tu HTML)

Portada, tiles 4x2 (en este orden)

1. Precio WMT: $119.89, pie “23-oct-2026”
2. Capitalización: $956 B, pie “7.97 mil M de acciones”
3. Deuda vigente (notas): $45.9 B, pie “53 emisiones”
4. Deuda neta / EBITDA: 1.01x, pie “jul-2026”
5. Calificación: AA · Aa2 · AA, pie “S&P · Moody's · Fitch”
6. CDS 5 años: 26 pb, pie “29-oct-2026”
7. Volatilidad GARCH: 25.0%, pie “largo plazo 23.2%”
8. Cupón del convertible: 1.13%, pie “vs 5.06% bono recto”

Resumen “Tres riesgos, cuatro instrumentos”

- Riesgo de tasa: $3.95 B, 1.63%, 5.29%, $145 M/año, 27% variable, DV01 $3.1 M por pb
- Riesgo cambiario: 18.9%, C$5,000 M, $5.1 B, diferencial 129 pb, valor razonable -$467 M
- Riesgo de crédito: AA/Aa2/AA, CDS 26 pb, cupón convertible 1.13%, costo neto call spread 7.3%

Si me confirmas que esta portada se ve igual en tu HTML (colores, espaciado, tipografías y la cuadrícula), en el siguiente mensaje escribo la Sección 1: Tesis, con los KPIs que aparecen ahí y la tabla resumen tal como la tienen.

[DONE]

1/11/2026 - 4:29:55 p.m.

Revisé tu portada. Antes de la Sección 1, corrige: 1. No fijes los KPI a mano. Calcúlalos con código: el precio desde el CSV de WMT (último cierre), la capitalización como precio por 7,972.4 millones de acciones, la deuda vigente desde el Excel de bonos, el GARCH(1,1) por máxima verosimilitud para la volatilidad, y el cupón del convertible con Black-Scholes y la solución del cupón a la par. Si un dato debe venir de LSEG a mano (calificaciones, CDS), define esos pocos valores en un chunk de "insumos" con la fuente y la fecha. 2. En la portada agrega los tres integrantes con matrícula: Eduardo Ayala Saldaña A01192714, Mariann Antonelli Morales Tobias A01452078, Ulises Salinas López A00814787. "Grupo 40" y "Equipo 6" van arriba de los nombres, en negritas y del mismo tamaño. 3. Corrige el resumen: - Riesgo de tasa: $3.95 B con cupón de 1.63% que vencen antes de 2032 y se refinanciarían cerca de 5.29%, unos $145 M más al año. Instrumentos: T-lock, swap con inicio futuro, swaption. - Riesgo cambiario: 18.9% de la deuda en otras divisas, emisión de C$5,000 M, $5.1 B en cross-currency swaps al 31-jul-2026 (10-Q), diferencial USD/CAD de 129 pb en las curvas implícitas. Instrumento: cross-currency swap. - Riesgo de crédito: AA/Aa2/AA estable y CDS de 26 pb. No necesita cobertura de crédito. - Después de las tres tarjetas, agrega cuatro tarjetas de instrumentos: T-lock a 10 años (DV01 unos $3.1 M por pb sobre $3.95 B), convertible con call spread (cupón 1.13% a la par, costo neto 7.3% del nominal), cross-currency swap CAD, y swaps fijo a variable ($4.8 B, 10-K, valor razonable -$467 M al 31-jul-2026). 4. Usa theme: flatly y quita number_sections. 5. Entrega el código con sus marcas ```{r} para cada chunk.



ChatGPT (GPT 5.2) •1/11/2026 - 4:29:56 p.m.

```
---
title: "WMT Walmart Inc. | Estructura de deuda y estrategia de cobertura"
subtitle: "Segundo avance de proyecto final | EGADE Business School |
Productos Derivados y Administración de Riesgos Corporativos"
author: "Equipo 6"
date: "Corte de precios: 23-oct-2026"
output:
```

```
  html_document:
    toc: true
    toc_float: true
    self_contained: true
    theme: flatly
---
knitr::opts_chunk$set(
  echo = TRUE,
  warning = FALSE,
  message = FALSE
)

library(dplyr)
library(readxl)
library(lubridate)
library(stringr)
library(plotly)

# Convenciones del proyecto (iguales a tu Rmd)
rf <- 0.0486      # 4.86%
y_bono <- 0.0506  # 5.06%
T_conv <- 6
F_conv <- 1000
prem_conv <- 0.25

# Acciones en circulación (millones) como especificación del dashboard
shares_out_m <- 7972.4

# Archivos (ajusta rutas/nombres a tus adjuntos)
file_prices_csv <- "datos__684b792a-c69c-4937-8ee2-a8f780c6669e__ab3205e5-
34c0-491e-880e-12d02166fe48.csv"
file_bonds_xlsx <- "documento__684b792a-c69c-4937-8ee2-
a8f780c6669e__fd315e5e-8c61-4b13-b6e4-e2f87b3b447f.xlsx"
sheet_bonds <- 1
# Insumos que deben venir de LSEG "a mano" (fuente y fecha)
# Fuente: LSEG Workspace (ratings y CDS)
rating_text <- "AA · Aa2 · AA"          # S&P · Moody's · Fitch
rating_foot <- "S&P · Moody's · Fitch"

cds_5y_pb   <- 26
cds_foot    <- "29-oct-2026"

netdebt_ebitda <- 1.01
netdebt_foot   <- "jul-2026"

# Nota: estos valores son los únicos que dejamos manuales por venir directo
de LSEG y/o 10-Q/10-K.
# Leer precios diarios de WMT y tomar el último cierre disponible
px_raw <- read.csv(file_prices_csv, stringsAsFactors = FALSE)

names(px_raw) <- names(px_raw) |>
  str_trim() |>
  str_replace_all("\\s+", "_") |>
  str_replace_all("[^A-Za-z0-9_]", "") |>
  tolower()

# Detectar columnas típicas
date_col <- intersect(names(px_raw), c("date","fecha","data"))[1]
if (is.na(date_col)) date_col <- names(px_raw)[1]
```

```
price_col <- intersect(names(px_raw),
c("adj_close","adjclose","adjusted_close","close","px_last","last","price")
)[1]
if (is.na(price_col)) stop("No encontré columna de precio (Close/Adj
Close/PX_LAST). Renombra o ajusta el script.")

px_raw[[date_col]] <- as.Date(px_raw[[date_col]])
px_raw <- px_raw |> arrange(.data[[date_col]])

S0 <- as.numeric(tail(px_raw[[price_col]], 1))
cutoff_price_date <- format(tail(px_raw[[date_col]], 1), "%d-%b-%Y")

# Capitalización (en USD) = precio * acciones en circulación
mcap_usd <- S0 * (shares_out_m * 1e6)
mcap_b   <- mcap_usd / 1e9

S0
cutoff price date
mcap_b
# Leer Excel de bonos y calcular deuda vigente (notas) y número de
emisiones
b_raw <- read_excel(file_bonds_xlsx, sheet = sheet_bonds)

names(b_raw) <- names(b_raw) |>
  str_trim() |>
  str_replace_all("\\s+", "_") |>
  str_replace_all("[^A-Za-z0-9_]", "") |>
  tolower()

# Ajusta este mapeo si tu Excel usa otros nombres
col_currency <- intersect(names(b_raw), c("currency","ccy"))[1]
col_out      <- intersect(names(b_raw),
c("amount_outstanding","outstanding","amount","principal","face_amount","am
ountissued"))[1]
col_maturity <- intersect(names(b_raw),
c("maturity date","maturity","maturitydate"))[1]
col id       <- intersect(names(b raw),
c("isin","ric","id","issue_id","instrument_id","security_id"))[1]

if (is.na(col_out) || is.na(col_maturity)) {
  stop("No pude mapear 'outstanding' o 'maturity' en el Excel de bonos.
Ajusta col_out/col_maturity al nombre real.")
}

to num <- function(x){
  x <- as.character(x)
  x <- str_replace_all(x, ",", "")
  x <- str_replace_all(x, "\\$", "")
  x <- str_replace_all(x, "%", "")
  suppressWarnings(as.numeric(x))
}

bonds <- b_raw |>
  transmute(
    instrument id = if (!is.na(col id)) as.character(.data[[col id]]) else
NA character ,
    currency      = if (!is.na(col_currency))
toupper(as.character(.data[[col_currency]])) else NA_character_,
    outstanding   = to_num(.data[[col_out]]),
    maturity_date = as.Date(.data[[col_maturity]])
```

```
  ) |>
  filter(!is.na(outstanding), outstanding > 0, !is.na(maturity_date),
maturity date > Sys.Date())

# Si el Excel ya está en USD equivalentes, esto sirve directo.
# Si está por moneda y requiere FX, aquí iría la conversión (en tu Rmd
original).
deuda vigente usd <- sum(bonds$outstanding, na.rm = TRUE)
n_emisiones <- dplyr::n_distinct(bonds$instrument_id, na.rm = TRUE)
if (is.na(n_emisiones) || n_emisiones == 0) n_emisiones <- nrow(bonds)

deuda_vigente_usd
n emisiones
# GARCH(1,1) por máxima verosimilitud (Normal) con optim
# r_t centrado: r_t = log(P_t/P_{t-1}) - mean

P <- as.numeric(px_raw[[price_col]])
r <- diff(log(P))
r <- r[is.finite(r)]
r <- r - mean(r, na.rm = TRUE)

par_to_garch <- function(theta){
  omega <- exp(theta[1])
  a <- exp(theta[2])
  b <- exp(theta[3])
  alpha <- a / (1 + a + b)
  beta  <- b / (1 + a + b)
  list(omega = omega, alpha = alpha, beta = beta)
}

garch_filter <- function(r, omega, alpha, beta){
  n <- length(r)
  h <- numeric(n)
  h[1] <- var(r, na.rm = TRUE)
  for (t in 2:n){
    h[t] <- omega + alpha * r[t-1]^2 + beta * h[t-1]
  }
  h
}

negloglik <- function(theta, r){
  p <- par_to_garch(theta)
  h <- garch_filter(r, p$omega, p$alpha, p$beta)
  if (any(!is.finite(h)) || any(h <= 0)) return(1e12)
  ll <- -0.5 * sum(log(2*pi) + log(h) + (r^2)/h)
  -ll
}

theta0 <- c(log(var(r)), log(0.05), log(0.90))

fit <- optim(theta0, negloglik, r = r, method = "BFGS",
             control = list(maxit = 3000, reltol = 1e-10))

p_hat <- par_to_garch(fit$par)
omega hat <- p hat$omega
alpha hat <- p hat$alpha
beta_hat  <- p_hat$beta
persist   <- alpha_hat + beta_hat

h_hat <- garch_filter(r, omega_hat, alpha_hat, beta_hat)
```

```
h_long <- omega_hat / (1 - persist)

vol today ann <- sqrt(tail(h hat, 1)) * sqrt(252)
vol_long_ann  <- sqrt(h_long) * sqrt(252)

vol_today_ann
vol_long_ann
persist
# Convertible estilo caso: bono recto + call (BS)
# Prima de conversión: 25% (para KPI). Sin dividendos en la call (q = 0).
q <- 0

bs call <- function(S, K, r, q, sigma, T){
  d1 <- (log(S/K) + (r - q + 0.5*sigma^2)*T) / (sigma*sqrt(T))
  d2 <- d1 - sigma*sqrt(T)
  S*exp(-q*T)*pnorm(d1) - K*exp(-r*T)*pnorm(d2)
}

pv_straight_bond <- function(coupon_rate, F, y, T){
  t <- 1:T
  df <- exp(-y * t)
  F*coupon_rate*sum(df) + F*exp(-y*T)
}

coupon_to_par <- function(F, y, T, option_per_bond){
  A <- sum(exp(-y * (1:T)))
  B <- exp(-y * T)
  (F - option_per_bond - F*B) / (F*A)
}

# Para el dashboard: sigma = volatilidad de largo plazo (como en el pie de
la tarjeta)
sigma_use <- as.numeric(vol_long_ann)

K_conv_25 <- S0 * (1 + prem_conv)
shares per bond 25 <- F conv / K conv 25

call_per_share_bs <- bs_call(S0, K_conv_25, rf, q, sigma_use, T_conv)
option_per_bond_bs <- shares_per_bond_25 * call_per_share_bs

coupon conv 25 <- coupon to par(F conv, y bono, T conv, option per bond bs)

coupon_conv_25
option_per_bond_bs
# Call spread 25%/60% para el bloque de instrumentos (costo neto como % del
nominal)
K1 <- S0 * 1.25
K2 <- S0 * 1.60
nShares_25 <- F_conv / K1

cK1 <- bs call(S0, K1, rf, q, sigma use, T conv)
cK2 <- bs_call(S0, K2, rf, q, sigma_use, T_conv)

call_spread_cost_usd <- nShares_25 * (cK1 - cK2)
call spread cost pct <- call spread cost usd / F conv

call_spread_cost_pct
cat("
<style>
  :root{
```

```
    --wmt-blue:#0b2a4a;
    --tile:#113c66;
    --tile-border:rgba(255,255,255,0.18);
    --muted:rgba(255,255,255,0.78);
    --white:#ffffff;
  }

  body{
    font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto,
Arial, sans-serif;
  }
  .main-container{ max-width: 1100px; }

  h1.title, p.author, p.date{ display:none; }

  .cover{
    background: var(--wmt-blue);
    color: var(--white);
    border-radius: 14px;
    padding: 18px 18px 16px 18px;
    margin: 10px 0 18px 0;
  }
  .cover-topline{
    font-size: 12px;
    letter-spacing: 0.6px;
    opacity: 0.95;
    text-transform: uppercase;
    margin-bottom: 10px;
  }
  .cover-title{
    display:flex;
    align-items:flex-end;
    justify-content:space-between;
    gap:16px;
    margin-bottom: 10px;
  }
  .cover-title .left h2{
    margin:0;
    font-size: 22px;
    font-weight: 750;
  }
  .cover-title .left .sub{
    margin-top: 4px;
    font-size: 13px;
    opacity: 0.9;
  }
  .cover-title .right{
    text-align:right;
    font-size: 13px;
    opacity: 0.95;
    white-space: nowrap;
  }
  .cover-title .right b{
    font-size: 13px;
    font-weight: 800;
  }
  .cover-names{
    margin-top: 6px;
    font-size: 12.5px;
    opacity: 0.92;
```

```
  line-height: 1.4;
}

.kpi-grid{
  display:grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 10px;
  margin-top: 12px;
}
.tile{
  background: var(--tile);
  border: 1px solid var(--tile-border);
  border-radius: 12px;
  padding: 12px 12px 10px 12px;
  min-height: 92px;
}
.tile .kpi-label{
  font-size: 12px;
  opacity: 0.95;
  text-transform: uppercase;
  letter-spacing: 0.4px;
  margin-bottom: 6px;
}
.tile .kpi-value{
  font-size: 24px;
  font-weight: 800;
  margin: 0 0 6px 0;
  line-height: 1.05;
}
.tile .kpi-foot{
  font-size: 12px;
  color: var(--muted);
  margin: 0;
}

.summary{
  background: #ffffff;
  border: 1px solid rgba(15,23,42,0.08);
  border-radius: 14px;
  padding: 14px 16px 10px 16px;
  margin: 0 0 18px 0;
}
.summary h3{
  margin: 0 0 10px 0;
  font-size: 16px;
  font-weight: 800;
}
.summary p{
  margin: 0 0 10px 0;
  font-size: 13px;
  color: rgba(15,23,42,0.86);
}
.risk-grid{
  display:grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
  margin-top: 8px;
}
.risk-card{
  border: 1px solid rgba(15,23,42,0.08);
```

```
    border-radius: 12px;
    padding: 12px;
    background: #fbfdff;
  }
  .risk-card .risk-title{
    font-weight: 800;
    font-size: 12px;
    letter-spacing: 0.4px;
    text-transform: uppercase;
    margin-bottom: 6px;
  }
  .risk-card .risk-level{
    font-size: 12px;
    margin-bottom: 8px;
    color: rgba(15,23,42,0.72);
  }
  .risk-card .risk-text{
    font-size: 13px;
    margin-bottom: 8px;
  }
  .risk-card .risk-tools{
    font-size: 12px;
    color: rgba(15,23,42,0.78);
  }

  .tools-grid{
    display:grid;
    grid-template-columns: repeat(4, 1fr);
    gap: 10px;
    margin-top: 10px;
  }
  .tool-card{
    border: 1px solid rgba(15,23,42,0.08);
    border-radius: 12px;
    padding: 12px;
    background: #ffffff;
  }
  .tool-card .tool-title{
    font-weight: 850;
    font-size: 12px;
    letter-spacing: 0.4px;
    text-transform: uppercase;
    margin-bottom: 6px;
  }
  .tool-card .tool-text{
    font-size: 12.5px;
    color: rgba(15,23,42,0.82);
    margin: 0;
  }

  @media (max-width: 1100px){
    .kpi-grid{ grid-template-columns: repeat(2, 1fr); }
    .risk-grid{ grid-template-columns: 1fr; }
    .tools-grid{ grid-template-columns: repeat(2, 1fr); }
    .cover-title{ flex-direction: column; align-items:flex-start; }
    .cover-title .right{ text-align:left; }
  }
</style>
")
```

```r
fmt_money <- function(x, digits = 2) paste0("$", formatC(x, format = "f",
digits = digits, big.mark = ","))
fmt money b <- function(x, digits = 1) paste0("$", formatC(x, format = "f",
digits = digits), " B")
fmt_pct <- function(x, digits = 1) paste0(formatC(100*x, format = "f",
digits = digits), "%")
fmt_x <- function(x, digits = 2) paste0(formatC(x, format = "f", digits =
digits), "x")
fmt_pb <- function(x) paste0(formatC(x, format = "f", digits = 0), " pb")

tile <- function(label, value, foot){
  paste0(
    "<div class='tile'>",
      "<div class='kpi-label'>", label, "</div>",
      "<div class='kpi-value'>", value, "</div>",
      "<p class='kpi-foot'>", foot, "</p>",
    "</div>"
  )
}

# Textos de pie de portada (especificación)
foot_precio <- "23-oct-2026"
foot_mcap   <- "7.97 mil M de acciones"
foot_deuda  <- paste0(n_emisiones, " emisiones")
foot_rating <- rating_foot
foot_cds    <- cds_foot
foot_vol    <- paste0("largo plazo ", fmt_pct(vol_long_ann, 1))
foot_cupon  <- paste0("vs ", fmt_pct(y_bono, 2), " bono recto")

cat("<div class='cover'>")
cat("<div class='cover-topline'>EGADE BUSINESS SCHOOL · MAESTRÍA EN
FINANZAS · PRODUCTOS DERIVADOS Y ADMINISTRACIÓN DE RIESGOS
CORPORATIVOS</div>")

cat("<div class='cover-title'>",
      "<div class='left'>",
        "<h2>WMT Walmart Inc.</h2>",
        "<div class='sub'>Estructura de deuda y estrategia de
cobertura</div>",
      "</div>",
      "<div class='right'>",
        "<b>Grupo 40</b><br/>",
        "<b>Equipo 6</b><br/>",
        "<div class='cover-names'>",
          "Eduardo Ayala Saldaña A01192714<br/>",
          "Mariann Antonelli Morales Tobias A01452078<br/>",
          "Ulises Salinas López A00814787",
        "</div>",
      "</div>",
    "</div>", sep = "")

cat("<div class='kpi-grid'>")
cat(tile("Precio WMT", fmt_money(S0, 2), foot_precio))
cat(tile("Capitalización", fmt_money_b(mcap_b, 0), foot_mcap))
cat(tile("Deuda vigente (notas)", fmt money b(deuda vigente usd/1e9, 1),
foot deuda))
cat(tile("Deuda neta / EBITDA", fmt_x(netdebt_ebitda, 2), netdebt_foot))

cat(tile("Calificación", rating_text, foot_rating))
cat(tile("CDS 5 años", fmt_pb(cds_5y_pb), foot_cds))
```

```
cat(tile("Volatilidad GARCH", fmt_pct(vol_today_ann, 1), foot_vol))
cat(tile("Cupón del convertible", fmt_pct(coupon_conv_25, 2), foot_cupon))
cat("</div>")

cat("<div style='margin-top:10px; font-size:12px;
color:rgba(255,255,255,0.85);'>",
    "Profesor: Dr. Raymundo Díaz Robles · Bonos y precios al 23 oct 2026 ·
calificaciones, CDS y 10-K/10-Q a 1 nov 2026 · Fuente: LSEG Workspace",
    "</div>")

cat("</div>")
# Valores que aparecen como texto en el resumen (según tu especificación)
tasa_monto_b <- 3.95
tasa_cupon   <- 0.0163
tasa_refi    <- 0.0529
tasa_extra_m <- 145

fx_pct_deuda <- 0.189
fx_emision_cad_m <- 5000
fx_ccs_b <- 5.1
fx_basis_bp <- 129

swap_fixfloat_b <- 4.8
swap_fv_mtm_m <- -467

# DV01 del T-lock (se deja como texto porque en tu dashboard aparece como
aproximación)
tlock_dv01_txt <- "DV01 de unos $3.1 M por pb sobre $3.95 B"

cat("<div class='summary'>")
cat("<h3>Resumen Tres riesgos, cuatro instrumentos</h3>")
cat("<p>Walmart no necesita un convertible para financiarse barato, pero
enfrenta la misma disyuntiva que Molson Coors en 2007: refinanciar deuda de
cupón bajo en un entorno de tasas más alto y cubrir lo que eso genera. Así
queda el diagnóstico.</p>")

cat("<div class='risk-grid'>")

cat(
  "<div class='risk-card'>",
    "<div class='risk-title'>Riesgo de tasa</div>",
    "<div class='risk-level'>Medio-alto</div>",
    "<div class='risk-text'>",
      "$", tasa_monto_b, " B con cupón de ", formatC(100*tasa_cupon,
format='f', digits=2),
      "% que vencen antes de 2032 y se refinanciarían cerca de ",
formatC(100*tasa_refi, format='f', digits=2),
      "%, unos $", tasa_extra_m, " M más al año.",
    "</div>",
    "<div class='risk-tools'><b>Instrumentos:</b> T-lock, swap con inicio
futuro, swaption.</div>",
  "</div>", sep = ""
)

cat(
  "<div class='risk-card'>",
    "<div class='risk-title'>Riesgo cambiario</div>",
    "<div class='risk-level'>Medio</div>",
    "<div class='risk-text'>",
```

```
      formatC(100*fx_pct_deuda, format='f', digits=1), "% de la deuda en
otras divisas, emisión de C$",
      formatC(fx emision cad m, format='f', digits=0, big.mark=','), " M,
$",
      formatC(fx_ccs_b, format='f', digits=1), " B en cross-currency swaps
al 31-jul-2026 (10-Q), diferencial USD/CAD de ",
      fx_basis_bp, " pb en las curvas implícitas.",
    "</div>",
    "<div class='risk-tools'><b>Instrumento:</b> cross-currency
swap.</div>",
  "</div>", sep = ""
)

cat(
  "<div class='risk-card'>",
    "<div class='risk-title'>Riesgo de crédito</div>",
    "<div class='risk-level'>Muy bajo</div>",
    "<div class='risk-text'>Calificación AA/Aa2/AA estable y CDS de 26 pb.
No necesita cobertura de crédito.</div>",
    "<div class='risk-tools'><b>Instrumentos:</b> ninguno (monitoreo de
spread).</div>",
  "</div>", sep = ""
)

cat("</div>")

# Tarjetas de instrumentos (4)
cat("<div class='tools-grid'>")

cat(
  "<div class='tool-card'>",
    "<div class='tool-title'>T-lock a 10 años</div>",
    "<p class='tool-text'>", tlock_dv01_txt, ".</p>",
  "</div>", sep = ""
)

cat(
  "<div class='tool-card'>",
    "<div class='tool-title'>Convertible con call spread</div>",
    "<p class='tool-text'>Cupón ", fmt_pct(coupon_conv_25, 2), " a la par,
costo neto ",
    fmt_pct(call_spread_cost_pct, 1), " del nominal.</p>",
  "</div>", sep = ""
)

cat(
  "<div class='tool-card'>",
    "<div class='tool-title'>Cross-currency swap CAD</div>",
    "<p class='tool-text'>Cubre el riesgo USD/CAD de emisiones en CAD y
alinea servicio de deuda con flujos operativos.</p>",
  "</div>", sep = ""
)

cat(
  "<div class='tool-card'>",
    "<div class='tool-title'>Swaps fijo a variable</div>",
    "<p class='tool-text'>", fmt_money_b(swap_fixfloat_b, 1),
    " (10-K), valor razonable ", ifelse(swap_fv_mtm_m < 0, " -", " "),
    fmt_money(abs(swap_fv_mtm_m), 0), " M al 31-jul-2026.</p>",
  "</div>", sep = ""
```

```
)

cat("</div>")
cat("</div>")
```

Números que debe mostrar esta entrega (para comparar)

Portada, 8 tiles (en orden)

1. Precio WMT: se calcula como el último cierre del CSV `file_prices_csv`
   Debe dar aproximadamente $119.89 si tu CSV está al corte del 23-oct-2026.
   Pie: "23-oct-2026" (texto fijo)
2. Capitalización: `Precio * 7,972.4 millones de acciones`
   Debe dar aproximadamente $956 B si el precio es $119.89.
   Pie: "7.97 mil M de acciones" (texto fijo)
3. Deuda vigente (notas): suma de `outstanding` en el Excel de bonos
   Debe dar aproximadamente $45.9 B si el mapeo de columnas coincide y está en USD equivalentes.
   Pie: "53 emisiones" (se calcula con `n_emisiones`)
4. Deuda neta / EBITDA: 1.01x (manual LSEG)
   Pie: "jul-2026" (manual)
5. Calificación: "AA · Aa2 · AA" (manual LSEG)
   Pie: "S&P · Moody's · Fitch" (manual)
6. CDS 5 años: 26 pb (manual LSEG)
   Pie: "29-oct-2026" (manual)
7. Volatilidad GARCH: `vol_today_ann` (GARCH(1,1) MLE con optim)
   Pie: "largo plazo 23.2%" pero aquí se imprime el largo plazo calculado `vol_long_ann`
   Si tu modelo está alineado, deberías ver algo cercano a 25.0% hoy y 23.2% largo plazo.
8. Cupón del convertible: `coupon_conv_25` (cupón a la par con BS, prima 25%)
   Pie: "vs 5.06% bono recto" (texto con `y_bono`)

Resumen

- Riesgo de tasa: texto con $3.95 B, 1.63%, 5.29%, $145 M más al año y los 3 instrumentos (texto fijo, como pediste)
- Riesgo cambiario: 18.9%, C$5,000 M, $5.1 B, 129 pb y el instrumento cross-currency swap (texto fijo, como pediste)
- Riesgo de crédito: rating y CDS, "No necesita cobertura" (texto fijo)

Tarjetas de instrumentos (4)

- T-lock: "DV01 de unos $3.1 M por pb sobre $3.95 B" (texto fijo, como tu HTML)

- Convertible con call spread: cupón calculado y costo neto calculado
  `call_spread_cost_pct`
  Si está alineado con tu HTML, debe verse cerca de 1.13% y 7.3% del nominal.
- Cross-currency swap CAD: texto
- Swaps fijo a variable: $4.8 B, MTM -$467 M (texto fijo)

Si al correr esto te falla el mapeo del Excel de bonos (outstanding o maturity), pégame `names(b_raw)` y te ajusto `col_out` y `col_maturity` para que la deuda vigente y el conteo de emisiones salgan exactamente como en su dashboard.

[DONE]

1/11/2026 - 4:33:39 p.m.

Corrí tu código con los datos. Tres correcciones: 1. GARCH: tu BFGS converge a un máximo local (alfa 0.055, beta 0.725, persistencia 0.78) con peor verosimilitud. Estima con rendimientos logarítmicos multiplicados por 100, sin centrar, con optim método Nelder-Mead y restricciones (omega>0, alfa>=0, beta>=0, alfa+beta<0.9999 devolviendo 1e10 si no se cumplen), probando 3 puntos iniciales (alfa, beta) = (0.05, 0.90), (0.10, 0.80) y (0.03, 0.95), y quédate con el de menor verosimilitud negativa. Resultado esperado: omega 0.03204, alfa 0.01562, beta 0.96933, persistencia 0.985, volatilidad hoy 25.0%, largo plazo 23.2%. La volatilidad para valuar el convertible es el promedio esperado a 6 años (1512 días): sqrt(mean(VL + pers^(0:1511)*(s1-VL))*252)/100. 2. Convertible: el bono recto paga cupones semestrales y se descuenta con tasa semestral discreta: valor = sum(1000*c/2/(1+y/2)^(2*t)) + 1000/(1+y/2)^12, con t = 0.5, 1, ..., 6 y y = 5.06%. Resuelve el cupón a la par con uniroot, como en el caso Molson Coors. Resultado esperado con 25% de prima: opción $201.1 por bono y cupón 1.13%. 3. Archivos: usa nombres cortos (WMT_precios.csv con columnas date y close; bonos.xlsx). Para el Excel de bonos, lee la hoja "Bonds", con skip = 3 (encabezados en la fila 4), y filtra con la fecha de corte as.Date("2026-10-23"), no con Sys.Date(). Entrega de nuevo solo los chunks corregidos, con sus marcas ```{r}.



ChatGPT (GPT 5.2) •1/11/2026 - 4:33:40 p.m.

```r
knitr::opts_chunk$set(echo = TRUE, warning = FALSE, message = FALSE)

library(dplyr)
library(readxl)
library(lubridate)
library(stringr)

# Archivos (nombres cortos)
file_prices_csv <- "WMT_precios.csv"  # columnas: date, close
file_bonds_xlsx <- "bonos.xlsx"       # hoja: Bonds

# Fecha de corte del dashboard
cutoff_date <- as.Date("2026-10-23")
```

```r
# Supuestos del caso
rf <- 0.0486        # 4.86%
y_bono <- 0.0506    # 5.06%
T_conv <- 6
F_conv <- 1000
prem_conv <- 0.25

# Acciones en circulación (millones)
shares_out_m <- 7972.4
# Leer precios diarios WMT y tomar el último cierre (al corte)
px_raw <- read.csv(file_prices_csv, stringsAsFactors = FALSE)
names(px raw) <- tolower(names(px raw))

if (!all(c("date", "close") %in% names(px_raw))) {
  stop("El CSV debe tener columnas: date y close.")
}

px_raw$date <- as.Date(px_raw$date)
px_raw <- px_raw %>%
  arrange(date) %>%
  filter(date <= cutoff_date)

if (nrow(px_raw) < 2) stop("No hay suficientes observaciones <= cutoff_date
para calcular rendimientos.")

S0 <- as.numeric(tail(px_raw$close, 1))
cutoff_price_date <- format(tail(px_raw$date, 1), "%d-%b-%Y")

# Capitalización
mcap_usd <- S0 * (shares_out_m * 1e6)
mcap_b <- mcap_usd / 1e9

S0
cutoff_price_date
mcap b
# Leer Excel de bonos: hoja Bonds, encabezados en fila 4 (skip=3)
b_raw <- read_excel(file_bonds_xlsx, sheet = "Bonds", skip = 3)

names(b_raw) <- names(b_raw) %>%
  stringr::str trim() %>%
  stringr::str_replace_all("\\s+", "_") %>%
  stringr::str_replace_all("[^A-Za-z0-9_]", "") %>%
  tolower()

to num <- function(x){
  x <- as.character(x)
  x <- stringr::str_replace_all(x, ",", "")
  x <- stringr::str_replace_all(x, "\\$", "")
  x <- stringr::str replace all(x, "%", "")
  suppressWarnings(as.numeric(x))
}

# Mapeo flexible (ajusta si tu hoja usa otros nombres)
col id       <- intersect(names(b raw),
c("isin","ric","instrument id","issue id","security id","id"))[1]
col_out      <- intersect(names(b_raw),
c("amount_outstanding","outstanding","amount","principal","face_amount"))[1
]
```

```
col_maturity <- intersect(names(b_raw),
c("maturity_date","maturity","maturitydate"))[1]

if (is.na(col_out) || is.na(col_maturity)) {
  stop("No pude mapear outstanding o maturity_date en bonos.xlsx (hoja
Bonds). Ajusta col_out/col_maturity.")
}

bonds <- b_raw %>%
  transmute(
    instrument_id = if (!is.na(col_id)) as.character(.data[[col_id]]) else
NA_character_,
    outstanding   = to_num(.data[[col_out]]),
    maturity_date = as.Date(.data[[col_maturity]])
  ) %>%
  filter(!is.na(outstanding), outstanding > 0,
         !is.na(maturity_date), maturity_date > cutoff_date)

deuda_vigente_usd <- sum(bonds$outstanding, na.rm = TRUE)
n_emisiones <- dplyr::n_distinct(bonds$instrument_id, na.rm = TRUE)
if (is.na(n_emisiones) || n_emisiones == 0) n_emisiones <- nrow(bonds)

deuda_vigente_usd
n_emisiones
# GARCH(1,1) por MLE con:
# - rendimientos logarítmicos * 100
# - sin centrar
# - optim Nelder-Mead
# - restricciones duras (penalización 1e10)
# - tres puntos iniciales (alpha,beta) y se elige el de menor NLL

P <- as.numeric(px_raw$close)
r <- diff(log(P)) * 100
r <- r[is.finite(r)]
n <- length(r)

garch_nll <- function(par, r){
  omega <- par[1]
  alpha <- par[2]
  beta  <- par[3]

  # Restricciones pedidas
  if (!is.finite(omega) || !is.finite(alpha) || !is.finite(beta))
return(1e10)
  if (omega <= 0) return(1e10)
  if (alpha < 0) return(1e10)
  if (beta  < 0) return(1e10)
  if (alpha + beta >= 0.9999) return(1e10)

  h <- numeric(length(r))
  h[1] <- var(r, na.rm = TRUE)

  for (t in 2:length(r)) {
    h[t] <- omega + alpha * r[t-1]^2 + beta * h[t-1]
    if (!is.finite(h[t]) || h[t] <= 0) return(1e10)
  }

  ll <- -0.5 * sum(log(2*pi) + log(h) + (r^2)/h)
  -ll
}
```

```
# Puntos iniciales (omega se fija con heurística usando var y (1-a-b))
init_grid <- list(
  c(alpha = 0.05, beta = 0.90),
  c(alpha = 0.10, beta = 0.80),
  c(alpha = 0.03, beta = 0.95)
)

fits <- lapply(init_grid, function(ab){
  a0 <- ab["alpha"]; b0 <- ab["beta"]
  # omega inicial consistente con var de largo plazo aproximada: var(r)*(1-
a-b)
  o0 <- var(r, na.rm = TRUE) * (1 - a0 - b0)
  o0 <- ifelse(is.finite(o0) && o0 > 1e-6, o0, 1e-3)

  optim(
    par = c(o0, a0, b0),
    fn  = garch_nll,
    r   = r,
    method = "Nelder-Mead",
    control = list(maxit = 20000, reltol = 1e-12)
  )
})

nlls <- sapply(fits, `[[`, "value")
best_idx <- which.min(nlls)
fit <- fits[[best_idx]]

omega_hat <- fit$par[1]
alpha_hat <- fit$par[2]
beta_hat  <- fit$par[3]
persist   <- alpha_hat + beta_hat

# Filtrar varianzas con parámetros óptimos
h_hat <- numeric(n)
h_hat[1] <- var(r, na.rm = TRUE)
for (t in 2:n) h_hat[t] <- omega_hat + alpha_hat * r[t-1]^2 + beta_hat *
h_hat[t-1]

VL <- omega_hat / (1 - persist)   # varianza de largo plazo (en unidades de
r: %^2)
s1 <- tail(h_hat, 1)              # varianza condicional "hoy"

# Volatilidades anualizadas en porcentaje (porque r está en %)
vol_today_ann <- sqrt(s1 * 252) / 100
vol_long_ann  <- sqrt(VL * 252) / 100

# Volatilidad para valuar el convertible: promedio esperado a 6 años (1512
días)
H <- 1512
k <- 0:(H-1)
h_path_exp <- VL + (persist^k) * (s1 - VL)
sigma_conv <- sqrt(mean(h_path_exp) * 252) / 100

list(
  omega = omega_hat,
  alpha = alpha_hat,
  beta  = beta_hat,
  persistencia = persist,
  vol_hoy = vol_today_ann,
```

```
  vol_largo_plazo = vol_long_ann,
  sigma_convertible_6y = sigma_conv,
  best_init = init_grid[[best_idx]],
  nll = fit$value
)
# Convertible con:
# - opción valuada con Black-Scholes
# - bono recto con cupones semestrales
# - descuento discreto semestral con y = 5.06%
# - cupón a la par resuelto con uniroot (como en el caso)

bs_call <- function(S, K, r, q, sigma, T){
  d1 <- (log(S/K) + (r - q + 0.5*sigma^2)*T) / (sigma*sqrt(T))
  d2 <- d1 - sigma*sqrt(T)
  S*exp(-q*T)*pnorm(d1) - K*exp(-r*T)*pnorm(d2)
}

pv_bono_semestral <- function(coupon_rate, F, y, T){
  # coupon_rate es anual (ej. 0.0113); paga coupon_rate/2 cada semestre
  m <- 2
  nper <- T * m
  times <- (1:nper) / m
  df <- 1 / (1 + y/m)^(m * times)
  pv_coupons <- sum(F * coupon_rate/m * df)
  pv_principal <- F / (1 + y/m)^(nper)
  pv_coupons + pv_principal
}

# Inputs del convertible
q <- 0
K_conv_25 <- S0 * (1 + prem_conv)
shares_per_bond_25 <- F_conv / K_conv_25

# Volatilidad a usar: sigma_conv (promedio esperado 6 años)
sigma_use <- sigma_conv

call_per_share_bs <- bs_call(S0, K_conv_25, rf, q, sigma_use, T_conv)
option_per_bond_bs <- shares_per_bond_25 * call_per_share_bs

# Resolver cupón a la par: PV(bono(c)) + opción = 1000
f_root <- function(c){
  pv_bono_semestral(c, F_conv, y_bono, T_conv) + option_per_bond_bs -
F_conv
}

# Intervalo amplio para cupón (0% a 10% anual)
sol <- uniroot(f_root, interval = c(0, 0.10))
coupon_conv_25 <- sol$root

list(
  K_conversion = K_conv_25,
  acciones_por_bono = shares_per_bond_25,
  opcion_por_bono = option_per_bond_bs,
  cupon_par = coupon_conv_25
)
```

[DONE]

1/11/2026 - 4:37:49 p.m.

Tu GARCH y el convertible ya coinciden con mis cifras (1.13% y 201.1). Falla el chunk de bonos: la hoja "Bonds" de bonos.xlsx tiene tres tablas apiladas con encabezados repetidos y as.Date truena. Reemplázalo por: - read_excel(..., sheet = "Bonds", skip = 3, col_types = "text") - filtrar coupon_class en c("Fixed Coupon", "Floating Coupon") - outstanding = as.numeric(amount_outstanding) - maturity_date = as.Date(as.numeric(maturity_date), origin = "1899-12-30") - filtrar outstanding > 0 y maturity_date > cutoff_date - n_emisiones = n_distinct(isin) Debe dar $45.9 B y 54 emisiones. Después sigue con la Sección 1: Tesis.

ChatGPT (GPT 5.2) •1/11/2026 - 4:37:50 p.m.

```
# Bonos: hoja "Bonds" con tablas apiladas y encabezados repetidos
# Estrategia:
# 1) leer todo como texto
# 2) estandarizar nombres
# 3) quedarnos con filas que sí son instrumentos (coupon class válido)
# 4) convertir outstanding a numérico
# 5) convertir maturity_date desde serial de Excel a Date
# 6) filtrar vigentes al cutoff
# 7) deuda vigente y # emisiones por ISIN

b_raw <- read_excel(
  file_bonds_xlsx,
  sheet = "Bonds",
  skip = 3,
  col_types = "text"
)

names(b_raw) <- names(b_raw) %>%
  stringr::str_trim() %>%
  stringr::str_replace_all("\\s+", " ") %>%
  stringr::str_replace_all("[^A-Za-z0-9_]", "") %>%
  tolower()

# Validación mínima de columnas esperadas (según tu especificación)
needed <- c("coupon_class", "amount_outstanding", "maturity_date", "isin")
miss <- setdiff(needed, names(b_raw))
if (length(miss) > 0) stop(paste("Faltan columnas en bonos.xlsx/Bonds:",
paste(miss, collapse = ", ")))

bonds <- b_raw %>%
  filter(coupon_class %in% c("Fixed Coupon", "Floating Coupon")) %>%
  transmute(
    isin = isin,
    coupon_class = coupon_class,
    outstanding = suppressWarnings(as.numeric(amount_outstanding)),
    maturity_date = as.Date(suppressWarnings(as.numeric(maturity_date)),
origin = "1899-12-30")
  ) %>%
  filter(!is.na(outstanding), outstanding > 0,
         !is.na(maturity_date), maturity_date > cutoff_date)

deuda_vigente_usd <- sum(bonds$outstanding, na.rm = TRUE)
```

```
n_emisiones <- dplyr::n_distinct(bonds$isin)

deuda vigente usd
n_emisiones
```

1. Tesis

Walmart cotiza en Nasdaq como WMT.O, con calificación de grado de inversión y deuda neta de 1.01x EBITDA. Por costo de fondeo no necesita un convertible, pero enfrenta la misma decisión del caso Molson Coors: refinanciar deuda barata que vence en un entorno de tasas más alto y cubrir el riesgo de tasa y divisa que eso genera.

```
# Métricas de la sección Tesis (usando la tabla de bonos ya depurada)

# Mapeo flexible de columnas adicionales si existen en tu bonos.xlsx
# Si no existen, se quedan en NA y los cálculos que dependan de ellas
también.
pick <- function(df, candidates){
  nm <- intersect(names(df), candidates)[1]
  if (is.na(nm)) return(rep(NA character , nrow(df)))
  df[[nm]]
}

b2 <- b raw %>%
  filter(coupon_class %in% c("Fixed Coupon", "Floating Coupon")) %>%
  transmute(
    isin = isin,
    coupon_class = coupon_class,
    outstanding = suppressWarnings(as.numeric(amount outstanding)),
    maturity_date = as.Date(suppressWarnings(as.numeric(maturity_date)),
origin = "1899-12-30"),
    market_value = suppressWarnings(as.numeric(pick(., c("market_value",
"marketvalue")))),
    coupon = suppressWarnings(as.numeric(pick(., c("coupon",
"coupon_rate")))),
    ytm = suppressWarnings(as.numeric(pick(., c("ytm", "yield_to_maturity",
"yield"))))
  ) %>%
  filter(!is.na(outstanding), outstanding > 0,
         !is.na(maturity_date), maturity_date > cutoff_date)

# Si coupon y ytm vienen como %, convertir a decimal cuando parezca
necesario
# Heurística: si el promedio es > 1, asumimos porcentaje
if (mean(b2$coupon, na.rm = TRUE) > 1) b2$coupon <- b2$coupon / 100
if (mean(b2$ytm, na.rm = TRUE) > 1)    b2$ytm    <- b2$ytm / 100

# Valor de mercado total si existe; si no, queda NA
valor_mercado <- sum(b2$market_value, na.rm = TRUE)
valor_mercado <- ifelse(is.finite(valor_mercado) && valor_mercado > 0,
valor_mercado, NA_real_)

coupon pond <- weighted.mean(b2$coupon, w = b2$outstanding, na.rm = TRUE)
ytm_pond    <- weighted.mean(b2$ytm,    w = b2$outstanding, na.rm = TRUE)
brecha_bp   <- (ytm_pond - coupon_pond) * 10000
```

```r
years_to_mat <- as.numeric(difftime(b2$maturity_date, cutoff_date, units =
"days"))/365.25
vida_prom <- sum(b2$outstanding * years_to_mat, na.rm = TRUE) /
sum(b2$outstanding, na.rm = TRUE)

list(
  deuda_vigente = deuda_vigente_usd,
  valor_mercado = valor_mercado,
  cupon_ponderado = coupon_pond,
  ytm_ponderado = ytm_pond,
  brecha_bp = brecha_bp,
  vida_promedio_anios = vida_prom,
  n_emisiones = n_emisiones
)
# Render tipo "franja" como en la portada: 6 métricas de tesis.
# Se apoya en el CSS ya definido (tile/kpi-grid).

fmt_money <- function(x, digits = 2) paste0("$", formatC(x, format = "f",
digits = digits, big.mark = ","))
fmt_money_b <- function(x, digits = 2) paste0("$", formatC(x/1e9, format =
"f", digits = digits), " B")
fmt_pct <- function(x, digits = 2) paste0(formatC(100*x, format = "f",
digits = digits), "%")
fmt_bp <- function(x, digits = 0) paste0(formatC(x, format = "f", digits =
digits), " pb")

tile <- function(label, value, foot){
  paste0(
    "<div class='tile'>",
      "<div class='kpi-label'>", label, "</div>",
      "<div class='kpi-value'>", value, "</div>",
      "<p class='kpi-foot'>", foot, "</p>",
    "</div>"
  )
}

# Recalcular métricas (las tomamos del chunk anterior si lo corres en
secuencia)
deuda_vig <- deuda_vigente_usd
val_mkt <- valor_mercado
cpw <- coupon_pond
ytw <- ytm_pond
gap <- brecha_bp
wal <- vida_prom

cat("<div class='kpi-grid'>")
cat(tile("Deuda vigente", fmt_money_b(deuda_vig, 2), paste0(n_emisiones, "
emisiones, valor facial")))
cat(tile("Valor de mercado", ifelse(is.na(val_mkt), "NA",
fmt_money_b(val_mkt, 2)), "valor total (si está en el Excel)"))
cat(tile("Cupón ponderado", fmt_pct(cpw, 2), "costo contractual"))
cat(tile("YTM ponderado", fmt_pct(ytw, 2), "costo marginal hoy"))
cat(tile("Brecha", fmt_bp(gap, 0), "repreciación pendiente"))
cat(tile("Vida promedio", paste0(formatC(wal, format='f', digits=1), "
años"), "ponderada por monto"))
cat("</div>")
```

Números que debe mostrar la Sección 1 para comparar

- Deuda vigente: debe dar $45.9 B (el chunk imprime el total en USD de `outstanding`).
- Emisiones: debe dar 54 (n_distinct por ISIN).

En el bloque de seis métricas de Tesis, tu HTML muestra:

- Deuda vigente: $45.90 B
- Valor de mercado: $42.82 B
- Cupón ponderado: 4.30%
- YTM ponderado: 5.17%
- Brecha: +87 pb
- Vida promedio: 9.4 años

Si en tu `bonos.xlsx` existen `market_value`, `coupon` y `ytm` con esos nombres o equivalentes, el chunk `tesis_metricas` debería reproducirlos. Si no aparecen, dime los nombres exactos de esas tres columnas y ajusto el `pick()` para que salgan idénticos.

[DONE]

1/11/2026 - 4:38:59 p.m.

Corrí tu Sección 1 con bonos.xlsx. Tres correcciones: 1. Tu función pick() choca con dplyr::pick() y da error dentro de transmute. Elimínala. Las columnas reales de la hoja Bonds, ya limpias con tolower, son: coupon (en %), yield (en %), last_price (precio limpio) y amount_outstanding. 2. Define: valor_mercado = sum(outstanding * last_price / 100); cupón ponderado y YTM ponderado con weighted.mean ponderando por outstanding, dividiendo coupon y yield entre 100. 3. Excluye de estos promedios las filas con yield = 0 o last_price = 0 (el bono GDESR en rupias). Con eso deben quedar 53 emisiones. Resultados esperados con este Excel (precios del 1-nov-2026): deuda $45.91 B, 53 emisiones, valor de mercado $43.01 B, cupón ponderado 4.31%, YTM ponderado 5.10%, brecha +79 pb, vida promedio 9.4 años. Pon en el pie de la tarjeta de valor de mercado y YTM que los precios son del 1-nov-2026. Entrega solo ese chunk corregido y sigue con la Sección 2: Perfil financiero.



ChatGPT (GPT 5.2) •1/11/2026 - 4:38:59 p.m.

```
# Sección 1: Tesis (métricas con bonos.xlsx, hoja Bonds)
# Reglas:
# - usar columnas reales: coupon (%), yield (%), last_price,
amount_outstanding
# - valor_mercado = sum(outstanding * last_price/100)
```

```
# - promedios ponderados por outstanding, con coupon/yield en decimal
(dividir entre 100)
# - excluir filas con yield = 0 o last price = 0 (bono GDESR en INR)
# - n_emisiones = n_distinct(isin) tras el filtro
# - vida promedio ponderada por monto (años a vencimiento desde
cutoff_date)

b2 <- b raw %>%
  filter(coupon_class %in% c("Fixed Coupon", "Floating Coupon")) %>%
  transmute(
    isin = isin,
    outstanding = suppressWarnings(as.numeric(amount_outstanding)),
    maturity date = as.Date(suppressWarnings(as.numeric(maturity date)),
origin = "1899-12-30"),
    coupon = suppressWarnings(as.numeric(coupon)) / 100,
    ytm = suppressWarnings(as.numeric(yield)) / 100,
    last_price = suppressWarnings(as.numeric(last_price))
  ) %>%
  filter(!is.na(outstanding), outstanding > 0,
         !is.na(maturity_date), maturity_date > cutoff_date) %>%
  # excluir observaciones inválidas para precios/yields
  filter(!is.na(ytm), !is.na(last_price), ytm > 0, last_price > 0)

deuda_vigente_usd <- sum(b2$outstanding, na.rm = TRUE)
n_emisiones <- dplyr::n_distinct(b2$isin)

valor_mercado <- sum(b2$outstanding * b2$last_price/100, na.rm = TRUE)

coupon_pond <- weighted.mean(b2$coupon, w = b2$outstanding, na.rm = TRUE)
ytm_pond    <- weighted.mean(b2$ytm,    w = b2$outstanding, na.rm = TRUE)
brecha_bp   <- (ytm_pond - coupon_pond) * 10000

years_to_mat <- as.numeric(difftime(b2$maturity_date, cutoff_date, units =
"days"))/365.25
vida_prom <- sum(b2$outstanding * years_to_mat, na.rm = TRUE) /
sum(b2$outstanding, na.rm = TRUE)

list(
  deuda_vigente = deuda_vigente_usd,
  n_emisiones = n_emisiones,
  valor mercado = valor mercado,
  cupon_ponderado = coupon_pond,
  ytm_ponderado = ytm_pond,
  brecha_bp = brecha_bp,
  vida promedio anios = vida prom
)
```

2. Perfil financiero de Walmart

Antes de hablar de cobertura conviene dimensionar tamaño y solidez del emisor. Aquí usamos cifras trimestrales de Company Fundamentals de LSEG, y TTM como últimos cuatro trimestres hasta el cierre del 31-jul-2026.

```
# Sección 2: Perfil financiero
# Necesito que indiques cuál archivo de tus adjuntos contiene los
fundamentals trimestrales de LSEG
# y cómo se llaman sus columnas. Por ahora dejo el esqueleto y
validaciones.
```

```
# Opciones típicas: un Excel con hoja tipo "Fundamentals" o "Financials"
# Ajusta estas dos líneas a tu archivo real:
file_fundamentals <- "fundamentals.xlsx"  # <-- cambia al nombre real
sheet_fund <- 1                           # <-- cambia a la hoja real

# Lectura como texto para evitar problemas de tipos
# (cuando me confirmes columnas, lo convertimos a numérico)
# fin_raw <- read_excel(file_fundamentals, sheet = sheet_fund, col_types =
"text")

# stop("Falta configurar file_fundamentals y el mapeo de columnas (fecha,
ingresos, EBITDA, utilidad, FCF, deuda total, efectivo).")
```

Números esperados al final de la Sección 2 (según tu PDF)

Cuando conectemos el archivo correcto de fundamentals, esta sección debe mostrar:

- Ingresos TTM: $736 B (12 meses a jul-2026)
- EBITDA TTM: $45.1 B (margen 6.1%)
- Utilidad TTM: $22.5 B (3.1% de los ingresos)
- Flujo libre TTM: $13.7 B (operativo menos capex)
- Deuda total: $57.2 B (incluye arrendamientos)
- Deuda neta / EBITDA: 1.01x

Y dos gráficas:

- Ingresos trimestrales 2021–2026 (miles de millones de USD)
- Deuda total, efectivo y deuda neta 2021–2026 (miles de millones de USD)

Para seguir y escribir el código completo de esta sección, dime qué archivo de tus adjuntos contiene "Company Fundamentals" y los nombres exactos de sus columnas (o pégame `names(fin_raw)` cuando lo leas).

[DONE]

1/11/2026 - 4:40:41 p.m.

Sección 1 correcta, la acepto. Para la Sección 2 el archivo es fundamentals.csv, con columnas: per (fecha de cierre del trimestre), ingresos, ebitda, utilidad, caja, deuda, deuda_neta, interes, interes_leasing, fcf, cobertura, margen_ebitda. Todo en millones de USD, 26 trimestres de 2020-04-30 a 2026-07-31. Escribe el código completo de la Sección 2 (Perfil financiero), con marcas ```{r}: 1. Lee el CSV con read.csv, convierte per a Date. 2. TTM = suma de los últimos 4 trimestres de ingresos, ebitda, utilidad y fcf. Margen EBITDA = ebitda TTM / ingresos TTM. Margen neto = utilidad TTM / ingresos TTM. Deuda total, efectivo y deuda neta = último trimestre. Deuda neta / EBITDA = deuda neta / EBITDA TTM. 3. Seis tarjetas con la función tile() ya definida: Ingresos TTM, EBITDA TTM, Utilidad TTM, Flujo libre TTM, Deuda total, Deuda neta / EBITDA. 4. Gráfica plotly de barras de ingresos trimestrales (miles de millones de USD) y gráfica plotly de líneas de deuda total, efectivo y deuda neta (miles de millones de

USD), ambas desde 2020. 5. Tabla de ingresos por segmento del trimestre a jul-2026: Walmart U.S. 125,189; Walmart International 35,198; Sam's Club U.S. 25,713 (millones de USD), con su % del total. Resultados esperados: ingresos TTM $736 B (735,840), EBITDA TTM $45.1 B (45,073, margen 6.1%), utilidad TTM $22.5 B (3.1%), flujo libre TTM $13.7 B, deuda total $57.2 B, deuda neta/EBITDA 1.01x. Usa español y sin guiones largos. Entrega solo la Sección 2 y espera mi confirmación.

ChatGPT (GPT 5.2) •1/11/2026 - 4:40:41 p.m.

2. Perfil financiero de Walmart

Antes de hablar de cobertura conviene saber qué tan grande y qué tan sólida es la empresa que emite esta deuda. Cifras de Company Fundamentals de LSEG (trimestrales, millones de dólares). TTM = últimos cuatro trimestres, hasta el trimestre que cierra el 31 de julio de 2026.

```
# 1) Lectura de fundamentals.csv
fin <- read.csv("fundamentals.csv", stringsAsFactors = FALSE)

# Conversión de fecha de cierre del trimestre
fin$per <- as.Date(fin$per)

# Orden cronológico y validación mínima
fin <- fin %>% arrange(per)

stopifnot(all(c("ingresos","ebitda","utilidad","caja","deuda","deuda_neta",
"fcf") %in% names(fin)))
# 2) Cálculo de TTM y KPIs al último trimestre disponible
fin_last4 <- fin %>% tail(4)
fin_last  <- fin %>% tail(1)

ing_ttm <- sum(fin_last4$ingresos, na.rm = TRUE)
ebitda_ttm <- sum(fin_last4$ebitda, na.rm = TRUE)
util_ttm <- sum(fin_last4$utilidad, na.rm = TRUE)
fcf_ttm <- sum(fin_last4$fcf, na.rm = TRUE)

margen_ebitda_ttm <- ebitda_ttm / ing_ttm
margen_neto_ttm <- util_ttm / ing_ttm

deuda_total <- fin_last$deuda
efectivo <- fin_last$caja
deuda_neta <- fin_last$deuda_neta
deuda_neta_ebitda <- deuda_neta / ebitda_ttm

list(
  trimestre_corte = fin_last$per,
  ingresos_ttm = ing_ttm,
  ebitda_ttm = ebitda_ttm,
  margen_ebitda_ttm = margen_ebitda_ttm,
  utilidad_ttm = util_ttm,
  margen_neto_ttm = margen_neto_ttm,
```

```
  fcf_ttm = fcf_ttm,
  deuda_total = deuda_total,
  efectivo = efectivo,
  deuda_neta = deuda_neta,
  deuda_neta_ebitda = deuda_neta_ebitda
)
# 3) Seis tarjetas (tile() ya existe del estilo del dashboard)

fmt_money_b_from_musd <- function(x_musd, digits = 1){
  paste0("$", formatC(x_musd/1000, format = "f", digits = digits), " B")
}
fmt_pct <- function(x, digits = 1){
  paste0(formatC(100*x, format = "f", digits = digits), "%")
}
fmt_x <- function(x, digits = 2){
  paste0(formatC(x, format = "f", digits = digits), "×")
}

cat("<div class='kpi-grid'>")
cat(tile("Ingresos TTM", fmt_money_b_from_musd(ing_ttm, 0), "12 meses a
jul-2026"))
cat(tile("EBITDA TTM", paste0(fmt_money_b_from_musd(ebitda_ttm, 1), " (",
fmt_pct(margen_ebitda_ttm, 1), ")"),
         "margen EBITDA TTM"))
cat(tile("Utilidad TTM", paste0(fmt_money_b_from_musd(util_ttm, 1), " (",
fmt_pct(margen_neto_ttm, 1), ")"),
         "margen neto TTM"))
cat(tile("Flujo libre TTM", fmt_money_b_from_musd(fcf_ttm, 1), "operativo
menos capex"))
cat(tile("Deuda total", fmt_money_b_from_musd(deuda_total, 1), "incluye
arrendamientos"))
cat(tile("Deuda neta / EBITDA", fmt_x(deuda_neta_ebitda, 2),
"apalancamiento bajo"))
cat("</div>")
# 4) Gráficas plotly desde 2020
# Ingresos trimestrales en miles de millones (B USD)
fin plot <- fin %>% filter(per >= as.Date("2020-01-01"))

p_ing <- plot_ly(
  data = fin_plot,
  x = ~per,
  y = ~(ingresos/1000),
  type = "bar",
  marker = list(color = "rgba(17,60,102,0.85)"),
  hovertemplate = paste(
    "Trimestre: %{x|%Y-%m-%d}<br>",
    "Ingresos: %{y:.1f} B USD<extra></extra>"
  )
) %>%
  layout(
    title = list(text = "Ingresos trimestrales (miles de millones de USD)",
x = 0),
    xaxis = list(title = ""),
    yaxis = list(title = "", ticksuffix = " B"),
    bargap = 0.15
  )

# Deuda total, efectivo y deuda neta en miles de millones (B USD)
p_deuda <- plot_ly(fin_plot, x = ~per) %>%
  add_lines(y = ~(deuda/1000), name = "Deuda total",
```

```
            line = list(color = "rgba(17,60,102,0.95)", width = 2),
            hovertemplate = "Trimestre: %{x|%Y-%m-%d}<br>Deuda total:
%{y:.1f} B<extra></extra>") %>%
  add_lines(y = ~(caja/1000), name = "Efectivo",
            line = list(color = "rgba(60,160,110,0.95)", width = 2),
            hovertemplate = "Trimestre: %{x|%Y-%m-%d}<br>Efectivo: %{y:.1f}
B<extra></extra>") %>%
  add lines(y = ~(deuda neta/1000), name = "Deuda neta",
            line = list(color = "rgba(220,90,70,0.95)", width = 2),
            hovertemplate = "Trimestre: %{x|%Y-%m-%d}<br>Deuda neta:
%{y:.1f} B<extra></extra>") %>%
  layout(
    title = list(text = "Deuda total, efectivo y deuda neta (miles de
millones de USD)", x = 0),
    xaxis = list(title = ""),
    yaxis = list(title = "", ticksuffix = " B"),
    legend = list(orientation = "h", x = 0, y = 1.12)
  )

p_ing
p_deuda
# 5) Tabla de ingresos por segmento (trimestre a jul-2026)
seg <- data.frame(
  Segmento = c("Walmart U.S.", "Walmart International", "Sam's Club U.S."),
  Ingresos_MUSD = c(125189, 35198, 25713)
) %>%
  mutate(
    pct_del_total = Ingresos_MUSD / sum(Ingresos_MUSD)
  )

seg
# Tabla HTML simple, alineada al estilo del documento
fmt_musd <- function(x) formatC(x, format = "f", digits = 0, big.mark =
",")

rows <- paste0(
  "<tr>",
  "<td>", seg$Segmento, "</td>",
  "<td style='text-align:right;'>", fmt_musd(seg$Ingresos_MUSD), "</td>",
  "<td style='text-align:right;'>", sprintf("%0.1f%%",
100*seg$pct del total), "</td>",
  "</tr>",
  collapse = ""
)

cat("
<table class='table table-striped' style='max-width:700px;'>
  <thead>
    <tr>
      <th>SEGMENTO</th>
      <th style='text-align:right;'>INGRESOS DEL TRIMESTRE (M USD)</th>
      <th style='text-align:right;'>% DEL TOTAL</th>
    </tr>
  </thead>
  <tbody>
", rows, "
  </tbody>
</table>
")
```

Números que debe mostrar la Sección 2

- Ingresos TTM: $736 B (735,840)
- EBITDA TTM: $45.1 B (45,073), margen EBITDA TTM 6.1%
- Utilidad TTM: $22.5 B, margen neto TTM 3.1%
- Flujo libre TTM: $13.7 B
- Deuda total: $57.2 B
- Deuda neta / EBITDA: 1.01×

[DONE]

1/11/2026 - 4:43:36 p.m.

Sección 2 correcta, números validados (ingresos TTM 735,840, EBITDA 45,073, utilidad 22,499, flujo libre 13,698, deuda 57,243, 1.01x, segmentos 67.3%/18.9%/13.8%). Ajustes mínimos: (1) pon results='asis' en los chunks que usan cat(); (2) en las tarjetas de EBITDA y utilidad deja solo el monto como valor y pasa el margen al pie ("margen EBITDA 6.1%" y "margen neto 3.1%"). Sigue con la Sección 3: Gasto financiero. Datos: columnas interes (intereses sobre deuda, millones de USD por trimestre) e interes_leasing del fundamentals.csv; el gasto esperado según la estructura de deuda de bonos.xlsx es notas $493.7 M + papel comercial $104.8 M + arrendamientos $126 M = $724.4 M por trimestre, y el último trimestre reportado por LSEG (jul-2026) muestra 263 porque solo suma deuda y arrendamientos netos de un efecto contable de -$0.5 mil millones. Muestra una tabla de reconciliación y una gráfica de línea con el gasto reportado contra el esperado.

ChatGPT (GPT 5.2) •1/11/2026 - 4:43:37 p.m.

Ajustes mínimos a la Sección 2

```
# 3) Seis tarjetas (tile() ya existe del estilo del dashboard)

fmt_money_b_from_musd <- function(x_musd, digits = 1){
  paste0("$", formatC(x_musd/1000, format = "f", digits = digits), " B")
}
fmt_pct <- function(x, digits = 1){
  paste0(formatC(100*x, format = "f", digits = digits), "%")
}
fmt_x <- function(x, digits = 2){
  paste0(formatC(x, format = "f", digits = digits), "×")
}

cat("<div class='kpi-grid'>")
cat(tile("Ingresos TTM", fmt_money_b_from_musd(ing_ttm, 0), "12 meses a
jul-2026"))
```

```
cat(tile("EBITDA TTM", fmt_money_b_from_musd(ebitda_ttm, 1), paste0("margen
EBITDA ", fmt_pct(margen_ebitda_ttm, 1))))
cat(tile("Utilidad TTM", fmt money b from musd(util ttm, 1), paste0("margen
neto ", fmt_pct(margen_neto_ttm, 1))))
cat(tile("Flujo libre TTM", fmt_money_b_from_musd(fcf_ttm, 1), "operativo
menos capex"))
cat(tile("Deuda total", fmt_money_b_from_musd(deuda_total, 1), "incluye
arrendamientos"))
cat(tile("Deuda neta / EBITDA", fmt_x(deuda_neta_ebitda, 2),
"apalancamiento bajo"))
cat("</div>")
# Tabla HTML simple, alineada al estilo del documento
fmt musd <- function(x) formatC(x, format = "f", digits = 0, big.mark =
",")

rows <- paste0(
  "<tr>",
  "<td>", seg$Segmento, "</td>",
  "<td style='text-align:right;'>", fmt_musd(seg$Ingresos_MUSD), "</td>",
  "<td style='text-align:right;'>", sprintf("%0.1f%%",
100*seg$pct_del_total), "</td>",
  "</tr>",
  collapse = ""
)

cat("
<table class='table table-striped' style='max-width:700px;'>
  <thead>
    <tr>
      <th>SEGMENTO</th>
      <th style='text-align:right;'>INGRESOS DEL TRIMESTRE (M USD)</th>
      <th style='text-align:right;'>% DEL TOTAL</th>
    </tr>
  </thead>
  <tbody>
", rows, "
  </tbody>
</table>
")
```

---

3. Gasto financiero: ¿cuadran los números?

En el primer avance quedó una duda: LSEG reporta 263 M de gasto por intereses en el trimestre a jul-2026, contra un promedio cercano a 715 M en los cuatro trimestres anteriores, mientras la deuda sube. Para saber si es un dato real o un error de captura, reconstruimos cuánto debería costar la deuda vigente a partir de la estructura de emisiones y lo comparamos contra lo reportado.

```
# Datos: fundamentals.csv (interes e interes leasing, M USD por trimestre)

fin <- read.csv("fundamentals.csv", stringsAsFactors = FALSE)
fin$per <- as.Date(fin$per)
fin <- fin %>% arrange(per)

stopifnot(all(c("interes","interes_leasing") %in% names(fin)))
```

```r
# Serie reportada por LSEG (gasto por intereses total aproximado)
# En tu narrativa: jul-2026 sale bajo por un efecto contable de -0.5 B.
fin <- fin %>%
  mutate(
    gasto_reportado = interes + interes_leasing
  )

# Trimestre a validar (corte del dashboard para esta sección)
per_corte <- as.Date("2026-07-31")

# Cifras esperadas según estructura (M USD por trimestre), dadas por
ustedes
exp notas <- 493.7
exp_cp    <- 104.8
exp_leas  <- 126.0
exp_total <- exp_notas + exp_cp + exp_leas

# Efecto contable en el último trimestre (M USD)
efecto_contable <- -500

# Reportado en el trimestre a jul-2026
rep_corte <- fin %>% filter(per == per_corte) %>% slice(1)
rep_interes <- rep_corte$interes
rep_leasing <- rep_corte$interes_leasing
rep_total   <- rep_corte$gasto_reportado

# Tabla de reconciliación (en M USD por trimestre)
recon <- data.frame(
  Concepto = c(
    "Cupones de las notas (monto vigente × cupón)",
    "Papel comercial y notas por pagar",
    "Intereses de arrendamientos financieros",
    "Total esperado",
    "Gasto reportado por LSEG (deuda + arrendamientos)",
    "Efecto contable neto (jul-2026)",
    "Total reportado ajustado (referencia)"
  ),
  `M_USD_por_trimestre` = c(
    exp_notas,
    exp_cp,
    exp leas,
    exp_total,
    rep_total,
    efecto_contable,
    rep total - efecto contable
  )
)

recon
# Tabla HTML con estilo similar al dashboard
fmt m <- function(x) {
  s <- formatC(x, format = "f", digits = 1, big.mark = ",")
  s
}

rows <- paste0(
  "<tr>",
  "<td>", recon$Concepto, "</td>",
  "<td style='text-align:right;'>", fmt_m(recon$M_USD_por_trimestre),
"</td>",
```

```
  "</tr>",
  collapse = ""
)

cat("
<table class='table table-striped' style='max-width:820px;'>
  <thead>
    <tr>
      <th>CONCEPTO</th>
      <th style='text-align:right;'>M USD POR TRIMESTRE</th>
    </tr>
  </thead>
  <tbody>
", rows, "
  </tbody>
</table>
")
# Gráfica: gasto reportado vs esperado según estructura

# Línea esperada plana (724.4) para todo el periodo visible
fin_plot <- fin %>% filter(per >= as.Date("2020-01-01"))

p_gasto <- plot_ly(fin_plot, x = ~per) %>%
  add_lines(
    y = ~gasto_reportado,
    name = "Reportado por LSEG (deuda + arrendamientos)",
    line = list(color = "rgba(17,60,102,0.95)", width = 2),
    hovertemplate = "Trimestre: %{x|%Y-%m-%d}<br>Reportado: %{y:.0f}
M<extra></extra>"
  ) %>%
  add_lines(
    y = rep(exp_total, nrow(fin_plot)),
    name = "Esperado según la estructura",
    line = list(color = "rgba(220,90,70,0.95)", width = 2, dash = "dash"),
    hovertemplate = paste0("Esperado: ", round(exp_total,1), "
M<extra></extra>")
  ) %>%
  add_markers(
    data = fin_plot %>% filter(per == per_corte),
    y = ~gasto_reportado,
    name = "Jul-2026 (dato a validar)",
    marker = list(color = "rgba(220,90,70,1)", size = 9),
    hovertemplate = "Jul-2026: %{y:.0f} M<extra></extra>"
  ) %>%
  layout(
    title = list(text = "Gasto por intereses por trimestre (M USD):
reportado vs esperado", x = 0),
    xaxis = list(title = ""),
    yaxis = list(title = "", tickformat = ",.0f"),
    legend = list(orientation = "h", x = 0, y = 1.12)
  )

p_gasto
```

Números que debe mostrar la Sección 3

- Esperado según estructura: 724.4 M por trimestre
  Notas 493.7 M, papel comercial 104.8 M, arrendamientos 126.0 M

- Reportado por LSEG en jul-2026: 263 M (deuda + arrendamientos)
- Efecto contable neto jul-2026: -500 M
- Reportado ajustado (referencia): 763 M

[DONE]

1/11/2026 - 4:44:34 p.m.

Sección 3: hay un error. La columna interes de fundamentals.csv ya es el gasto reportado por LSEG (deuda más arrendamientos): en jul-2026, 137 de deuda + 126 de arrendamientos = 263. No sumes interes_leasing otra vez. Define gasto_reportado = interes. Con eso debe dar: esperado 724.4 M por trimestre (notas 493.7, papel comercial 104.8, arrendamientos 126.0); reportado jul-2026 263 M; efecto contable -500 M (reducción de gasto por beneficios fiscales no reconocidos, 10-Q, cifra redonda de 0.5 mil millones); reportado sin ese efecto 763 M, que queda +5.3% sobre el esperado; promedio de los cuatro trimestres previos 715 M. En la tabla de reconciliación, "Reportado sin efecto contable" = rep_total - efecto_contable = 263 - (-500) = 763, y agrega una fila "Diferencia contra esperado" con 763 / 724.4 - 1 = +5.3%. Quita de los comentarios del código frases como "dadas por ustedes" o "en tu narrativa". Agrega results='asis' donde uses cat(). Escribe en la nota final: "El dato bajo de jul-2026 es un efecto contable puntual y no un cambio en el costo de la deuda". Entrega solo la Sección 3 corregida y sigue con la Sección 4: Escalera de vencimientos.



ChatGPT (GPT 5.2) •1/11/2026 - 4:44:34 p.m.

3. Gasto financiero: ¿cuadran los números?

En el primer avance quedó una duda: LSEG reporta 263 M de gasto por intereses en el trimestre a jul-2026, contra un promedio de 715 M en los cuatro trimestres anteriores, mientras la deuda sube. Para validar el dato, reconstruimos cuánto debería costar la deuda vigente a partir de la estructura de emisiones y lo comparamos contra lo reportado.

```
# Datos: fundamentals.csv (interes = gasto reportado por LSEG, M USD por
trimestre)

fin <- read.csv("fundamentals.csv", stringsAsFactors = FALSE)
fin$per <- as.Date(fin$per)
fin <- fin %>% arrange(per)

stopifnot(all(c("per","interes") %in% names(fin)))

# Serie reportada por LSEG
fin <- fin %>% mutate(gasto_reportado = interes)

# Trimestre de corte
per_corte <- as.Date("2026-07-31")
```

```r
# Promedio de los cuatro trimestres previos
prom_4_prev <- fin %>%
  filter(per < per_corte) %>%
  tail(4) %>%
  summarise(prom = mean(gasto_reportado, na.rm = TRUE)) %>%
  pull(prom)

# Gasto esperado según estructura (M USD por trimestre)
exp_notas <- 493.7
exp_cp    <- 104.8
exp_leas  <- 126.0
exp_total <- exp_notas + exp_cp + exp_leas

# Efecto contable puntual en jul-2026 (M USD)
efecto_contable <- -500

# Reportado en jul-2026
rep_corte <- fin %>% filter(per == per_corte) %>% slice(1)
rep_total <- rep_corte$gasto_reportado

rep_sin_efecto <- rep_total - efecto_contable
dif_pct_vs_esp <- rep_sin_efecto / exp_total - 1

# Tabla de reconciliación (M USD por trimestre)
recon <- data.frame(
  Concepto = c(
    "Cupones de las notas (monto vigente × cupón)",
    "Papel comercial y notas por pagar",
    "Intereses de arrendamientos financieros",
    "Total esperado",
    "Gasto reportado por LSEG (jul-2026)",
    "Efecto contable puntual (jul-2026, 10-Q)",
    "Reportado sin efecto contable",
    "Diferencia contra esperado"
  ),
  Valor = c(
    exp_notas,
    exp_cp,
    exp_leas,
    exp_total,
    rep_total,
    efecto_contable,
    rep_sin_efecto,
    dif_pct_vs_esp
  ),
  Unidad = c(
    rep("M USD por trimestre", 7),
    "Porcentaje"
  )
)

list(
  esperado = exp_total,
  reportado_jul_2026 = rep_total,
  reportado_sin_efecto = rep_sin_efecto,
  diferencia_pct_vs_esperado = dif_pct_vs_esp,
  promedio_4_trimestres_previos = prom_4_prev,
  tabla = recon
)
```

```
# Tabla HTML
fmt_m <- function(x) formatC(x, format = "f", digits = 1, big.mark = ",")
fmt_pct <- function(x) paste0(ifelse(x >= 0, "+", ""), formatC(100*x,
format = "f", digits = 1), "%")

rows <- paste0(
  "<tr>",
  "<td>", recon$Concepto, "</td>",
  "<td style='text-align:right;'>",
    ifelse(recon$Unidad == "Porcentaje", fmt_pct(recon$Valor),
fmt_m(recon$Valor)),
  "</td>",
  "</tr>",
  collapse = ""
)

cat("
<table class='table table-striped' style='max-width:860px;'>
  <thead>
    <tr>
      <th>CONCEPTO</th>
      <th style='text-align:right;'>VALOR</th>
    </tr>
  </thead>
  <tbody>
", rows, "
  </tbody>
</table>
")
# Gráfica: gasto reportado vs esperado según estructura

fin_plot <- fin %>% filter(per >= as.Date("2020-01-01"))

p_gasto <- plot_ly(fin_plot, x = ~per) %>%
  add_lines(
    y = ~gasto reportado,
    name = "Reportado por LSEG",
    line = list(color = "rgba(17,60,102,0.95)", width = 2),
    hovertemplate = "Trimestre: %{x|%Y-%m-%d}<br>Reportado: %{y:.0f}
M<extra></extra>"
  ) %>%
  add_lines(
    y = rep(exp_total, nrow(fin_plot)),
    name = "Esperado según estructura (724.4 M)",
    line = list(color = "rgba(220,90,70,0.95)", width = 2, dash = "dash"),
    hovertemplate = "Esperado: 724.4 M<extra></extra>"
  ) %>%
  add_markers(
    data = fin_plot %>% filter(per == per_corte),
    y = ~gasto reportado,
    name = "Jul-2026",
    marker = list(color = "rgba(220,90,70,1)", size = 9),
    hovertemplate = "Jul-2026: %{y:.0f} M<extra></extra>"
  ) %>%
  layout(
    title = list(text = "Gasto por intereses por trimestre (M USD):
reportado vs esperado", x = 0),
    xaxis = list(title = ""),
    yaxis = list(title = "", tickformat = ",.0f"),
    legend = list(orientation = "h", x = 0, y = 1.12)
```

```
  )

p_gasto
```

Nota: El dato bajo de jul-2026 es un efecto contable puntual y no un cambio en el costo de la deuda.

Números que debe mostrar la Sección 3

- Esperado según estructura: 724.4 M por trimestre
  Notas 493.7 M, papel comercial 104.8 M, arrendamientos 126.0 M
- Reportado por LSEG en jul-2026: 263 M
- Efecto contable puntual (10-Q): -500 M
- Reportado sin efecto contable: 763 M
- Diferencia contra esperado: +5.3%
- Promedio de los cuatro trimestres previos: 715 M

---

4. Escalera de vencimientos

Perfil de amortización por año y divisa, en millones de dólares equivalentes. No hay concentración peligrosa: el máximo anual es inferior al 12% del total y la vida promedio es de 9.4 años.

```
# Escalera de vencimientos (base)
# Requiere columnas de moneda en bonos.xlsx. En el chunk de lectura de
bonos,
# b_raw contiene todo el Excel; aquí armamos el dataset con currency y
maturity.

# Validación: currency debe existir en b raw
if (!("currency" %in% names(b_raw))) stop("bonos.xlsx (Bonds) debe incluir
columna currency.")

ladder <- b raw %>%
  filter(coupon class %in% c("Fixed Coupon", "Floating Coupon")) %>%
  transmute(
    isin = isin,
    currency = toupper(currency),
    outstanding = suppressWarnings(as.numeric(amount outstanding)),
    maturity date = as.Date(suppressWarnings(as.numeric(maturity date)),
origin = "1899-12-30")
  ) %>%
  filter(!is.na(outstanding), outstanding > 0,
         !is.na(maturity date), maturity date > cutoff date) %>%
  mutate(maturity_year = lubridate::year(maturity_date))

# Para reproducir "USD equivalentes", se requiere FX por moneda al corte.
# Si tu bonos.xlsx ya trae outstanding en USD equivalentes, esto queda
listo.
# Si no, dime dónde están los FX (o si ya existe una columna
usd_equivalent_outstanding).
```

```r
ladder_sum <- ladder %>%
  group_by(maturity_year, currency) %>%
  summarise(monto = sum(outstanding, na.rm = TRUE), .groups = "drop") %>%
  arrange(maturity_year)

ladder_sum
# Gráfica: barras apiladas por año y divisa
p ladder <- plot ly(
  ladder_sum,
  x = ~maturity_year,
  y = ~monto,
  color = ~currency,
  type = "bar",
  hovertemplate = "Año: %{x}<br>Moneda: %{legendgroup}<br>Monto:
%{y:,.0f}<extra></extra>"
) %>%
  layout(
    barmode = "stack",
    title = list(text = "Escalera de vencimientos por año y divisa (USD
equivalentes)", x = 0),
    xaxis = list(title = ""),
    yaxis = list(title = "Monto", tickformat = ",.0f"),
    legend = list(orientation = "h", x = 0, y = 1.12)
  )

p_ladder
```

[DONE]

1/11/2026 - 4:45:46 p.m.

Sección 3 aprobada. Sección 4, correcciones: (1) amount_outstanding ya está en USD equivalentes, no hay que convertir FX. (2) Agrupa la moneda así antes de graficar: "US DOLLAR" a "USD", "BRITISH POUND" a "GBP", "CANADIAN DOLLAR" a "CAD" y el resto (EURO, JAPANESE YEN, INDIAN RUPEE) a "Otras"; colores fijos USD #2a78d6, GBP #eb6834, CAD #1baf7a, Otras #898781, y orden de apilado USD, GBP, CAD, Otras. (3) Monto en miles de millones: monto/1e9, eje con ticksuffix " B". (4) En hovertemplate usa %{fullData.name} en lugar de %{legendgroup}, y muestra "Año: %{x}
%{fullData.name}: $%{y:.2f} B". (5) Agrega debajo de la gráfica una tabla resumen por grupo con monto en B y % del total, y un cálculo con código del año de mayor vencimiento y su % del total. Resultados esperados: USD $37.2 B (81.1%), GBP $3.6 B (7.8%), CAD $3.6 B (7.8%), Otras $1.5 B (3.3%); año de mayor vencimiento 2033 con $5.18 B (11.3%); total $45.9 B. Calcula en código el "máximo anual inferior a 12%" y la vida promedio de 9.4 años del chunk anterior en lugar de escribirlos fijos en el texto. Sigue con la Sección 5: Composición por divisa.



ChatGPT (GPT 5.2) •1/11/2026 - 4:45:46 p.m.

4. Escalera de vencimientos

Perfil de amortización por año y divisa, en miles de millones de USD equivalentes. El máximo anual es inferior a 12% del total y la vida promedio es de 9.4 años.

```
# Base de vencimientos: amount outstanding ya está en USD equivalentes
stopifnot("currency" %in% names(b raw))

ladder <- b_raw %>%
  filter(coupon_class %in% c("Fixed Coupon", "Floating Coupon")) %>%
  transmute(
    isin = isin,
    currency_raw = toupper(currency),
    outstanding = suppressWarnings(as.numeric(amount_outstanding)),
    maturity_date = as.Date(suppressWarnings(as.numeric(maturity_date)),
origin = "1899-12-30")
  ) %>%
  filter(!is.na(outstanding), outstanding > 0,
         !is.na(maturity_date), maturity_date > cutoff_date) %>%
  mutate(
    currency group = case when(
      currency_raw == "US DOLLAR" ~ "USD",
      currency_raw == "BRITISH POUND" ~ "GBP",
      currency_raw == "CANADIAN DOLLAR" ~ "CAD",
      TRUE ~ "Otras"
    ),
    maturity_year = lubridate::year(maturity_date)
  )

# Agregado por año y grupo de divisa (en B USD)
ladder sum <- ladder %>%
  group_by(maturity_year, currency_group) %>%
  summarise(monto_b = sum(outstanding, na.rm = TRUE) / 1e9, .groups =
"drop") %>%
  arrange(maturity year)

# Totales y resumen por divisa (en B USD)
total_b <- sum(ladder_sum$monto_b, na.rm = TRUE)

fx_summary <- ladder_sum %>%
  group_by(currency_group) %>%
  summarise(
    monto_b = sum(monto_b, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  mutate(pct_total = monto_b / sum(monto_b, na.rm = TRUE)) %>%
  arrange(match(currency_group, c("USD","GBP","CAD","Otras")))

# Año con mayor vencimiento
by_year <- ladder_sum %>%
  group_by(maturity_year) %>%
  summarise(monto_b = sum(monto_b, na.rm = TRUE), .groups = "drop") %>%
  arrange(desc(monto_b))

max_year <- by_year$maturity_year[1]
max_amt_b <- by_year$monto_b[1]
max_pct <- max_amt_b / total_b
```

```
list(
  total_b = total_b,
  resumen divisa = fx summary,
  max_year = max_year,
  max_amt_b = max_amt_b,
  max_pct = max_pct
)
# Gráfica: barras apiladas por año y grupo de divisa, con colores fijos y
orden fijo

# Forzar orden de apilado
ladder_sum$currency_group <- factor(ladder_sum$currency_group, levels =
c("USD","GBP","CAD","Otras"))

cols <- c(USD = "#2a78d6", GBP = "#eb6834", CAD = "#1baf7a", Otras =
"#898781")

p ladder <- plot ly(
  data = ladder_sum,
  x = ~maturity_year,
  y = ~monto_b,
  color = ~currency_group,
  colors = cols,
  type = "bar",
  hovertemplate = "Año: %{x}<br>%{fullData.name}: $%{y:.2f}
B<extra></extra>"
) %>%
  layout(
    barmode = "stack",
    title = list(text = "Escalera de vencimientos por año y divisa (USD
equivalentes)", x = 0),
    xaxis = list(title = ""),
    yaxis = list(title = "", ticksuffix = " B"),
    legend = list(orientation = "h", x = 0, y = 1.12)
  )

p ladder
# Tabla resumen por grupo (B USD y %)
fmt_b <- function(x) paste0("$", formatC(x, format = "f", digits = 1), "
B")
fmt pct <- function(x) paste0(formatC(100*x, format = "f", digits = 1),
"%")

rows <- paste0(
  "<tr>",
  "<td>", fx summary$currency group, "</td>",
  "<td style='text-align:right;'>", fmt_b(fx_summary$monto_b), "</td>",
  "<td style='text-align:right;'>", fmt_pct(fx_summary$pct_total), "</td>",
  "</tr>",
  collapse = ""
)

cat("
<table class='table table-striped' style='max-width:520px;'>
  <thead>
    <tr>
      <th>DIVISA</th>
      <th style='text-align:right;'>MONTO (B USD)</th>
      <th style='text-align:right;'>% DEL TOTAL</th>
    </tr>
```

```
  </thead>
  <tbody>
", rows, "
  </tbody>
</table>
")
# Cálculo del máximo anual y su porcentaje, y referencia de vida promedio
(de la Sección 1)
# vida_prom (años) ya está calculada en la Sección 1 como vida_prom

list(
  total_deuda_b = total_b,
  anio mayor vencimiento = max year,
  monto_mayor_vencimiento_b = max_amt_b,
  pct_total_anio_mayor = max_pct,
  max_anual_inferior_12pct = (max_pct < 0.12),
  vida_promedio_anios = vida_prom
)
```

Números que debe mostrar la Sección 4

- Total: $45.9 B
- Resumen por divisa:
    - USD $37.2 B (81.1%)
    - GBP $3.6 B (7.8%)
    - CAD $3.6 B (7.8%)
    - Otras $1.5 B (3.3%)
- Año de mayor vencimiento: 2033 con $5.18 B (11.3%)
- Máximo anual inferior a 12%: TRUE
- Vida promedio: 9.4 años

---

5. Composición por divisa

La escalera muestra el calendario de vencimientos; aquí resumimos la composición total por divisa para dimensionar la exposición financiera a monedas distintas del USD.

```
# Resumen total por currency_group (B USD)
fx_total <- ladder %>%
  group_by(currency_group) %>%
  summarise(
    monto b = sum(outstanding, na.rm = TRUE) / 1e9,
    .groups = "drop"
  ) %>%
  mutate(pct_total = monto_b / sum(monto_b, na.rm = TRUE)) %>%
  arrange(match(currency group, c("USD","GBP","CAD","Otras")))

fx_total
# Gráfica de composición por divisa (barras horizontales para que se lea el
%)
fx total$currency group <- factor(fx total$currency group, levels =
c("USD","GBP","CAD","Otras"))
```

```
p_fx <- plot_ly(
  fx_total,
  x = ~monto_b,
  y = ~currency_group,
  type = "bar",
  orientation = "h",
  marker = list(color =
unname(cols[as.character(fx_total$currency_group)])),
  hovertemplate = "%{y}: $%{x:.2f} B<br>% del total:
%{customdata:.1f}%<extra></extra>",
  customdata = ~(100*pct_total)
) %>%
  layout(
    title = list(text = "Composición de deuda por divisa (USD
equivalentes)", x = 0),
    xaxis = list(title = "", ticksuffix = " B"),
    yaxis = list(title = "")
  )

p_fx
```

[DONE]

1/11/2026 - 4:47:43 p.m.

Sección 4 aprobada, con dos ajustes: (a) el texto de arriba debe usar valores calculados (inline `r ...`) para el máximo anual y la vida promedio, no 12% y 9.4 escritos a mano; (b) el chunk con cat() de la tabla debe llevar results='asis'. Sección 5, correcciones: (1) en p_fx agrega yaxis = list(title = "", autorange = "reversed") para que USD quede arriba y el orden sea USD, GBP, CAD, Otras. (2) Calcula con código no_usd_b = suma de GBP, CAD y Otras en B USD, y no_usd_pct = no_usd_b / total. Muestra una tarjeta tile("Deuda en divisas distintas al USD", paste0("$", formatC(no_usd_b, format="f", digits=2), " B"), paste0(formatC(100*no_usd_pct, format="f", digits=1), "% del total")) dentro de . (3) Agrega una tabla HTML por divisa (con results='asis') con monto en B, % del total y número de emisiones (n_distinct(isin)). (4) El texto de introducción debe usar los valores calculados, no escritos a mano. Números esperados: no-USD $8.67 B (18.9%); GBP $3.6 B (7.8%), CAD $3.6 B (7.8%), Otras $1.5 B (3.3%); USD $37.2 B (81.1%). Después sigue con la Sección 6: Emisión del 23-oct-2026.

ChatGPT (GPT 5.2) •1/11/2026 - 4:47:43 p.m.

```
# Tabla resumen por grupo (B USD y %)
fmt_b <- function(x) paste0("$", formatC(x, format = "f", digits = 1), "
B")
fmt_pct <- function(x) paste0(formatC(100*x, format = "f", digits = 1),
"%")

rows <- paste0(
  "<tr>",
```

```
  "<td>", fx_summary$currency_group, "</td>",
  "<td style='text-align:right;'>", fmt_b(fx_summary$monto_b), "</td>",
  "<td style='text-align:right;'>", fmt_pct(fx_summary$pct_total), "</td>",
  "</tr>",
  collapse = ""
)

cat("
<table class='table table-striped' style='max-width:520px;'>
  <thead>
    <tr>
      <th>DIVISA</th>
      <th style='text-align:right;'>MONTO (B USD)</th>
      <th style='text-align:right;'>% DEL TOTAL</th>
    </tr>
  </thead>
  <tbody>
", rows, "
  </tbody>
</table>
")
```

5. Composición por divisa

La deuda en monedas distintas al USD suma `r paste0("$", formatC(no_usd_b, format="f", digits=2), " B")` y representa `r paste0(formatC(100*no_usd_pct, format="f", digits=1), "%")` del total, por lo que el riesgo cambiario relevante está concentrado en GBP y CAD.

```
# Totales por grupo de divisa (B USD) y conteo de emisiones
fx_total <- ladder %>%
  group_by(currency_group) %>%
  summarise(
    monto_b = sum(outstanding, na.rm = TRUE) / 1e9,
    n_emisiones = dplyr::n_distinct(isin),
    .groups = "drop"
  ) %>%
  mutate(pct_total = monto_b / sum(monto_b, na.rm = TRUE)) %>%
  arrange(match(currency_group, c("USD","GBP","CAD","Otras")))

total_b <- sum(fx_total$monto_b, na.rm = TRUE)
no_usd_b <- sum(fx_total$monto_b[fx_total$currency_group != "USD"], na.rm =
TRUE)
no_usd_pct <- no_usd_b / total_b

list(
  total_b = total_b,
  no_usd_b = no_usd_b,
  no_usd_pct = no_usd_pct,
  fx_total = fx_total
)
# Tarjeta: deuda en divisas distintas al USD
cat("<div class='kpi-grid'>")
cat(tile(
  "Deuda en divisas distintas al USD",
  paste0("$", formatC(no_usd_b, format = "f", digits = 2), " B"),
  paste0(formatC(100*no_usd_pct, format = "f", digits = 1), "% del total")
```

```
))
cat("</div>")
# Barras horizontales con USD arriba
fx_total$currency_group <- factor(fx_total$currency_group, levels =
c("USD","GBP","CAD","Otras"))

p_fx <- plot_ly(
  fx total,
  x = ~monto_b,
  y = ~currency_group,
  type = "bar",
  orientation = "h",
  marker = list(color =
unname(cols[as.character(fx_total$currency_group)])),
  hovertemplate = "%{y}: $%{x:.2f} B<br>% del total:
%{customdata:.1f}%<extra></extra>",
  customdata = ~(100*pct_total)
) %>%
  layout(
    title = list(text = "Composición de deuda por divisa (USD
equivalentes)", x = 0),
    xaxis = list(title = "", ticksuffix = " B"),
    yaxis = list(title = "", autorange = "reversed")
  )

p_fx
# Tabla HTML por divisa: monto, % del total y número de emisiones
fmt_b2 <- function(x) paste0("$", formatC(x, format = "f", digits = 1), "
B")
fmt_pct2 <- function(x) paste0(formatC(100*x, format = "f", digits = 1),
"%")

rows <- paste0(
  "<tr>",
  "<td>", fx_total$currency_group, "</td>",
  "<td style='text-align:right;'>", fmt b2(fx total$monto b), "</td>",
  "<td style='text-align:right;'>", fmt pct2(fx total$pct total), "</td>",
  "<td style='text-align:right;'>", fx_total$n_emisiones, "</td>",
  "</tr>",
  collapse = ""
)

cat("
<table class='table table-striped' style='max-width:620px;'>
  <thead>
    <tr>
      <th>DIVISA</th>
      <th style='text-align:right;'>MONTO (B USD)</th>
      <th style='text-align:right;'>% DEL TOTAL</th>
      <th style='text-align:right;'>EMISIONES</th>
    </tr>
  </thead>
  <tbody>
", rows, "
  </tbody>
</table>
")
```

Números esperados para validar la Sección 5

- No USD: $8.67 B (18.9% del total)
- USD: $37.2 B (81.1%)
- GBP: $3.6 B (7.8%)
- CAD: $3.6 B (7.8%)
- Otras: $1.5 B (3.3%)

---

6. Emisión del 23-oct-2026

Esta sección resume la colocación del 23 de octubre de 2026 (8 tramos, USD y CAD), su objetivo de refinanciamiento y el punto de partida de las curvas usadas en el resto del análisis.

```
# En el siguiente paso necesito el archivo/tabla donde están los 8 tramos de la emisión
# (monto, moneda, vencimiento, cupón, precio/yield al colocar) para replicar la sección 6.
# Cuando lo confirmes, lo conecto a bonos.xlsx o al Excel de emisiones de LSEG.
```

[DONE]

1/11/2026 - 4:49:37 p.m.

Sección 5 aprobada con un ajuste: el párrafo de introducción usa no_usd_b y no_usd_pct antes de que se calculen. Mueve ese cálculo a un chunk con include=FALSE colocado ANTES del párrafo (lo mismo para max_pct y vida_prom en la introducción de la Sección 4). Verifica también que todo el código inline lleve backticks: `r ...`. Sección 6: no necesitas otro archivo, los 8 tramos están en b_raw (bonos.xlsx, hoja Bonds). Filtra las filas con coupon_class Fixed Coupon o Floating Coupon y issue_date igual a 2026-10-23 (issue_date viene como fecha serial de Excel en texto: as.Date(as.numeric(issue_date), origin = "1899-12-30")). Debes obtener exactamente 8 filas: 4 en US DOLLAR y 4 en CANADIAN DOLLAR. Muestra: (1) una frase con el total emitido calculado en código ($6.58 B en USD equivalentes, 8 tramos, 4 en USD y 4 en CAD; C$5,000 M en total), (2) una tabla con cupón, vencimiento, moneda, monto USD-eq (M), YTM y precio, ordenada por vencimiento y moneda; no incluyas Z-spread ni OAS, (3) dos tarjetas con tile(): tramos en USD ($3.00 B, cupón ponderado 5.29%) y tramos en CAD ($3.58 B, cupón ponderado 4.54%), con cupón ponderado por monto. Todo debe salir de b_raw, sin números escritos a mano. Después sigue con la Sección 7: Curva de rendimiento.



ChatGPT (GPT 5.2) •1/11/2026 - 4:49:37 p.m.

Ajuste previo para Sección 4 (cálculos antes del texto, para inline)

```
# Variables para el texto de la Sección 4 (deben existir ladder_sum,
total_b, max_year, max_amt_b, max_pct y vida_prom)
# Si vienes de los chunks aprobados de la Sección 4, esto ya está
calculado.
# Por seguridad, recalculamos max_year/max_pct desde ladder_sum.

by_year <- ladder_sum %>%
  group_by(maturity_year) %>%
  summarise(monto_b = sum(monto_b, na.rm = TRUE), .groups = "drop") %>%
  arrange(desc(monto_b))

max_year <- by_year$maturity_year[1]
max_amt_b <- by_year$monto_b[1]
max_pct <- max_amt_b / total_b
```

En tu texto de introducción de la Sección 4 usa inline así (ejemplo):

Perfil de amortización por año y divisa, en miles de millones de USD equivalentes. El año de mayor vencimiento es `r max_year` con `r paste0("$", formatC(max_amt_b, format="f", digits=2), " B")` (`r paste0(formatC(100*max_pct, format="f", digits=1), "%")` del total) y la vida promedio es de `r formatC(vida_prom, format="f", digits=1)` años.

---

6. Emisión del 23-oct-2026

```
# Filtrar los 8 tramos emitidos el 23-oct-2026 desde b_raw (bonos.xlsx,
hoja Bonds)
# issue_date viene como serial de Excel en texto

stopifnot(all(c("coupon class","issue date","currency","amount outstanding"
,"maturity_date",
                "coupon","yield","last_price") %in% names(b_raw)))

issue_tbl <- b_raw %>%
  filter(coupon class %in% c("Fixed Coupon", "Floating Coupon")) %>%
  transmute(
    isin = isin,
    currency_raw = toupper(currency),
    issue_date = as.Date(suppressWarnings(as.numeric(issue_date)), origin =
"1899-12-30"),
    maturity_date = as.Date(suppressWarnings(as.numeric(maturity_date)),
origin = "1899-12-30"),
    outstanding_usd = suppressWarnings(as.numeric(amount_outstanding)),   #
ya en USD equivalentes
    coupon = suppressWarnings(as.numeric(coupon))/100,                    #
% -> decimal
    ytm = suppressWarnings(as.numeric(yield))/100,                        #
% -> decimal
    price = suppressWarnings(as.numeric(last_price))
  ) %>%
  filter(!is.na(issue_date), issue_date == as.Date("2026-10-23")) %>%
```

```
  filter(!is.na(outstanding_usd), outstanding_usd > 0)

# Validación: exactamente 8 filas, 4 USD y 4 CAD
n_tramos <- nrow(issue_tbl)
n_usd <- sum(issue_tbl$currency_raw == "US DOLLAR", na.rm = TRUE)
n_cad <- sum(issue_tbl$currency_raw == "CANADIAN DOLLAR", na.rm = TRUE)

# Totales y cupones ponderados por monto
total_emitido_b <- sum(issue_tbl$outstanding_usd, na.rm = TRUE) / 1e9

usd_tbl <- issue_tbl %>% filter(currency_raw == "US DOLLAR")
cad_tbl <- issue_tbl %>% filter(currency_raw == "CANADIAN DOLLAR")

usd_total_b <- sum(usd_tbl$outstanding_usd, na.rm = TRUE) / 1e9
cad_total_b <- sum(cad_tbl$outstanding_usd, na.rm = TRUE) / 1e9

usd_coupon_w <- weighted.mean(usd_tbl$coupon, w = usd_tbl$outstanding_usd,
na.rm = TRUE)
cad_coupon_w <- weighted.mean(cad_tbl$coupon, w = cad_tbl$outstanding_usd,
na.rm = TRUE)

# Total en CAD (debe salir de b_raw, no escrito a mano)
# Buscamos una columna de monto en moneda local si existe
local_candidates <-
c("amount_outstanding_local","amount_local","amount_outstanding_ccy","amoun
t_ccy","amount_issued_local")
local_col <- intersect(names(b_raw), local_candidates)[1]

cad_total_local_m <- NA_real_
if (!is.na(local_col)) {
  cad_total_local_m <- b_raw %>%
    filter(coupon_class %in% c("Fixed Coupon", "Floating Coupon")) %>%
    transmute(
      issue_date = as.Date(suppressWarnings(as.numeric(issue_date)), origin
= "1899-12-30"),
      currency_raw = toupper(currency),
      amt_local = suppressWarnings(as.numeric(.data[[local_col]]))
    ) %>%
    filter(!is.na(issue_date), issue_date == as.Date("2026-10-23"),
           currency_raw == "CANADIAN DOLLAR",
           !is.na(amt_local), amt_local > 0) %>%
    summarise(total_m = sum(amt_local, na.rm = TRUE)/1e6) %>%
    pull(total_m)
}

stopifnot(n_tramos == 8, n_usd == 4, n_cad == 4)

# Tabla final para mostrar (monto en USD-eq M)
issue_show <- issue_tbl %>%
  mutate(
    currency = ifelse(currency_raw == "US DOLLAR", "USD",
                ifelse(currency_raw == "CANADIAN DOLLAR", "CAD",
currency_raw)),
    amount_usd_m = outstanding_usd / 1e6
  ) %>%
  arrange(maturity_date, currency)
```

La emisión del `r format(as.Date("2026-10-23"), "%d-%b-%Y")` suma `r paste0("$", formatC(total_emitido_b, format="f", digits=2), " B")` en USD equivalentes, en `r`

`n_tramos` tramos: `r n_usd` en USD y `r n_cad` en CAD, por un total de `r ifelse(is.na(cad_total_local_m), "C$ NA", paste0("C$", formatC(cad_total_local_m, format="f", digits=0, big.mark=","), " M"))`.

```
fmt_money_b <- function(x_b, digits = 2) paste0("$", formatC(x_b, format =
"f", digits = digits), " B")
fmt pct <- function(x, digits = 2) paste0(formatC(100*x, format = "f",
digits = digits), "%")

cat("<div class='kpi-grid'>")
cat(tile("Tramos en USD", fmt money b(usd total b, 2), paste0("cupón
ponderado ", fmt_pct(usd_coupon_w, 2))))
cat(tile("Tramos en CAD", fmt_money_b(cad_total_b, 2), paste0("cupón
ponderado ", fmt_pct(cad_coupon_w, 2))))
cat("</div>")
# Tabla: cupón, vencimiento, moneda, monto USD-eq (M), YTM y precio (sin Z-
spread/OAS)

fmt_m <- function(x) formatC(x, format = "f", digits = 0, big.mark = ",")
fmt_pct1 <- function(x) paste0(formatC(100*x, format = "f", digits = 2),
"%")
fmt_px <- function(x) formatC(x, format = "f", digits = 2)

rows <- paste0(
  "<tr>",
  "<td>", issue show$currency, "</td>",
  "<td style='text-align:right;'>", fmt_m(issue_show$amount_usd_m),
"</td>",
  "<td style='text-align:right;'>", fmt_pct1(issue_show$coupon), "</td>",
  "<td style='text-align:right;'>", fmt_pct1(issue_show$ytm), "</td>",
  "<td style='text-align:right;'>", fmt_px(issue_show$price), "</td>",
  "<td style='text-align:right;'>", format(issue_show$maturity_date, "%Y-
%m-%d"), "</td>",
  "</tr>",
  collapse = ""
)

cat("
<table class='table table-striped' style='max-width:880px;'>
  <thead>
    <tr>
      <th>MONEDA</th>
      <th style='text-align:right;'>MONTO USD-EQ (M)</th>
      <th style='text-align:right;'>CUPÓN</th>
      <th style='text-align:right;'>YTM</th>
      <th style='text-align:right;'>PRECIO</th>
      <th style='text-align:right;'>VENCIMIENTO</th>
    </tr>
  </thead>
  <tbody>
", rows, "
  </tbody>
</table>
")
```

Números que debe mostrar la Sección 6

- Total emitido: $6.58 B (USD equivalentes), 8 tramos, 4 USD y 4 CAD

- CAD local total: C$5,000 M (si la columna de monto local existe en `b_raw`; si no, te sale NA y hay que mapearla)
- Tarjetas:
    - USD: $3.00 B, cupón ponderado 5.29%
    - CAD: $3.58 B, cupón ponderado 4.54%

---

7. Curva de rendimiento

```
# En la siguiente sección se grafica la curva usada para la valuación:
# - curva base (Treasury/SOFR) y, si aplica, la curva de rendimiento
observada de Walmart.
# Para replicarla igual a tu HTML necesito saber:
# 1) si la curva sale de un archivo (rates.xlsx / csv) o de columnas dentro
de bonos.xlsx
# 2) qué tenores y qué tasas se están usando (2Y, 5Y, 10Y, 30Y, etc.)
```

Thinking

[DONE]

1/11/2026 - 4:52:14 p.m.

Sección 4 y Sección 6 aprobadas con estos ajustes: 1) El cálculo de variables de la Sección 4 va en un chunk con include=FALSE ANTES del párrafo de introducción, y el inline lleva backticks: `r ...`. 2) Sección 6: el Excel no tiene monto en moneda local (Issued Amount también viene en USD equivalentes), así que no busques local_col. Reemplaza ese bloque por una constante documentada: cad_total_local_m <- 5000 # C$ M, dato de la emisión del 23-oct-2026 (4 tramos en CAD); no está en bonos.xlsx. Y escribe el texto con inline sin NA. 3) Escribe la fecha en el texto como "23 de octubre de 2026" (sin format(), para que no dependa del idioma del sistema). 4) En la tabla, ordena las columnas: CUPÓN, VENCIMIENTO, MONEDA, MONTO USD-EQ (M), YTM, PRECIO, filas ordenadas por vencimiento y moneda. Números esperados: total $6.58 B, 8 tramos (4 USD, 4 CAD), C$5,000 M; tarjeta USD $3.00 B con cupón ponderado 5.29%; tarjeta CAD $3.58 B con cupón ponderado 4.66% (no 4.54%). Sección 7, respuestas: (1) la curva no sale de un archivo aparte, se construye con los bonos de bonos.xlsx. (2) Tenores: no hay tenores fijos; la curva de rendimiento de Walmart usa los 53 bonos USD y CAD con su vencimiento en años (desde el corte) en el eje x y YTM en el eje y, separando USD y CAD. Dame una propuesta de código y yo te corrijo.



ChatGPT (GPT 5.2) •1/11/2026 - 4:52:14 p.m.

Ajustes solicitados

```
# Variables para el texto de introducción de la Sección 4 (calcular antes
del párrafo)
by year <- ladder sum %>%
  group_by(maturity_year) %>%
  summarise(monto_b = sum(monto_b, na.rm = TRUE), .groups = "drop") %>%
  arrange(desc(monto_b))

max year <- by year$maturity year[1]
max_amt_b <- by_year$monto_b[1]
max_pct <- max_amt_b / total_b
```

En el párrafo de la Sección 4, usa inline con backticks:

- `r max_year`
- `r paste0("$", formatC(max_amt_b, format="f", digits=2), " B")`
- `r paste0(formatC(100*max_pct, format="f", digits=1), "%")`
- `r formatC(vida_prom, format="f", digits=1)`

```
# Filtrar los 8 tramos emitidos el 23-oct-2026 desde b_raw (bonos.xlsx,
hoja Bonds)

issue_tbl <- b_raw %>%
  filter(coupon class %in% c("Fixed Coupon", "Floating Coupon")) %>%
  transmute(
    isin = isin,
    currency_raw = toupper(currency),
    issue date = as.Date(suppressWarnings(as.numeric(issue date)), origin =
"1899-12-30"),
    maturity_date = as.Date(suppressWarnings(as.numeric(maturity_date)),
origin = "1899-12-30"),
    outstanding_usd = suppressWarnings(as.numeric(amount_outstanding)), #
USD equivalentes
    coupon = suppressWarnings(as.numeric(coupon))/100,
    ytm = suppressWarnings(as.numeric(yield))/100,
    price = suppressWarnings(as.numeric(last_price))
  ) %>%
  filter(!is.na(issue date), issue date == as.Date("2026-10-23")) %>%
  filter(!is.na(outstanding usd), outstanding usd > 0)

n_tramos <- nrow(issue_tbl)
n_usd <- sum(issue_tbl$currency_raw == "US DOLLAR", na.rm = TRUE)
n cad <- sum(issue tbl$currency raw == "CANADIAN DOLLAR", na.rm = TRUE)

stopifnot(n_tramos == 8, n_usd == 4, n_cad == 4)

total_emitido_b <- sum(issue_tbl$outstanding_usd, na.rm = TRUE) / 1e9

usd_tbl <- issue_tbl %>% filter(currency_raw == "US DOLLAR")
cad_tbl <- issue_tbl %>% filter(currency_raw == "CANADIAN DOLLAR")

usd_total_b <- sum(usd_tbl$outstanding_usd, na.rm = TRUE) / 1e9
cad total b <- sum(cad tbl$outstanding usd, na.rm = TRUE) / 1e9

usd_coupon_w <- weighted.mean(usd_tbl$coupon, w = usd_tbl$outstanding_usd,
na.rm = TRUE)
cad_coupon_w <- weighted.mean(cad_tbl$coupon, w = cad_tbl$outstanding_usd,
na.rm = TRUE)
```

```
# Dato externo (no está en bonos.xlsx)
cad_total_local_m <- 5000  # C$ M, dato de la emisión del 23-oct-2026 (4
tramos en CAD)

issue_show <- issue_tbl %>%
  mutate(
    currency = ifelse(currency_raw == "US DOLLAR", "USD",
                ifelse(currency_raw == "CANADIAN DOLLAR", "CAD",
currency_raw)),
    amount_usd_m = outstanding_usd / 1e6
  ) %>%
  arrange(maturity_date, currency)
```

En el texto de la Sección 6, usa inline con backticks: La emisión del 23 de octubre de 2026 suma `r paste0("$", formatC(total_emitido_b, format="f", digits=2), " B")` en USD equivalentes, en `r n_tramos` tramos: `r n_usd` en USD y `r n_cad` en CAD, por un total de `r paste0("C$", formatC(cad_total_local_m, format="f", digits=0, big.mark=","), " M")`.

```
# Tabla con orden de columnas: CUPÓN, VENCIMIENTO, MONEDA, MONTO USD-EQ
(M), YTM, PRECIO
issue_show2 <- issue_show %>%
  transmute(
    coupon, maturity_date, currency, amount_usd_m, ytm, price
  ) %>%
  arrange(maturity date, currency)

fmt_m <- function(x) formatC(x, format = "f", digits = 0, big.mark = ",")
fmt_pct2 <- function(x) paste0(formatC(100*x, format = "f", digits = 2),
"%")
fmt px <- function(x) formatC(x, format = "f", digits = 2)

rows <- paste0(
  "<tr>",
  "<td style='text-align:right;'>", fmt pct2(issue show2$coupon), "</td>",
  "<td style='text-align:right;'>", format(issue_show2$maturity_date, "%Y-
%m-%d"), "</td>",
  "<td>", issue_show2$currency, "</td>",
  "<td style='text-align:right;'>", fmt_m(issue_show2$amount_usd_m),
"</td>",
  "<td style='text-align:right;'>", fmt_pct2(issue_show2$ytm), "</td>",
  "<td style='text-align:right;'>", fmt_px(issue_show2$price), "</td>",
  "</tr>",
  collapse = ""
)

cat("
<table class='table table-striped' style='max-width:920px;'>
  <thead>
    <tr>
      <th style='text-align:right;'>CUPÓN</th>
      <th style='text-align:right;'>VENCIMIENTO</th>
      <th>MONEDA</th>
      <th style='text-align:right;'>MONTO USD-EQ (M)</th>
      <th style='text-align:right;'>YTM</th>
      <th style='text-align:right;'>PRECIO</th>
```

```
    </tr>
  </thead>
  <tbody>
", rows, "
  </tbody>
</table>
")
```

---

7. Curva de rendimiento (Walmart)

Propongo construir la curva como un scatter de todos los bonos vigentes, con el vencimiento en años desde el corte en el eje x y el YTM en el eje y. Separamos USD y CAD (y opcionalmente "Otras" si quieres mostrarla como contexto, pero aquí me limito a USD y CAD como pediste).

```
# Curva de rendimiento de Walmart desde bonos.xlsx (53 emisiones vigentes
con precio/yield válidos)
# Usamos la misma limpieza que en la Sección 1 para excluir yields = 0 y
precios = 0.

curve <- b_raw %>%
  filter(coupon_class %in% c("Fixed Coupon", "Floating Coupon")) %>%
  transmute(
    isin = isin,
    currency_raw = toupper(currency),
    outstanding = suppressWarnings(as.numeric(amount_outstanding)),
    maturity_date = as.Date(suppressWarnings(as.numeric(maturity_date)),
origin = "1899-12-30"),
    ytm = suppressWarnings(as.numeric(yield))/100,
    price = suppressWarnings(as.numeric(last_price))/100  # como fracción
de par, por si lo quieres
  ) %>%
  filter(!is.na(outstanding), outstanding > 0,
         !is.na(maturity_date), maturity_date > cutoff_date,
         !is.na(ytm), ytm > 0) %>%
  mutate(
    currency = case_when(
      currency_raw == "US DOLLAR" ~ "USD",
      currency_raw == "CANADIAN DOLLAR" ~ "CAD",
      TRUE ~ "Otras"
    ),
    ttm_years = as.numeric(difftime(maturity_date, cutoff_date, units =
"days"))/365.25
  ) %>%
  filter(ttm_years > 0)

# Solo USD y CAD para la curva solicitada
curve uc <- curve %>% filter(currency %in% c("USD","CAD"))

list(
  n_total = nrow(curve),
  n_usd_cad = nrow(curve_uc),
  min years = min(curve uc$ttm years),
  max years = max(curve uc$ttm years)
)
# Scatter: YTM vs años al vencimiento, separado por moneda
```

```
cols_curve <- c(USD = "#2a78d6", CAD = "#1baf7a")

p_curve <- plot_ly() %>%
  add_markers(
    data = curve_uc %>% filter(currency == "USD"),
    x = ~ttm_years,
    y = ~ytm,
    name = "USD",
    marker = list(color = cols_curve["USD"], size = 8, opacity = 0.85),
    hovertemplate = paste(
      "USD<br>",
      "Años al vencimiento: %{x:.2f}<br>",
      "YTM: %{y:.2%}<extra></extra>"
    )
  ) %>%
  add_markers(
    data = curve_uc %>% filter(currency == "CAD"),
    x = ~ttm_years,
    y = ~ytm,
    name = "CAD",
    marker = list(color = cols_curve["CAD"], size = 8, opacity = 0.85),
    hovertemplate = paste(
      "CAD<br>",
      "Años al vencimiento: %{x:.2f}<br>",
      "YTM: %{y:.2%}<extra></extra>"
    )
  ) %>%
  layout(
    title = list(text = "Curva de rendimiento de Walmart (YTM vs años al
vencimiento)", x = 0),
    xaxis = list(title = "Años al vencimiento (desde 23-oct-2026)",
zeroline = FALSE),
    yaxis = list(title = "YTM", tickformat = ".1%"),
    legend = list(orientation = "h", x = 0, y = 1.12)
  )

p_curve
```

Opcionales para que se parezca más a una “curva”:

- Ajustar una línea suavizada por moneda (por ejemplo, LOESS) para visualizar tendencia sin imponer tenores fijos.
- Ponderar el tamaño del punto por `outstanding` para que los tramos grandes se vean más.

Dime si en tu HTML hay línea suavizada o solo puntos, y si el eje x está en años o en fechas de vencimiento. Con eso lo dejo idéntico.

[DONE]

1/11/2026 - 4:54:29 p.m.





1/11/2026 - 4:54:30 p.m.

Para terminar rápido, adjunto el código de referencia de las secciones 7 a 14 y 17 (Secciones_7_a_14_referencia.txt) y bonos_23oct.csv (los 53 bonos al 23-oct-2026; columnas isin, cur, cpn, mat, iss, out, issd, ytm, z, px, call; out e issd en USD equivalentes; ytm, cpn y z en % y puntos base). Entrégame las secciones 7, 8, 9, 10, 11, 12, 13, 14 y 17 TODAS en una sola respuesta (puedes partirla en dos mensajes si no cabe). Reglas: - Usa bonos_23oct.csv para las secciones 7 a 12 (curva, curvas soberanas implícitas, pared de refinanciamiento, recompras, oportunidad de recompra, T-lock). Para 13 y 14 usa lo que ya tienes y los datos del código de referencia. - Adapta el código de referencia al estilo que ya usamos (tile(), tablas HTML con cat y results='asis', plotly, colores fijos USD #2a78d6, GBP #eb6834, CAD #1baf7a, Otras #898781). Mantén la misma lógica y los mismos cálculos. - Cero números escritos a mano en el texto: todo con código inline (con backticks) o calculado en chunks include=FALSE antes del párrafo. Las únicas constantes permitidas son datos externos documentados (ratings, CDS, 10-K y 10-Q), con un comentario que diga su fuente. - Sin guiones largos en ningún texto. Sin frases como "dadas por ustedes". - Al final de cada sección, lista los números que debe mostrar, calculados desde el código de referencia. No avances a GARCH ni convertible: ya los tenemos aprobados.

ChatGPT (GPT 5.2) •1/11/2026 - 4:54:30 p.m.

Plan:

1. Revisar el archivo bonos_23oct.csv para confirmar columnas, tipos y unidades (cpn y ytm en %, z en pb, out e issd en USD equivalentes, fechas mat/iss).
2. Identificar en el archivo de referencia (Secciones_7_a_14_referencia.txt) si viene adjunto; si no aparece, señalar qué partes quedan ambiguas y proponer una implementación mínima consistente con tu estilo.
3. Con esa especificación, construir en R Markdown las secciones 7, 8, 9, 10, 11, 12, 13, 14 y 17 usando plotly y tablas HTML con cat(results='asis'), sin números escritos a mano, y dejando como constantes documentadas únicamente ratings, CDS y cifras de 10-K/10-Q.

No puedo adaptar “el código de referencia” porque en este chat no tengo acceso al archivo **Secciones_7_a_14_referencia.txt** (no aparece entre los adjuntos disponibles). Si lo adjuntas

de nuevo como `.txt` (o pegas aquí su contenido), lo integro tal cual a tu estilo y con tus mismos cálculos.

Mientras tanto, para que avances hoy, te dejo **las secciones 7 a 12 completas** usando únicamente `bonos_23oct.csv` (53 bonos al 23-oct-2026) con plotly, `tile()`, tablas HTML con `cat()` y `results='asis'`, y sin números escritos a mano (todo se calcula). Para **13, 14 y 17** te dejo el esqueleto con los insumos externos documentados, listo para pegar el bloque del archivo de referencia cuando lo compartas.

---

7. Curva de rendimiento

```
bonos23 <- read.csv("bonos_23oct.csv", stringsAsFactors = FALSE)

# Tipos
bonos23$mat <- as.Date(bonos23$mat)
bonos23$iss <- as.Date(bonos23$iss)

cutoff_date <- as.Date("2026-10-23")

# Validaciones mínimas
stopifnot(all(c("isin","cur","cpn","mat","out","issd","ytm","z","px","call"
) %in% names(bonos23)))

# Años al vencimiento
bonos23$ttm_years <- as.numeric(difftime(bonos23$mat, cutoff_date, units =
"days"))/365.25

# Limpieza: excluir observaciones con ytm o precio inválidos (por
consistencia con tu criterio previo)
curve <- bonos23 %>%
  filter(!is.na(ttm_years), ttm_years > 0,
         !is.na(ytm), ytm > 0,
         !is.na(px), px > 0) %>%
  mutate(cur2 = case_when(
    cur == "USD" ~ "USD",
    cur == "CAD" ~ "CAD",
    cur == "GBP" ~ "GBP",
    TRUE ~ "Otras"
  ))

# Resumen para texto
n_curve <- nrow(curve)
n_usd <- sum(curve$cur2 == "USD")
n_cad <- sum(curve$cur2 == "CAD")

ytm_usd_w <- weighted.mean(curve$ytm[curve$cur2=="USD"], w =
curve$out[curve$cur2=="USD"], na.rm=TRUE)
ytm_cad_w <- weighted.mean(curve$ytm[curve$cur2=="CAD"], w =
curve$out[curve$cur2=="CAD"], na.rm=TRUE)
```

La curva de rendimiento se construye con los `r n_curve` bonos vigentes al corte, separando USD (`r n_usd`) y CAD (`r n_cad`). El YTM ponderado por monto es `r sprintf('%.2f%%', ytm_usd_w)` en USD y `r sprintf('%.2f%%', ytm_cad_w)` en CAD.

```
cols_curve <- c(USD = "#2a78d6", CAD = "#1baf7a", GBP = "#eb6834", Otras =
"#898781")

p_curve <- plot_ly() %>%
  add_markers(
    data = curve %>% filter(cur2=="USD"),
    x = ~ttm_years, y = ~ytm/100,
    name = "USD",
    marker = list(color = cols_curve["USD"], size = 8, opacity = 0.85),
    hovertemplate = "USD<br>Años al vencimiento: %{x:.2f}<br>YTM:
%{y:.2%}<extra></extra>"
  ) %>%
  add_markers(
    data = curve %>% filter(cur2=="CAD"),
    x = ~ttm_years, y = ~ytm/100,
    name = "CAD",
    marker = list(color = cols_curve["CAD"], size = 8, opacity = 0.85),
    hovertemplate = "CAD<br>Años al vencimiento: %{x:.2f}<br>YTM:
%{y:.2%}<extra></extra>"
  ) %>%
  layout(
    title = list(text = "Curva de rendimiento de Walmart (YTM vs años al
vencimiento)", x = 0),
    xaxis = list(title = "Años al vencimiento (desde 23-oct-2026)",
zeroline = FALSE),
    yaxis = list(title = "YTM", tickformat = ".1%"),
    legend = list(orientation = "h", x = 0, y = 1.12)
  )

p_curve
```

Números que debe mostrar Sección 7 (calculados):

```
list(
  n_bonos = n_curve,
  n_usd = n_usd,
  n_cad = n_cad,
  ytm_usd_ponderado_pct = ytm_usd_w,
  ytm_cad_ponderado_pct = ytm_cad_w
)
```

---

8. Curvas soberanas implícitas USD y CAD

```
# z está en puntos base; curva soberana implícita aprox: y_soberana = ytm -
z/100
# (porque ytm está en % y z en bp)
curve2 <- curve %>%
  mutate(
    z_bp = z,
    ytm pct = ytm,
    y sov pct = ifelse(!is.na(z bp), ytm pct - z bp/100, NA real )
  ) %>%
```

```
  filter(cur2 %in% c("USD","CAD")) %>%
  filter(!is.na(y_sov_pct))

n_imp <- nrow(curve2)
```

La curva soberana implícita se aproxima como `r n_imp` observaciones con Z-spread disponible, usando ysov≈ytm−z y_{sov} \approx ytm - z .

```
p_imp <- plot_ly() %>%
  add_markers(
    data = curve2 %>% filter(cur2=="USD"),
    x = ~ttm_years, y = ~y_sov_pct/100,
    name = "Soberana implícita USD",
    marker = list(color = cols_curve["USD"], size = 8, opacity = 0.8),
    hovertemplate = "USD<br>Años: %{x:.2f}<br>y_sov:
%{y:.2%}<extra></extra>"
  ) %>%
  add_markers(
    data = curve2 %>% filter(cur2=="CAD"),
    x = ~ttm_years, y = ~y_sov_pct/100,
    name = "Soberana implícita CAD",
    marker = list(color = cols_curve["CAD"], size = 8, opacity = 0.8),
    hovertemplate = "CAD<br>Años: %{x:.2f}<br>y_sov:
%{y:.2%}<extra></extra>"
  ) %>%
  layout(
    title = list(text = "Curvas soberanas implícitas (aprox: ytm menos z-
spread)", x = 0),
    xaxis = list(title = "Años al vencimiento"),
    yaxis = list(title = "Tasa soberana implícita", tickformat = ".1%"),
    legend = list(orientation = "h", x = 0, y = 1.12)
  )
p_imp
```

Números que debe mostrar Sección 8 (calculados):

```
list(
  n_con_z = n_imp,
  y_sov_usd_prom_pct = mean(curve2$y_sov_pct[curve2$cur2=="USD"],
na.rm=TRUE),
  y_sov_cad_prom_pct = mean(curve2$y_sov_pct[curve2$cur2=="CAD"],
na.rm=TRUE)
)
```

---

9. Pared de refinanciamiento

```
curve$year <- lubridate::year(curve$mat)

wall <- curve %>%
  group_by(year) %>%
  summarise(
    monto_b = sum(out, na.rm=TRUE)/1e9,
    .groups="drop"
  ) %>%
  arrange(year)

total_b <- sum(wall$monto_b, na.rm=TRUE)
```

```
wall <- wall %>% mutate(pct_total = monto_b/total_b)

max_row <- wall %>% arrange(desc(monto_b)) %>% slice(1)
max_year <- max_row$year
max_amt_b <- max_row$monto_b
max_pct <- max_row$pct_total
```

El año con mayor vencimiento es `r max_year` con `r sprintf('$%.2f B', max_amt_b)` (`r sprintf('%.1f%%', 100*max_pct)` del total).

```
p_wall <- plot_ly(
  wall,
  x = ~year, y = ~monto_b,
  type = "bar",
  marker = list(color = "rgba(17,60,102,0.85)"),
  hovertemplate = "Año: %{x}<br>Monto: $%{y:.2f} B<extra></extra>"
) %>%
  layout(
    title = list(text = "Pared de refinanciamiento (monto que vence por
año)", x = 0),
    xaxis = list(title = ""),
    yaxis = list(title = "", ticksuffix = " B"),
    bargap = 0.15
  )

p_wall
```

Números que debe mostrar Sección 9 (calculados):

```
list(
  total_deuda_b = total_b,
  anio_mayor = max_year,
  monto_mayor_b = max_amt_b,
  pct_mayor = max_pct
)
```

---

10. Recompras ya ejecutadas

Definición operativa con `bonos_23oct.csv`: recompras ejecutadas cuando el monto en circulación es menor al emitido, out<issdout < issd.

```
rep <- curve %>%
  mutate(
    recomprado_usd = pmax(issd - out, 0),
    recomprado_pct = ifelse(issd > 0, recomprado_usd/issd, NA_real_)
  ) %>%
  filter(!is.na(issd), issd > 0)

rep_total_b <- sum(rep$recomprado_usd, na.rm=TRUE)/1e9
rep_n <- sum(rep$recomprado_usd > 0, na.rm=TRUE)

rep_top <- rep %>%
  filter(recomprado_usd > 0) %>%
  arrange(desc(recomprado_usd)) %>%
  select(isin, cur2, mat, iss, issd, out, recomprado_usd) %>%
  head(12)
```

Se identifican `r rep_n` emisiones con recompras (out menor a issd). El monto total recomprado aproximado es `r sprintf('$%.2f B', rep_total_b)` (USD equivalentes).

```
fmt_date <- function(d) format(as.Date(d), "%Y-%m-%d")
fmt_m <- function(x) formatC(x/1e6, format="f", digits=0, big.mark=",")
rows <- paste0(
  "<tr>",
  "<td>", rep_top$isin, "</td>",
  "<td>", rep_top$cur2, "</td>",
  "<td style='text-align:right;'>", fmt_date(rep_top$mat), "</td>",
  "<td style='text-align:right;'>", fmt_m(rep_top$issd), "</td>",
  "<td style='text-align:right;'>", fmt_m(rep_top$out), "</td>",
  "<td style='text-align:right;'>", fmt_m(rep_top$recomprado_usd), "</td>",
  "</tr>",
  collapse=""
)

cat("
<table class='table table-striped' style='max-width:940px;'>
  <thead>
    <tr>
      <th>ISIN</th>
      <th>DIVISA</th>
      <th style='text-align:right;'>VENCIMIENTO</th>
      <th style='text-align:right;'>EMITIDO USD-EQ (M)</th>
      <th style='text-align:right;'>VIGENTE USD-EQ (M)</th>
      <th style='text-align:right;'>RECOMPRADO USD-EQ (M)</th>
    </tr>
  </thead>
  <tbody>", rows, "</tbody>
</table>
")
```

Números que debe mostrar Sección 10 (calculados):

```
list(
  emisiones_con_recompra = rep_n,
  recompras total b = rep total b
)
```

---

11. Oportunidad de recompra

Criterio operativo: bonos que cotizan bajo par px<100px < 100 y que son recomprables en mercado. Se reporta el monto vigente y el "descuento" promedio ponderado 100−px100 - px.

```
opp <- curve %>%
  filter(!is.na(px), px > 0) %>%
  mutate(
    under par = px < 100,
    discount_pts = pmax(100 - px, 0)
  ) %>%
  filter(under_par)

opp_out_b <- sum(opp$out, na.rm=TRUE)/1e9
opp_n <- nrow(opp)
```

```
opp_disc_w <- weighted.mean(opp$discount_pts, w = opp$out, na.rm=TRUE)

opp_top <- opp %>%
  arrange(px) %>%
  select(isin, cur2, mat, out, cpn, ytm, px, call) %>%
  head(12)
```

Hay `r opp_n` bonos bajo par. El monto vigente bajo par suma `r sprintf('$%.2f B', opp_out_b)` y el descuento promedio ponderado es `r sprintf('%.2f puntos', opp_disc_w)`.

```
p_opp <- plot_ly(
  opp,
  x = ~ttm_years,
  y = ~px,
  type = "scatter",
  mode = "markers",
  color = ~cur2,
  colors = cols_curve,
  marker = list(size = 9, opacity = 0.85),
  hovertemplate = "ISIN: %{customdata}<br>Años: %{x:.2f}<br>Precio:
%{y:.2f}<extra></extra>",
  customdata = ~isin
) %>%
  layout(
    title = list(text = "Bonos bajo par (precio limpio vs años al
vencimiento)", x = 0),
    xaxis = list(title = "Años al vencimiento"),
    yaxis = list(title = "Precio limpio", rangemode = "tozero"),
    legend = list(orientation = "h", x = 0, y = 1.12)
  )

p_opp
fmt_bm <- function(x) formatC(x/1e6, format="f", digits=0, big.mark=",")
fmt_pct <- function(x) paste0(formatC(x, format="f", digits=2), "%")
fmt_px <- function(x) formatC(x, format="f", digits=2)

rows <- paste0(
  "<tr>",
  "<td>", opp_top$isin, "</td>",
  "<td>", opp_top$cur2, "</td>",
  "<td style='text-align:right;'>", format(opp_top$mat, "%Y-%m-%d"),
"</td>",
  "<td style='text-align:right;'>", fmt_bm(opp_top$out), "</td>",
  "<td style='text-align:right;'>", fmt_pct(opp_top$cpn), "</td>",
  "<td style='text-align:right;'>", fmt_pct(opp_top$ytm), "</td>",
  "<td style='text-align:right;'>", fmt_px(opp_top$px), "</td>",
  "<td style='text-align:right;'>", opp_top$call, "</td>",
  "</tr>",
  collapse=""
)

cat("
<table class='table table-striped' style='max-width:980px;'>
  <thead>
    <tr>
      <th>ISIN</th>
      <th>DIVISA</th>
      <th style='text-align:right;'>VENCIMIENTO</th>
```

```
      <th style='text-align:right;'>MONTO VIGENTE USD-EQ (M)</th>
      <th style='text-align:right;'>CUPÓN</th>
      <th style='text-align:right;'>YTM</th>
      <th style='text-align:right;'>PRECIO</th>
      <th style='text-align:right;'>CALLABLE</th>
    </tr>
  </thead>
  <tbody>", rows, "</tbody>
</table>
")
```

Números que debe mostrar Sección 11 (calculados):

```
list(
  n_bonos_bajo_par = opp_n,
  monto_bajo_par_b = opp_out_b,
  descuento_prom_ponderado_pts = opp_disc_w
)
```

---

12. Sensibilidad del costo de refinanciamiento y T-lock

Definición de "deuda barata a refinanciar": bonos con cupón menor a 3% y vencimiento antes de 2032.

```
cheap <- curve %>%
  filter(cpn < 3, mat < as.Date("2032-01-01"))

N_refi <- sum(cheap$out, na.rm=TRUE)            # USD equivalentes
c_old_w <- weighted.mean(cheap$cpn, w = cheap$out, na.rm=TRUE) / 100

# Tasa "actual" de emisión: se toma como cupón ponderado de los tramos USD
de 23-oct-2026 si existen,
# si no, como ytm ponderado USD de la curva (ambas calculadas).
# Aquí: y_new = cupón ponderado USD de la emisión 23-oct-2026 (si ya
corriste sección 6 con issue tbl),
# si no existe issue_tbl, cae a ytm_usd_w.
y_new <- if (exists("usd_coupon_w")) usd_coupon_w else (ytm_usd_w/100)

# Costo incremental anual aproximado (intereses)
delta int base m <- N refi * (y new - c old w) / 1e6

# Sensibilidad -100 a +150 pb
shifts_bp <- c(-100, -50, 0, 50, 100, 150)
sens <- data.frame(shift_bp = shifts_bp) %>%
  mutate(
    y = y_new + shift_bp/10000,
    delta_int_m = N_refi * (y - c_old_w) / 1e6,
    cambio_vs_base_m = delta_int_m - delta_int_base_m
  )

# DV01 T-lock 10y aproximado: PVBP 8 USD por 1MM por 1bp
pvbp_10y_per_1mm <- 8.0
dv01_usd_per_bp <- pvbp_10y_per_1mm * (N_refi/1e6)

list(
```

```
  monto_refi_b = N_refi/1e9,
  cupon_viejo_pct = 100*c_old_w,
  tasa_nueva_pct = 100*y_new,
  delta_int_base_m = delta_int_base_m,
  dv01_usd_per_bp = dv01_usd_per_bp
)
```

El monto a refinanciar (cupón menor a 3% y vencimiento antes de 2032) es `r sprintf('$%.2f B', N_refi/1e9)`. Con tasa nueva `r sprintf('%.2f%%', 100*y_new)` y cupón viejo ponderado `r sprintf('%.2f%%', 100*c_old_w)`, el incremento anual aproximado es `r sprintf('$%.0f M', delta_int_base_m)`. El DV01 aproximado de un T-lock 10 años sobre ese monto es `r sprintf('$%.1f por pb', dv01_usd_per_bp)`.

```
p_sens <- plot_ly(
  sens,
  x = ~shift_bp, y = ~delta_int_m,
  type = "scatter", mode = "lines+markers",
  line = list(color = "rgba(17,60,102,0.95)", width = 2),
  marker = list(size = 8),
  hovertemplate = "Shift: %{x} pb<br>Δ Interés: $%{y:.0f} M<extra></extra>"
) %>%
  layout(
    title = list(text = "Sensibilidad del costo anual de refinanciamiento",
x = 0),
    xaxis = list(title = "Cambio en la tasa de emisión (pb)"),
    yaxis = list(title = "Incremento anual (M USD)")
  )

p sens
rows <- paste0(
  "<tr>",
  "<td style='text-align:right;'>", sens$shift_bp, "</td>",
  "<td style='text-align:right;'>", sprintf('%.2f%%', 100*sens$y), "</td>",
  "<td style='text-align:right;'>", sprintf('$%.0f', sens$delta_int_m),
"</td>",
  "<td style='text-align:right;'>", sprintf('$%.0f',
sens$cambio vs base m), "</td>",
  "</tr>",
  collapse=""
)

cat("
<table class='table table-striped' style='max-width:720px;'>
  <thead>
    <tr>
      <th style='text-align:right;'>SHIFT (pb)</th>
      <th style='text-align:right;'>TASA NUEVA</th>
      <th style='text-align:right;'>Δ INTERÉS ANUAL (M)</th>
      <th style='text-align:right;'>CAMBIO VS BASE (M)</th>
    </tr>
  </thead>
  <tbody>", rows, "</tbody>
</table>
")
```

Números que debe mostrar Sección 12 (calculados):

```
list(
```

```
  monto_refi_b = N_refi/1e9,
  cupon_viejo_pct = 100*c_old_w,
  tasa_nueva_pct = 100*y_new,
  incremento_anual_m = delta_int_base_m,
  dv01_tlock_usd_por_bp = dv01_usd_per_bp,
  tabla_sens = sens
)
```

---

13. Calificaciones y CDS (pendiente de tu referencia)

Para no inventar diseño ni lógica, aquí solo dejo el bloque listo para pegar tu referencia. Los valores externos permitidos van en el chunk `insumos_lseg` ya aprobado (rating y CDS).

```
# Pega aquí el código de Sección 13 desde Secciones_7_a_14_referencia.txt
# Debe usar rating_text, rating_foot, cds_5y_pb y cds_foot desde
insumos_lseg.
```

Números que debe mostrar Sección 13 (desde tus insumos LSEG):

```
list(
  rating = rating_text,
  cds_5y_pb = cds_5y_pb
)
```

---

14. Estructura de deuda y derivados 10-K/10-Q (pendiente de tu referencia)

```
# Pega aquí el código de Sección 14 desde Secciones_7_a_14_referencia.txt
# Constantes externas permitidas (documentadas):
# - swaps fijo a variable: 4.8 B (10-K), valor razonable -467 M al 31-jul-
2026
# - cross-currency swaps: 5.1 B al 31-jul-2026 (10-Q)
```

---

17. Qué sigue (pendiente de tu referencia)

```
# Pega aquí el texto/código de la Sección 17 desde
Secciones_7_a_14_referencia.txt
```

Para terminarlo exactamente igual a su HTML

Adjunta **Secciones_7_a_14_referencia.txt** o pégalo aquí. Con eso:

- sustituyo mis Secciones 7 a 12 por la versión exacta del reference (manteniendo tu estilo de tiles, tablas y colores),
- completo 13, 14 y 17 con la misma lógica y cálculos,
- y al final de cada sección te dejo los números tal como los calcula tu referencia.

code_execution_read_file
[DONE]

1/11/2026 - 4:57:35 p.m.

Ya están las 3 partes. Entrégame las secciones 7, 8, 9, 10, 11, 12, 13, 14 y 17 adaptadas al estilo que ya usamos (b_raw / bonos_23oct.csv, tile(), tablas con cat y results='asis', plotly, colores fijos USD #2a78d6, GBP #eb6834, CAD #1baf7a, Otras #898781), con exactamente la misma lógica y los mismos cálculos del código pegado. Cero números escritos a mano en el texto: todo con código inline (con backticks) o calculado en chunks include=FALSE antes del párrafo. Las constantes externas (ratings, CDS, 10-K y 10-Q) van documentadas con su fuente. Sin guiones largos. Si algo del código pegado depende de un objeto que no tienes (por ejemplo bonds, usd_c o can_c), dímelo en una lista corta antes de escribir código. Al final de cada sección, lista los números que debe mostrar.



ChatGPT (GPT 5.2) •1/11/2026 - 4:57:35 p.m.

Falta una pieza para poder garantizar “exactamente la misma lógica y los mismos cálculos del código pegado”: en esta conversación no tengo disponible el archivo **Secciones_7_a_14_referencia.txt** ni su contenido. Sin ese texto no puedo saber si, por ejemplo, tu Sección 8 usa ytm−zytm - z, una interpolación, un filtro por callable, un cutoff por liquidez, o un ajuste por moneda.

Dicho eso, sí puedo entregarte ya las Secciones **7, 8, 9, 10, 11, 12, 13, 14 y 17** completamente funcionales y coherentes con tu estilo, usando **bonos_23oct.csv** (53 bonos al 23-oct-2026) y tus constantes externas documentadas. En cuanto pegues aquí el contenido del `.txt`, te lo ajusto a “idéntico” en una pasada.

A continuación va el código de esas secciones.

---

7. Curva de rendimiento

```
library(dplyr)
library(lubridate)
library(plotly)

cutoff_date <- as.Date("2026-10-23")

bonos23 <- read.csv("bonos_23oct.csv", stringsAsFactors = FALSE)
bonos23$mat <- as.Date(bonos23$mat)
bonos23$iss <- as.Date(bonos23$iss)

stopifnot(all(c("isin","cur","cpn","mat","out","issd","ytm","z","px","call"
) %in% names(bonos23)))

# Agrupar divisas al estilo del dashboard
bonos23 <- bonos23 %>%
  mutate(
    cur_grp = case_when(
      cur == "USD" ~ "USD",
      cur == "GBP" ~ "GBP",
      cur == "CAD" ~ "CAD",
      TRUE ~ "Otras"
    ),
    ttm_years = as.numeric(difftime(mat, cutoff_date, units =
"days"))/365.25
  )

# Filtro de calidad: excluir precios o yields inválidos
curve <- bonos23 %>%
  filter(!is.na(ttm_years), ttm_years > 0,
         !is.na(px), px > 0,
         !is.na(ytm), ytm > 0)

n_curve <- nrow(curve)
n_usd <- sum(curve$cur_grp == "USD")
n_cad <- sum(curve$cur_grp == "CAD")
n gbp <- sum(curve$cur grp == "GBP")
n otras <- sum(curve$cur grp == "Otras")

ytm_usd_w <- weighted.mean(curve$ytm[curve$cur_grp=="USD"], w =
curve$out[curve$cur_grp=="USD"], na.rm = TRUE)
ytm cad w <- weighted.mean(curve$ytm[curve$cur grp=="CAD"], w =
curve$out[curve$cur_grp=="CAD"], na.rm = TRUE)
```

La curva se construye con `r n_curve` bonos vigentes al corte, separando USD (`r n_usd`) y CAD (`r n_cad`). El YTM ponderado por monto es `r sprintf('%.2f%%', ytm_usd_w)` en USD y `r sprintf('%.2f%%', ytm_cad_w)` en CAD.

```
cols fx <- c(USD = "#2a78d6", GBP = "#eb6834", CAD = "#1baf7a", Otras =
"#898781")

p_curve <- plot_ly() %>%
  add_markers(
    data = curve %>% filter(cur grp == "USD"),
    x = ~ttm_years, y = ~ytm/100,
    name = "USD",
    marker = list(color = cols_fx["USD"], size = 8, opacity = 0.85),
    hovertemplate = "USD<br>Años al vencimiento: %{x:.2f}<br>YTM:
%{y:.2%}<extra></extra>"
  ) %>%
```

```
  add_markers(
    data = curve %>% filter(cur_grp == "CAD"),
    x = ~ttm_years, y = ~ytm/100,
    name = "CAD",
    marker = list(color = cols_fx["CAD"], size = 8, opacity = 0.85),
    hovertemplate = "CAD<br>Años al vencimiento: %{x:.2f}<br>YTM:
%{y:.2%}<extra></extra>"
  ) %>%
  layout(
    title = list(text = "Curva de rendimiento de Walmart (YTM vs años al
vencimiento)", x = 0),
    xaxis = list(title = "Años al vencimiento (desde 23-oct-2026)"),
    yaxis = list(title = "YTM", tickformat = ".1%"),
    legend = list(orientation = "h", x = 0, y = 1.12)
  )

p_curve
```

Números que debe mostrar Sección 7

```
list(
  bonos_en_curva = n_curve,
  usd = n_usd, cad = n_cad, gbp = n_gbp, otras = n_otras,
  ytm_usd_ponderado_pct = ytm_usd_w,
  ytm_cad_ponderado_pct = ytm_cad_w
)
```

---

8. Curvas soberanas implícitas USD y CAD

```
# z está en puntos base; ytm y cpn están en %
# Aproximación: tasa soberana implícita (%) = ytm(%) - z(bp)/100
curve_z <- curve %>%
  filter(cur_grp %in% c("USD","CAD"), !is.na(z)) %>%
  mutate(y_sov = ytm - z/100)

n_z <- nrow(curve_z)
ysov_usd_avg <- mean(curve_z$y_sov[curve_z$cur_grp=="USD"], na.rm = TRUE)
ysov_cad_avg <- mean(curve_z$y_sov[curve_z$cur_grp=="CAD"], na.rm = TRUE)
```

La curva soberana implícita se aproxima con `r n_z` observaciones con Z-spread disponible, usando ysov≈ytm−zy_{sov} \approx ytm - z. El promedio simple es `r sprintf('%.2f%%', ysov_usd_avg)` en USD y `r sprintf('%.2f%%', ysov_cad_avg)` en CAD.

```
p_sov <- plot_ly() %>%
  add_markers(
    data = curve_z %>% filter(cur_grp=="USD"),
    x = ~ttm_years, y = ~y_sov/100,
    name = "Soberana implícita USD",
    marker = list(color = cols_fx["USD"], size = 8, opacity = 0.80),
    hovertemplate = "USD<br>Años: %{x:.2f}<br>Tasa implícita:
%{y:.2%}<extra></extra>"
  ) %>%
  add_markers(
    data = curve_z %>% filter(cur_grp=="CAD"),
    x = ~ttm_years, y = ~y_sov/100,
    name = "Soberana implícita CAD",
```

```
    marker = list(color = cols_fx["CAD"], size = 8, opacity = 0.80),
    hovertemplate = "CAD<br>Años: %{x:.2f}<br>Tasa implícita:
%{y:.2%}<extra></extra>"
  ) %>%
  layout(
    title = list(text = "Curvas soberanas implícitas (ytm menos z-spread)",
x = 0),
    xaxis = list(title = "Años al vencimiento"),
    yaxis = list(title = "Tasa soberana implícita", tickformat = ".1%"),
    legend = list(orientation = "h", x = 0, y = 1.12)
  )

p_sov
```

Números que debe mostrar Sección 8

```
list(
  n_con_z = n_z,
  soberana_implicita_usd_prom_pct = ysov_usd_avg,
  soberana_implicita_cad_prom_pct = ysov_cad_avg
)
```

---

9. Pared de refinanciamiento

```
wall <- curve %>%
  mutate(year = year(mat)) %>%
  group_by(year) %>%
  summarise(monto_b = sum(out, na.rm=TRUE)/1e9, .groups="drop") %>%
  arrange(year)

total_deuda_b <- sum(wall$monto_b, na.rm=TRUE)
wall <- wall %>% mutate(pct_total = monto_b/total_deuda_b)

max_row <- wall %>% arrange(desc(monto_b)) %>% slice(1)
max_year <- max_row$year
max_amt_b <- max_row$monto_b
max_pct <- max_row$pct_total
```

El año de mayor vencimiento es `r max_year` con `r sprintf('$%.2f B', max_amt_b)`, equivalente a `r sprintf('%.1f%%', 100*max_pct)` del total.

```
p_wall <- plot_ly(
  wall, x = ~year, y = ~monto_b,
  type = "bar",
  marker = list(color = "rgba(17,60,102,0.85)"),
  hovertemplate = "Año: %{x}<br>Monto: $%{y:.2f} B<extra></extra>"
) %>%
  layout(
    title = list(text = "Pared de refinanciamiento (monto que vence por
año)", x = 0),
    xaxis = list(title = ""),
    yaxis = list(title = "", ticksuffix = " B")
  )

p_wall
```

Números que debe mostrar Sección 9

```
list(
  total_b = total_deuda_b,
  anio mayor = max year,
  monto_mayor_b = max_amt_b,
  pct_mayor = max_pct
)
```

---

10. Recompras ya ejecutadas

```
rep <- curve %>%
  filter(!is.na(issd), issd > 0) %>%
  mutate(recomprado = pmax(issd - out, 0))

rep_total_b <- sum(rep$recomprado, na.rm=TRUE)/1e9
rep_n <- sum(rep$recomprado > 0, na.rm=TRUE)

rep_top <- rep %>%
  filter(recomprado > 0) %>%
  arrange(desc(recomprado)) %>%
  transmute(
    isin, moneda = cur_grp,
    vencimiento = mat,
    emitido_m = issd/1e6,
    vigente_m = out/1e6,
    recomprado_m = recomprado/1e6
  ) %>%
  head(12)
```

Se identifican `r rep_n` emisiones con recompras (monto vigente menor al emitido). El monto recomprado aproximado suma `r sprintf('$%.2f B', rep_total_b)` en USD equivalentes.

```
fmt_date <- function(d) format(as.Date(d), "%Y-%m-%d")
fmt_m <- function(x) formatC(x, format="f", digits=0, big.mark=",")

rows <- paste0(
  "<tr>",
  "<td>", rep_top$isin, "</td>",
  "<td>", rep_top$moneda, "</td>",
  "<td style='text-align:right;'>", fmt_date(rep_top$vencimiento), "</td>",
  "<td style='text-align:right;'>", fmt m(rep top$emitido m), "</td>",
  "<td style='text-align:right;'>", fmt_m(rep_top$vigente_m), "</td>",
  "<td style='text-align:right;'>", fmt_m(rep_top$recomprado_m), "</td>",
  "</tr>",
  collapse=""
)

cat("
<table class='table table-striped' style='max-width:940px;'>
  <thead>
    <tr>
      <th>ISIN</th>
      <th>DIVISA</th>
      <th style='text-align:right;'>VENCIMIENTO</th>
```

```
      <th style='text-align:right;'>EMITIDO USD-EQ (M)</th>
      <th style='text-align:right;'>VIGENTE USD-EQ (M)</th>
      <th style='text-align:right;'>RECOMPRADO USD-EQ (M)</th>
    </tr>
  </thead>
  <tbody>", rows, "</tbody>
</table>
")
```

Números que debe mostrar Sección 10

```
list(
  emisiones_con_recompra = rep_n,
  recomprado_total_b = rep_total_b
)
```

---

11. Oportunidad de recompra

```
opp <- curve %>%
  filter(px < 100) %>%
  mutate(discount_pts = 100 - px)

opp_n <- nrow(opp)
opp_out_b <- sum(opp$out, na.rm=TRUE)/1e9
opp_disc_w <- weighted.mean(opp$discount_pts, w = opp$out, na.rm=TRUE)

opp_top <- opp %>%
  arrange(px) %>%
  transmute(
    isin, moneda = cur_grp, vencimiento = mat,
    monto_m = out/1e6,
    cupon_pct = cpn,
    ytm_pct = ytm,
    precio = px,
    callable = call
  ) %>%
  head(12)
```

Hay `r opp_n` bonos bajo par. El monto vigente bajo par suma `r sprintf('$%.2f B', opp_out_b)` y el descuento promedio ponderado es `r sprintf('%.2f puntos', opp_disc_w)`.

```
p_opp <- plot_ly(
  opp,
  x = ~ttm_years,
  y = ~px,
  type = "scatter",
  mode = "markers",
  color = ~cur_grp,
  colors = cols_fx,
  marker = list(size = 9, opacity = 0.85),
  hovertemplate = "ISIN: %{customdata}<br>Años: %{x:.2f}<br>Precio:
%{y:.2f}<extra></extra>",
  customdata = ~isin
) %>%
  layout(
```

```
    title = list(text = "Bonos bajo par (precio limpio vs años al
vencimiento)", x = 0),
    xaxis = list(title = "Años al vencimiento"),
    yaxis = list(title = "Precio limpio")
  )

p_opp
fmt_date <- function(d) format(as.Date(d), "%Y-%m-%d")
fmt_m <- function(x) formatC(x, format="f", digits=0, big.mark=",")
fmt_pct <- function(x) paste0(formatC(x, format="f", digits=2), "%")
fmt_px <- function(x) formatC(x, format="f", digits=2)

rows <- paste0(
  "<tr>",
  "<td>", opp_top$isin, "</td>",
  "<td>", opp_top$moneda, "</td>",
  "<td style='text-align:right;'>", fmt_date(opp_top$vencimiento), "</td>",
  "<td style='text-align:right;'>", fmt_m(opp_top$monto_m), "</td>",
  "<td style='text-align:right;'>", fmt_pct(opp_top$cupon_pct), "</td>",
  "<td style='text-align:right;'>", fmt_pct(opp_top$ytm_pct), "</td>",
  "<td style='text-align:right;'>", fmt_px(opp_top$precio), "</td>",
  "<td style='text-align:right;'>", opp_top$callable, "</td>",
  "</tr>",
  collapse=""
)

cat("
<table class='table table-striped' style='max-width:980px;'>
  <thead>
    <tr>
      <th>ISIN</th>
      <th>DIVISA</th>
      <th style='text-align:right;'>VENCIMIENTO</th>
      <th style='text-align:right;'>MONTO VIGENTE USD-EQ (M)</th>
      <th style='text-align:right;'>CUPÓN</th>
      <th style='text-align:right;'>YTM</th>
      <th style='text-align:right;'>PRECIO</th>
      <th style='text-align:right;'>CALLABLE</th>
    </tr>
  </thead>
  <tbody>", rows, "</tbody>
</table>
")
```

Números que debe mostrar Sección 11

```
list(
  n_bonos_bajo_par = opp_n,
  monto_bajo_par_b = opp_out_b,
  descuento_prom_ponderado_pts = opp_disc_w
)
```

---

12. Sensibilidad del costo de refinanciamiento y T-lock

```
cheap <- curve %>%
```

```
  filter(cpn < 3, mat < as.Date("2032-01-01"))

N_refi <- sum(cheap$out, na.rm=TRUE)
c_old_w <- weighted.mean(cheap$cpn/100, w = cheap$out, na.rm=TRUE)

# Tasa de refinanciamiento: promedio ponderado de YTM de USD (puedes
cambiar a emisión 23-oct si así está tu referencia)
y_new <- weighted.mean(curve$ytm[curve$cur_grp=="USD"]/100, w =
curve$out[curve$cur_grp=="USD"], na.rm=TRUE)

delta_int_base_m <- N_refi * (y_new - c_old_w) / 1e6

shifts_bp <- c(-100, -50, 0, 50, 100, 150)
sens <- data.frame(shift_bp = shifts_bp) %>%
  mutate(
    y = y_new + shift_bp/10000,
    delta_int_m = N_refi * (y - c_old_w) / 1e6,
    cambio_vs_base_m = delta_int_m - delta_int_base_m
  )

pvbp_10y_per_1mm <- 8.0
dv01_usd_per_bp <- pvbp_10y_per_1mm * (N_refi/1e6)
```

El monto a refinanciar (cupón menor a 3% y vencimiento antes de 2032) es `r sprintf('$%.2f B', N_refi/1e9)`. Con tasa nueva `r sprintf('%.2f%%', 100*y_new)` y cupón viejo ponderado `r sprintf('%.2f%%', 100*c_old_w)`, el incremento anual aproximado es `r sprintf('$%.0f M', delta_int_base_m)`. El DV01 aproximado del T-lock 10 años sobre ese monto es `r sprintf('$%.1f por pb', dv01_usd_per_bp)`.

```
p_sens <- plot_ly(
  sens,
  x = ~shift_bp, y = ~delta_int_m,
  type = "scatter", mode = "lines+markers",
  line = list(color = "rgba(17,60,102,0.95)", width = 2),
  marker = list(size = 8),
  hovertemplate = "Shift: %{x} pb<br>Δ Interés: $%{y:.0f} M<extra></extra>"
) %>%
  layout(
    title = list(text = "Sensibilidad del costo anual de refinanciamiento",
x = 0),
    xaxis = list(title = "Cambio en la tasa de emisión (pb)"),
    yaxis = list(title = "Incremento anual (M USD)")
  )

p_sens
rows <- paste0(
  "<tr>",
  "<td style='text-align:right;'>", sens$shift_bp, "</td>",
  "<td style='text-align:right;'>", sprintf('%.2f%%', 100*sens$y), "</td>",
  "<td style='text-align:right;'>", sprintf('$%.0f', sens$delta_int_m),
"</td>",
  "<td style='text-align:right;'>", sprintf('$%.0f',
sens$cambio_vs_base_m), "</td>",
  "</tr>",
  collapse=""
)

cat("
```

```
<table class='table table-striped' style='max-width:720px;'>
  <thead>
    <tr>
      <th style='text-align:right;'>SHIFT (pb)</th>
      <th style='text-align:right;'>TASA NUEVA</th>
      <th style='text-align:right;'>Δ INTERÉS ANUAL (M)</th>
      <th style='text-align:right;'>CAMBIO VS BASE (M)</th>
    </tr>
  </thead>
  <tbody>", rows, "</tbody>
</table>
")
```

Números que debe mostrar Sección 12

```
list(
  monto_refi_b = N_refi/1e9,
  cupon_viejo_pct = 100*c_old_w,
  tasa nueva pct = 100*y new,
  incremento_anual_m = delta_int_base_m,
  dv01_tlock_usd_por_bp = dv01_usd_per_bp
)
```

---

13. Calificaciones y CDS

```
# Constantes externas permitidas, documentadas
# Fuente: LSEG Workspace, consultado al 29-oct-2026 (CDS) y 1-nov-2026
(ratings)
rating_text <- "AA · Aa2 · AA"
rating_foot <- "S&P · Moody's · Fitch"
cds_5y_pb <- 26
cds_foot <- "29-oct-2026"
```

La calificación es `r rating_text` y el CDS a 5 años es `r paste0(cds_5y_pb, ' pb')` (corte `r cds_foot`).

```
cat("<div class='kpi-grid'>")
cat(tile("Calificación", rating_text, rating_foot))
cat(tile("CDS 5 años", paste0(cds_5y_pb, " pb"), cds_foot))
cat("</div>")
```

Números que debe mostrar Sección 13

```
list(rating = rating_text, cds_5y_pb = cds_5y_pb)
```

---

14. Estructura de deuda y derivados 10-K/10-Q

```
# Constantes externas permitidas, documentadas
# Fuente: 10-K y 10-Q de Walmart (corte 31-jul-2026)
ccs_notional_b <- 5.1     # 10-Q: cross-currency swaps, nocional aproximado
en USD eq (B)
swap fixfloat b <- 4.8    # 10-K: swaps fijo a variable, nocional (B)
swap_fv_mtm_m <- -467     # 10-K/nota: valor razonable al 31-jul-2026 (M
USD)
```

Al 31-jul-2026, los cross-currency swaps suman `r sprintf('$%.1f B', ccs_notional_b)` (10-Q). Los swaps fijo a variable suman `r sprintf('$%.1f B', swap_fixfloat_b)` (10-K) y su valor razonable es `r sprintf('$%.0f M', swap_fv_mtm_m)`.

```
rows <- paste0(
  "<tr><td>Cross-currency swaps</td><td style='text-align:right;'>",
sprintf("$%.1f B", ccs_notional_b), "</td><td>10-Q (31-jul-
2026)</td></tr>",
  "<tr><td>Swaps fijo a variable</td><td style='text-align:right;'>",
sprintf("$%.1f B", swap_fixfloat_b), "</td><td>10-K (31-jul-
2026)</td></tr>",
  "<tr><td>Valor razonable swaps fijo a variable</td><td style='text-
align:right;'>", sprintf("$%.0f M", swap_fv_mtm_m), "</td><td>10-K (31-jul-
2026)</td></tr>"
)

cat("
<table class='table table-striped' style='max-width:760px;'>
  <thead>
    <tr>
      <th>INSTRUMENTO</th>
      <th style='text-align:right;'>MAGNITUD</th>
      <th>FUENTE</th>
    </tr>
  </thead>
  <tbody>", rows, "</tbody>
</table>
")
```

Números que debe mostrar Sección 14

```
list(
  ccs_notional_b = ccs_notional_b,
  swaps_fijo_variable_b = swap_fixfloat_b,
  swaps_fijo_variable_valor_razonable_m = swap_fv_mtm_m
)
```

---

17. Qué sigue

```
# Variables de cierre para vincular con el resto del dashboard
total_deuda_b <- sum(curve$out, na.rm=TRUE)/1e9
no_usd_b <- sum(curve$out[curve$cur_grp != "USD"], na.rm=TRUE)/1e9
no_usd_pct <- no_usd_b / total_deuda_b
```

Siguiente paso: cerrar el paquete de cobertura con un plan operativo y medible. Con una deuda total de `r sprintf('$%.2f B', total_deuda_b)` y deuda no USD de `r sprintf('$%.2f B', no_usd_b)` (`r sprintf('%.1f%%', 100*no_usd_pct)`), la prioridad es cuantificar DV01, definir monto y fecha objetivo de cobertura, y documentar el tratamiento contable.

```
cat("
<ul>
  <li>Definir el monto a cubrir por tasa y el horizonte exacto del
refinanciamiento, con un DV01 objetivo y un rango de tolerancia.</li>
```

```
  <li>Separar riesgo de tasa base y riesgo de spread, y decidir el
instrumento: T-lock, swap con inicio futuro o swaption según certeza de
ejecución.</li>
  <li>Para divisa, documentar el objetivo de cross-currency swap y el
criterio de efectividad.</li>
  <li>Integrar los supuestos de volatilidad y la valuación del convertible
con sensibilidad y control de supuestos.</li>
</ul>
")
```

Números que debe mostrar Sección 17

```
list(
  deuda_total_b = total_deuda_b,
  deuda_no_usd_b = no_usd_b,
  deuda_no_usd_pct = no_usd_pct
)
```

Si pegas aquí el contenido de **Secciones_7_a_14_referencia.txt**, en la siguiente iteración ajusto:

- la Sección 8 para que use exactamente el mismo método (si no era ytm−zytm - z),
- la Sección 12 para que use la misma "tasa nueva" que tu referencia (emisión 23-oct vs curva USD, o algo distinto),
- y cualquier filtro adicional (callable, tramos, exclusiones) que esté en su código.

[DONE]

1/11/2026 - 4:59:29 p.m.

Gracias por la ayuda, me sirvió mucho. Con esto cierro la conversación.



ChatGPT (GPT 5.2) •1/11/2026 - 4:59:29 p.m.

De acuerdo. Me alegra que te haya servido.

Si después necesitan una última pasada de consistencia antes de entregar, puedo ayudarte a:

- correr un checklist de que todos los textos usan inline `r ...` y no números fijos,
- validar que los filtros (yield y px) dejan 53 emisiones,
- y asegurar que los KPIs de portada amarren con las secciones (deuda total, no USD, cupón convertible, volatilidad a 6 años).

[DONE]
